Introduction

AUDK stands for ABOV Unified Development Kit and is a reference software package that makes software development easier. AUDK provides users with insight into the integrated operations of the APIs provided by ABOV, and it saves the effort, time, and cost for their system development.

AUDK32 is a dedicated SDK for ABOV 32bit devices incorporating the ARM Cortex-M core. It provides development environment, configuration system, unified API code and reference examples.

This document describes how to configure and set up the development environment in working with AUDK32.

Reference document

The following documents can be downloaded from the ABOV website www.abovsemi.com.

- STD10-HB_AUDK32 CLI Example_Eng.pdf
- AUDK32 Release Note
- AUDK32 API Reference in Windows help (*.chm) format
- Starter Kit documents of ABOV 32bit MCUs

Contents

Introduction 1
Reference document 1
1. AUDK32 overview 7
1.1 Introduction 7
1.2 ABOV boards 8
1.3 Limitations 8
1.4 License 8
2. System requirements 9
2.1 Operating system requirements 9
2.2 IDE Software requirements 9
2.3 Starter Kit board 9
2.4 Additional software 9
2.5 Micro-B 5-pin USB cable 10
3. Installation of AUDK32 11
3.1 Naming of AUDK32 deliverables 11
3.2 Download AUDK32 12
3.3 Directory structure of AUDK32 12
4. Configuration of FPA 13
4.1 Description 13
4.2 FPA files 13
4.3 FLM in KEIL uVision 5 14
4.3.1 Install FLM 14
4.3.2 Selecting the target in KEIL project 15
4.3.3 FLM options in KEIL project 16
4.4 Flashloader in IAR Embedded Workbench 21
4.4.1 Flashloader by selection target in workspace 22
5. AUDK32 in KEIL uVision5 23
5.1 KEIL project 23
5.2 Configuration Wizard annotations 24
5.3 Select target 25
5.4 abov_config.h 26
5.4.1 Select Starter Kit 26
5.4.2 Select BARE 26
5.4.3 Config debug channel 27
5.5 abov_example_config.h 28
5.5.1 Example selection in Starter Kit 28
5.5.2 Multiple examples selection in Bare board 29
5.5.3 Error messages 30
5.6 Scatter loader (*.sct) file 31
5.7 Building and run example 34
5.8 Debugging with A-Link 36
6. AUDK32 in IAR Embedded Workbench 39
6.1 IAR Workspace 40
6.2 Select target 41
6.3 abov_config.h 42
6.3.1 Select Starter Kit 42
6.3.2 Select BARE 43
6.3.3 Config debug channel 44

6.4 abov_example_config.h 45
6.4.1 Example selection of Starter Kit 45
6.4.2 Multiple examples selection in Bare board 46
6.5 Linker (*.icf) file 47
6.6 Build and run example 49
6.7 Debugging with A-Link 51
7. AUDK32 in Eclipse Embedded C/C++ 54
7.1 Eclipse Workspace 55
7.1.1 Open workspace and project 55
7.2 Select target 57
7.3 abov_config.h 58
7.3.1 CMSIS Configuration Wizard 59
7.3.2 Select Starter Kit 60
7.3.3 Select BARE 61
7.3.4 Config debug channel 62
7.4 abov_example_config.h 63
7.4.1 Example selection of Starter Kit 63
7.4.2 Multiple examples selection in Bare board 64
7.5 Linker (*.ld) file 65
7.6 Build and run example 66
7.6.1 Build target 67
7.6.2 Program binary into device 68
7.7 Debugging with A-Link 70
7.7.1 Source level debugging 70
7.7.2 Browsing peripherals registers 71
8. Serial debug configuration 72
8.1 Default serial debug channel of Starter Kit 72
8.2 Changing serial debug channel 74
8.2.1 Shared interrupt vector of serial debug channels 75
8.3 Debug message in terminal 76
8.4 Setup the new line in terminal 77
9. Appendix 78
9.1 Eclipse IDE for Embedded C/C++ Developers 78
9.1.1 Download program and plug-in 78
9.1.1.1 Eclipse IDE for Embedded C/C++ Developers 78
9.1.1.2 GNU ARM cross toolchain 78
9.1.1.3 CMSIS Pack plug-in 78
9.1.1.4 xPack Windows build tools 78
9.1.2 Installation 80
9.1.2.1 Eclipse IDE for Embedded C/C++ Developers 80
9.1.2.2 GNU ARM cross toolchain 80
9.1.2.3 CMSIS Pack plug-in 81
9.1.2.4 xPack Windows build tools 84
9.1.3 Setup workspace 85
9.1.3.1 Open workspace 85
9.1.3.2 Select project 86
9.1.3.3 Check project 87
9.1.3.4 Toolchain configuration 88
9.1.3.5 xPack Windows build tools configuration 91
Revision history 93

List of tables

Table 1. Definition of ABOV boards 8
Table 2. Naming of AUDK32 deliverables 11
Table 3. Versioning format: X, Y, Z 11
Table 4. Directory description of AUDK32 12
Table 5. Directory description of AUDK32 13
Table 6. FLM file in AUDK32 14
Table 7. Flashloader files in AUDK32 21
Table 8. Scatter files 31

List of figures

Figure 1. Software architecture of AUDK32 7
Figure 2. Micro-B 5-pin USB cable 10
Figure 3. Directory configuration of AUDK32 S/W package 12
Figure 4. Location of FPA files in AUDK32 13
Figure 5. Copy and paste FLM files 14
Figure 6. Change target device in ‘Select Target’ 15
Figure 7. Click ‘Target Options...’ icon 16
Figure 8. ‘Debug’ tab in ‘Options for Target…’ window 16
Figure 9. ‘Settings’ button on ‘Debug’ tab 17
Figure 10. ‘Flash Download’ tab in ‘CMSIS-DAP Cortex-M Target Driver Setup’ window 17
Figure 11. ‘Add’ button in ‘Flash Download’ 18
Figure 12. Selection of FLM files in ‘Add Flash Programming Algorithm’ window 18
Figure 13. FLM register for target device in ‘Programming Algorithm’ 19
Figure 14. Auto-update of ‘RAM for Algorithm’ by ‘Select Target’ settings 20
Figure 15. IAR flashloader file location of A31G22x 21
Figure 16. Auto-update of flashloader by selected target 22
Figure 17. Configuration of files in ‘Config’ group of KEIL IDE 23
Figure 18. Configuration Wizard annotations in AUDK32 24
Figure 19. Changing target device using ‘Select Target’ function in KEIL IDE 25
Figure 20. Selection of Starter Kit in KEIL IDE 26
Figure 21. Selection of Bare board in KEIL IDE 26
Figure 22. Config debug channel in KEIL IDE 27
Figure 23. GPIO_LED_BLINKING in case of Starter Kit board 28
Figure 24. Selection of multiple examples in KEIL IDE 29
Figure 25. Build error message with improper example 30
Figure 26. Build error message with insufficient resources 30
Figure 27. Configuration of memory information by ac5_a31xxxx.sct 31
Figure 28. ac5_a31xxxx.sct scatter file of Linker tab 32
Figure 29. Auto-update memory configuration from selected scatter file 33
Figure 30. Selection of ac5_a31g21x.sct in linker tab for A31G21x 33
Figure 31. gpio_led_blinking.c example in KEIL IDE 34
Figure 32. Build target in KEIL IDE 34
Figure 33. Download image to flash memory in KEIL IDE 35
Figure 34. State of gpio_led_blinking example 35
Figure 35. Debug session in KEIL IDE 36
Figure 36. Break point in KEIL IDE 37
Figure 37. System viewer in KEIL IDE 38
Figure 38. Configuration files in ‘Config’ group of the IAR IDE 40
Figure 39. Changing target Sub-Family using ‘Select Target’ function in IAR IDE 41
Figure 40. Selection of Starter Kit in IAR IDE 42
Figure 41. Selection of Bare board in IAR IDE 43
Figure 42. Configure Debug channel 44
Figure 43. GPIO_LED_BLINKING in case of Starter Kit board 45
Figure 44. Selection of multiple examples 46
Figure 45. Configuration of memory in a31xxxx.icf by text editor 47
Figure 46. Configuration of memory in a31xxxx.icf by linker option 48
Figure 47. gpio_led_blinking.c example in IAR IDE 49
Figure 48. Build target in IAR IDE 49

Figure 49. Download image to flash memory in IAR IDE 50
Figure 50. State of gpio_led_blinking example 50
Figure 51. Debug session of IAR IDE 51
Figure 52. Break point in IAR IDE 52
Figure 53. System viewer in IAR IDE 53
Figure 54. Bare workspace and bare project in Eclipse IDE 55
Figure 55. Changing target Sub-Family 57
Figure 56. CMSIS Configuration Wizard 59
Figure 57. Selection of Starter Kit in Eclipse IDE 60
Figure 58. Selection of Bare board in Eclipse IDE 61
Figure 59. Configure Debug channel 62
Figure 60. GPIO_LED_BLINKING in case of Starter Kit board 63
Figure 61. Selection of multiple examples 64
Figure 62. Configuration of memory in gcc_a31xxxx.ld 65
Figure 63. gpio_led_blinking.c example in Eclipse IDE 66
Figure 64. Target list in Eclipse IDE 67
Figure 65. Build target in Eclipse IDE 67
Figure 66. How to trigger GDB in Eclipse IDE 68
Figure 67. Debug session information 69
Figure 68. State of gpio_led_blinking example 69
Figure 69. Source level debugging in Eclipse IDE 70
Figure 70. Browsing peripheral registers in Eclipse IDE 71
Figure 71. Serial debug interface of A31G22x Starter Kit 72
Figure 72. Serial debug configuration in AUDK32 73
Figure 73. H/W connection to change debug channel 74
Figure 74. Modify ‘abov_config.h’ to switch a new debug channel 74
Figure 75. To edit serial debug configuration file when sharing interrupt vector 75
Figure 76. Result of AUDK32 debug log API 76
Figure 77. Serial port configuration in terminal program 76
Figure 78. Configuration of new line in PC terminal program 77
Figure 79. Management of access to GNU toolchain 80
Figure 80. STEP1. Process to register CMSIS Pack plug-in to be installed 81
Figure 81. STEP2. Select target plug-in to install 82
Figure 82. STEP3. Complete license agreement and trust artifacts 83
Figure 83. STEP4. Restart Eclipse IDE 83
Figure 84. STEP1. Run Eclipse and register workspace 85
Figure 85. STEP2. Register project directory 86
Figure 86. STEP3. Check project 87
Figure 87. STEP4. Open Property dialog 88
Figure 88. STEP5. Configure toolchain path within project scope 89
Figure 89. STEP6. Add toolchain path in Project Property window 90
Figure 90. STEP7. Configure Build tools path within project scope 91
Figure 91. STEP8. Add ‘xPack Windows build tools’ path in ‘Build Tools Path’ window 92
Figure 92. STEP9. Check path of Tool chain and ‘xPack Windows build tools’ 92

1. AUDK32 overview

1.1 Introduction

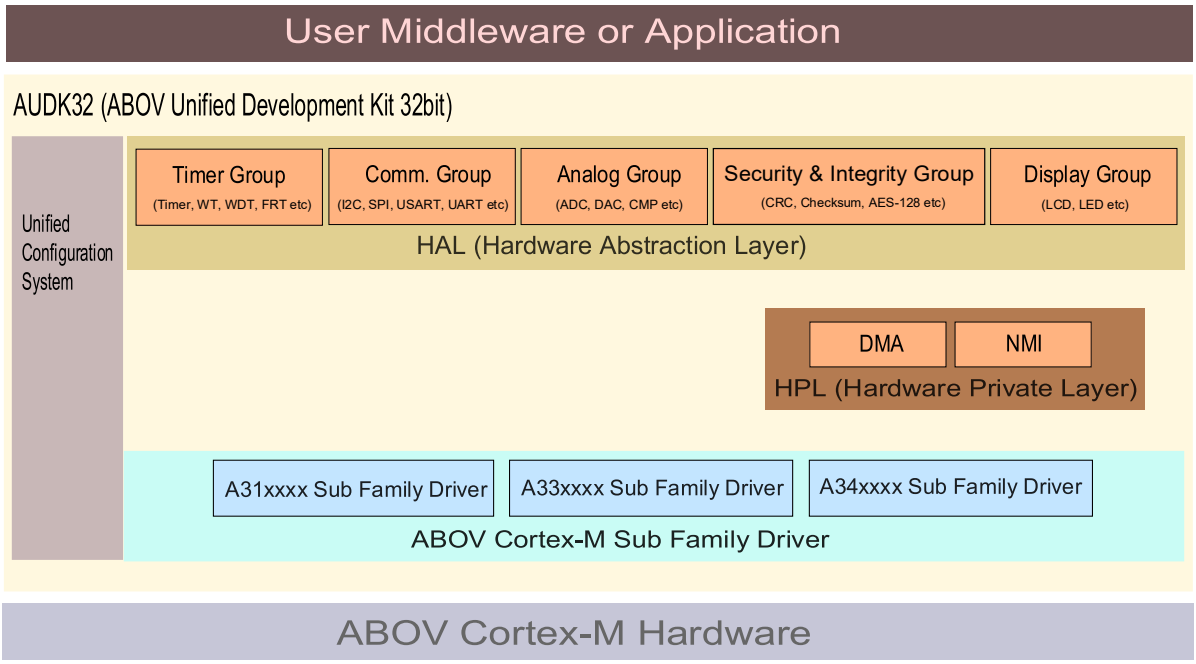
Usually, developers want to quickly find the MCUs that meet their requirements (Core architecture, features, memory size, performance, etc.). They encounter the requirement to either migrate their existing designs to different target devices or create new applications for their new project using those new target devices.

AUDK32 is an integrated software package that minimizes the time and cost in developing software with the target device and maximizes reusability of the software.

The standard Hardware Abstract Layer (HAL) in AUDK32 is a unified driver which is designed to cover MCU variants of ABOV device family with a single API set. In addition, it provides the conceptual MCU operation and a uniformed way to develop software. Therefore, developers can migrate their software easily to other ABOV 32bit MCUs with HAL saving time and cost for the project development.

Figure 1 shows the software architecture of the AUDK32.

Figure 1. Software architecture of AUDK32



1.2 ABOV boards

Example code provided in AUDK32 is related to the hardware platform (i.e., the target board) on which it operates.

Table 1 shows information about the boards commonly provided by ABOV.

Table 1. Definition of ABOV boards

Board	Description
Bare	• Bare doesn't mean an actual board but virtual one. • Most peripherals on ABOV 32bit devices are commonly supported. The example selected for a supported peripheral to test can be run or tested on any platform. In this case, we don't need a specific board but common board. This is why bare is common virtual platform.
Starter Kit	Starter Kit has LEDs and some buttons only.
STKS	• STKS means a Starter Kit Shield Board that can be extended to connect to the Starter Kit's I/O pin connector. • As STKS does not have an MCU, it must be connected to STK. • STKS typically deploys functions such as IR and display (7-segment or LCD) controlled by StarterKit.

1.3 Limitations

- If users select a peripheral example that is not supported on the target device, the users may see a warning message. To avoid displaying warning messages, refer to the user's manual to check which peripherals are supported on the target devices before selecting an example to test.
- Some peripherals are scheduled to be supported soon.
- High level software components such as middleware or real-time OS will be available in the near future.

1.4 License

AUDK32 is licensed under terms that are found in the LICENSE file located at Document directory.

If this file is delivered or shared without applicable license terms, the terms of the BSD-3-Clause license shall be applied.

- Reference URL: https://opensource.org/licenses/BSD-3-Clause

2. System requirements

2.1 Operating system requirements

AUDK32 requires the following operating system to run IDE:

- Windows 64bit series supported by KEIL uVision5 and IAR Embedded Workbench 8.50

2.2 IDE Software requirements

AUDK32 requires software integrated development tools introduced below, which are based on the ARM Cortex-M core:

KEIL uVision5

- URL: https://www2.keil.com/mdk5/uvision/

IAR Embedded Workbench 8.50(1)

- URL: https://www.iar.com/products/architectures/arm/iar-build-tools-for-arm/

NOTE:

1. IAR Embedded Workbench 8.5 or higher version is recommended since the compatibility to lower version is not guaranteed.

2.3 Starter Kit board

AUDK32 software supports the ABOV 32bit Starter Kit boards.

Refer to the URL below for more information:

- URL: https://www.abov.co.kr/en/tools_support/evaluation_tools.php?category=32bit_starter_kit

2.4 Additional software

A terminal PC program should be used for transmitting and receiving data through serial debug channels of the Starter Kit.

The terminal program used in this document is as follows:

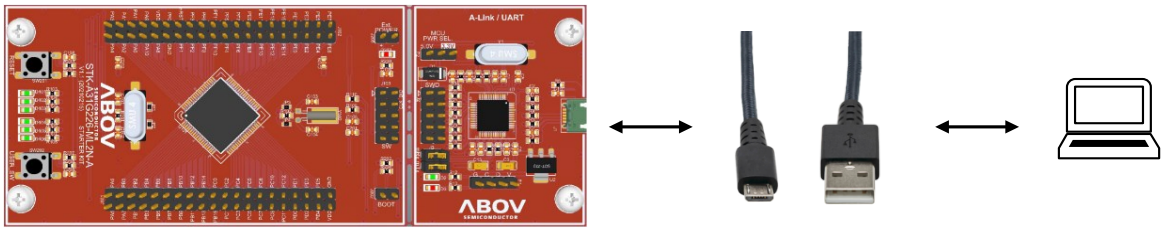
TeraTerm

- URL: https://ttssh2.osdn.jp/index.html.en

2.5 Micro-B 5-pin USB cable

To supply power to the Starter Kit, use serial communication function and debug a software via A-Link debugger, a Micro-B 5-pin USB cable must be connected to PC and the Starter Kit as shown in Figure 2.

Figure 2. Micro-B 5-pin USB cable



Starter Kit Micro-B 5-pin USB cable PC

3. Installation of AUDK32

3.1 Naming of AUDK32 deliverables

AUDK32 is named in the ABOV format shown in Table 2.

[AUDK32]_[Family or Sub-Family]-[SDK Versioning]

Example) AUDK32_A31G22x-1.0.0

Example) AUDK32_A31xxxx-1.0.0

Table 2. Naming of AUDK32 deliverables

Format	Description
AUDK32	ABOV Unified Development Kit for 32bit MCU
Family	'Family' is a group of ABOV 32bit MCU devices that have the same ARM Cortex-M core. • A31xxxx: Includes target devices with prefix 'A31-'. (A31Gxxx, A31Mxxx, etc.) • A33xxxx: Includes target devices with prefix 'A33-'. (A33Gxxx, A33Mxxx, etc.) • A34xxxx: Includes target devices with prefix 'A34-'. (A34Mxxx, etc.)
Sub-Family	'Sub-Family' is a MCU product line belongs to an ABOV 32bit MCU family. Each Sub-Family includes a variety of MCU portfolios and device variants, followed by 'x's after its name. • A31xxxx: Includes target devices with prefix 'A31-'. (A31G2xx, A31G3xx, A31M2xx, etc.) • A31Gxxx: Includes target devices with prefix 'A31G-'. (A31G2xx, A31G3xx, etc.) • A31G22x: Includes target devices with prefix 'A31G22-'. (A31G22x)
SDK versioning	The AUDK32 complies with the semantic versioning specification 2.0.0. • URL: https://semver.org/ For more information on the versioning format, refer to Table 3.

Table 3. Versioning format: X, Y, Z

Version number	Description
X	Generation number of main AUDK Big change on build, configuration environment or S/W architecture
Y	Big change number of main AUDK New chipset, new feature or minor change on S/W architecture
Z	Bug fix number of main AUDK

3.2 Download AUDK32

The AUDK32 package can be downloaded from the ABOV website shown below:

- URL: https://www.abov.co.kr/kr/products/product_documentation.php?category=32bit_mcu&device=&type=current

3.3 Directory structure of AUDK32

After downloading and decompressing the AUDK32 package, you can find out the directory structure shown in Figure 3.

Figure 3 and Table 4 show and describe the directory configuration of the AUDK32_A31Gxxx-1.0.0 package, respectively.

Figure 3. Directory configuration of AUDK32 S/W package

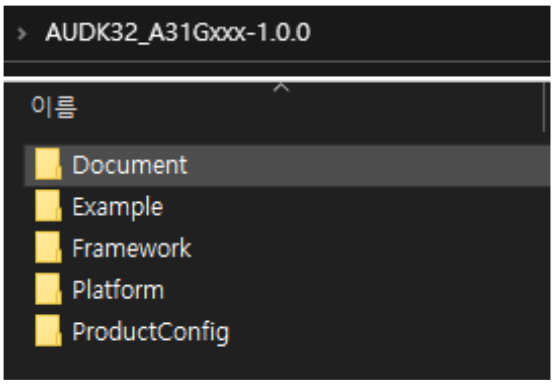


Table 4. Directory description of AUDK32

Directory	Description
Document	It contains reference documents for AUDK32, example code and AUDK32 API including license and release note as well. There is the reference of AUDK32 API in Windows help file format at Doxygen directory.
Example	It contains example projects and code files for verifying each peripheral module with the predefined scenario. AUDK32 incorporates KEIL and IAR IDE build environment. Therefore, KEIL and IAR project files are available here.
Framework	It contains CMSIS framework and ABOV utility such as debug APIs.
Platform	It contains standard HAL currently and will be extended to support high level software components in the near future.
ProductConfig	It contains configuration components of target device, board and example to test, and auxiliary files for FPA and debug system as below. • FPA for KEIL or IAR • SVD/SFR of target platform to debug register inside of MCU.

4. Configuration of FPA

4.1 Description

Flash Programming Algorithm (FPA) is a binary file which implements the algorithm to download a user image or data into internal flash area and verify the downloading.

ABOV Starter Kit includes A-Link debugger and supports CMSIS-DAP interface specification. Through this interface, FPA can be downloaded to RAM of the target board or MCU, and then ready for flashing the user image or data.

The user image is downloaded to code flash memory or the user data is downloaded to data flash memory, and then the downloaded user image or data is verified.

For this reason, the development environment must support FPA.

The following sections describe how to set up FPA in KEIL or IAR of AUDK32 and take A31G22x as a representative device for your understanding.

4.2 FPA files

The FPA files are provided for each [Sub-Family] and located in the '~/ProductConfig' directory. For example, the FPA files of A31G22x can be found in the '~/ProductConfig/A31G22x/FPA' directory, as shown in Figure 4.

Figure 4. Location of FPA files in AUDK32



The FPA files provided for each Sub-Family are different according to the compile environment, as shown in Table 5.

Section 4.3 and section 4.4 describe how to set FPA up in KEIL and IAR compiler environment.

Table 5. Directory description of AUDK32

Directory	Development environment
Flashloader	IAR Embedded Workbench
FLM	KEIL uVision5

4.3 FLM in KEIL uVision 5

The FPA files of the KEIL uVision 5 are located in the '~/ProductConfig/A31G22x/FPA/FLM' directory. They comply the naming format shown in Table 6, for each Sub-Family.

Table 6. FLM file in AUDK32

FLM file name	FLM target
[Sub-Family]_code.flm	Code flash memory of Sub-Family
[Sub-Family]_data.flm	Data flash memory of Sub-Family

The following sections describe the preparation before using FPA and how to set up FPA KEIL compile environment.

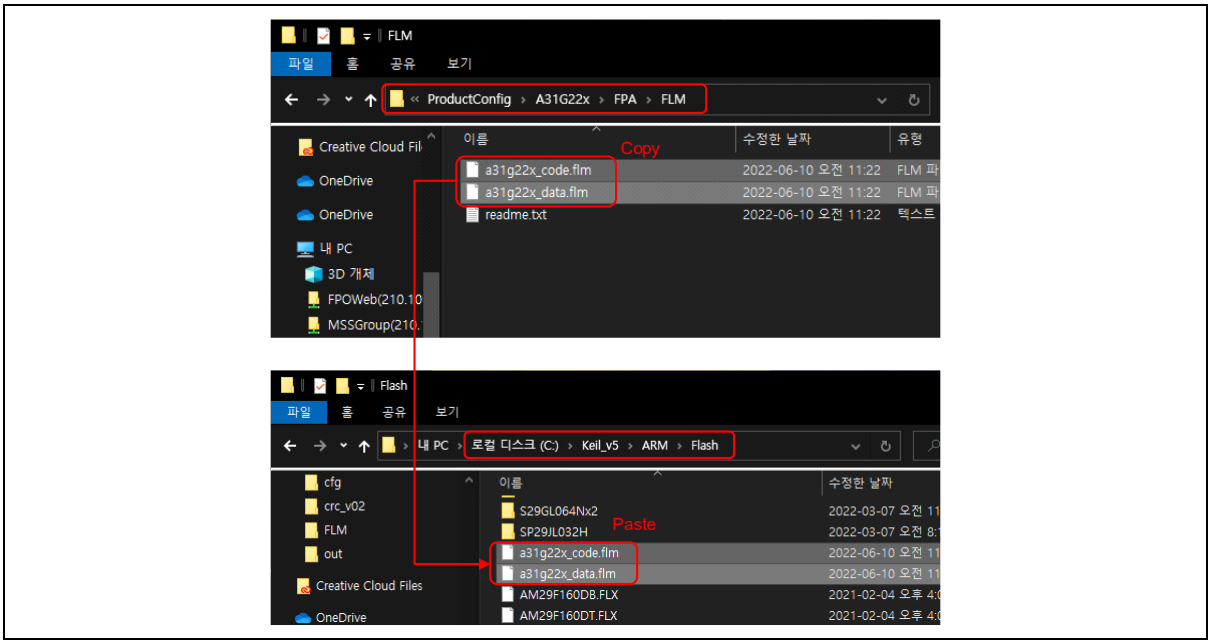
4.3.1 Install FLM

KEIL uVision5 expects that FPA files are located at KEIL installation directory.
Therefore, whenever FPA files provided by AUDK32 are updated, user must update FPA files at KEIL installation directory with them.

You can set up FLM files by following the procedure below. In this section, A31G22x is used as the target device to describe the following procedure:

1. Copy the 'a31g22x_code.flm' and 'a31g22x_data.flm' files from the '~/ProductConfig/A31G22x/FPA/FLM' directory path.
2. Paste the copied files under the 'C:\Keil_v5\ARM\Flash' directory where uVision5 is installed.

Figure 5. Copy and paste FLM files



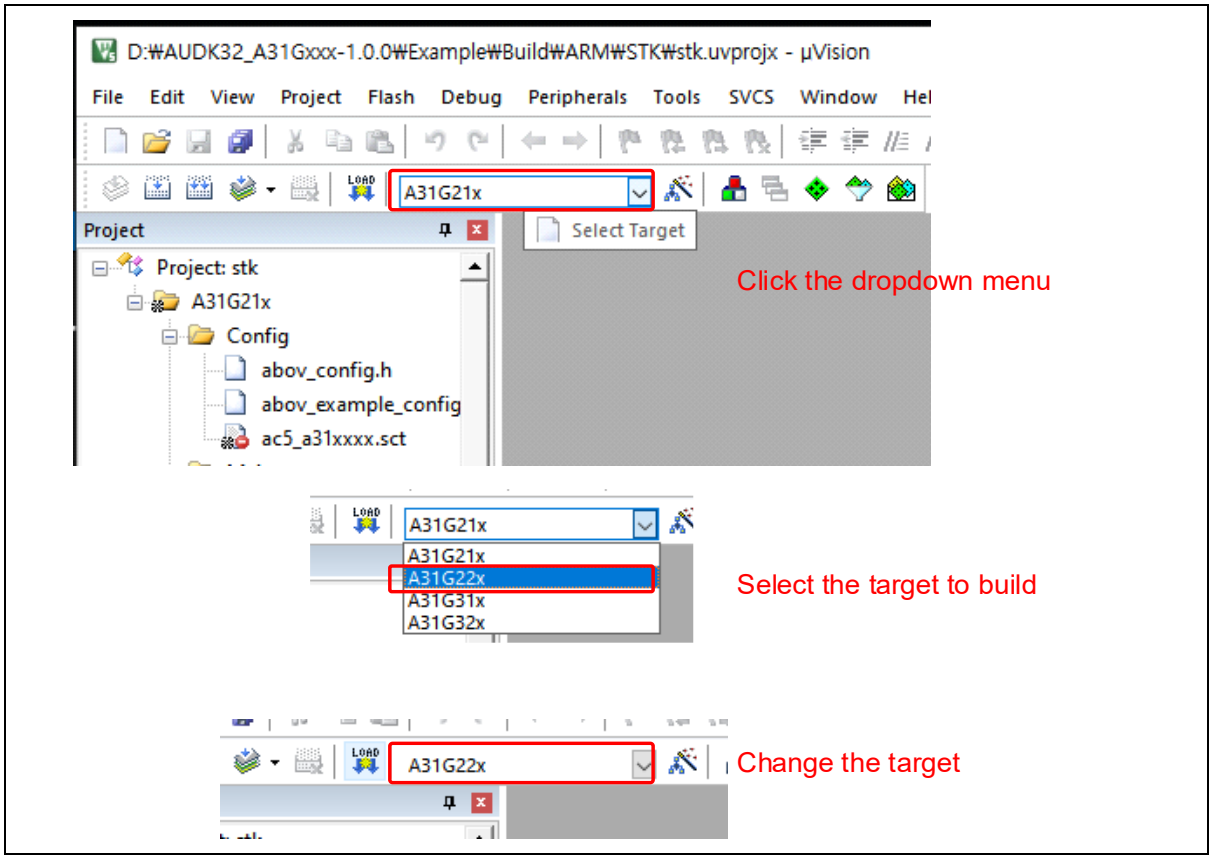
4.3.2 Selecting the target in KEIL project

The single KEIL uVision project file in AUDK32 supports multiple MCU Sub-Families. However, FLM is provided for each Sub-Family.

Therefore, the target must be selected before using a certain FLM file.

The 'Select Target' of KEIL uVision project is already set to the default value 'A31G21x'. If you want to change the target, select another Sub-Family from the drop-down menu.

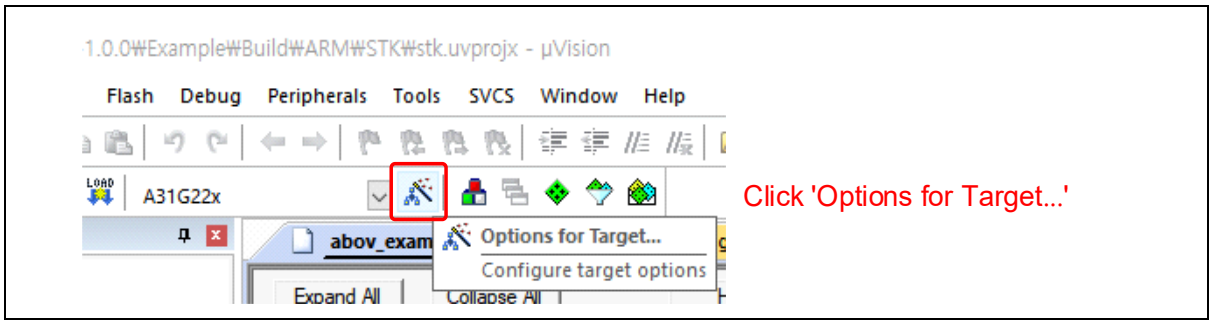
Figure 6. Change target device in 'Select Target'



4.3.3 FLM options in KEIL project

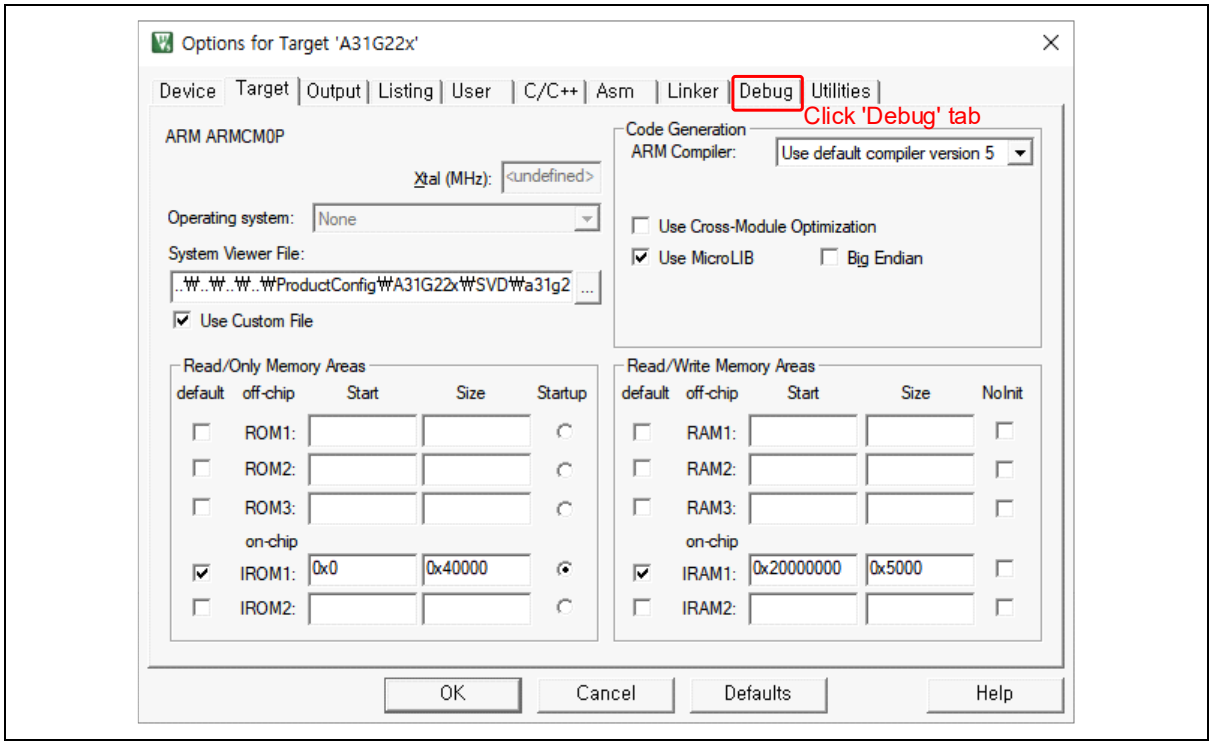
Click the 'Option for Target' icon for debug configuration to link FLM with A-Link debugger after the target selection.

Figure 7. Click 'Target Options...' icon



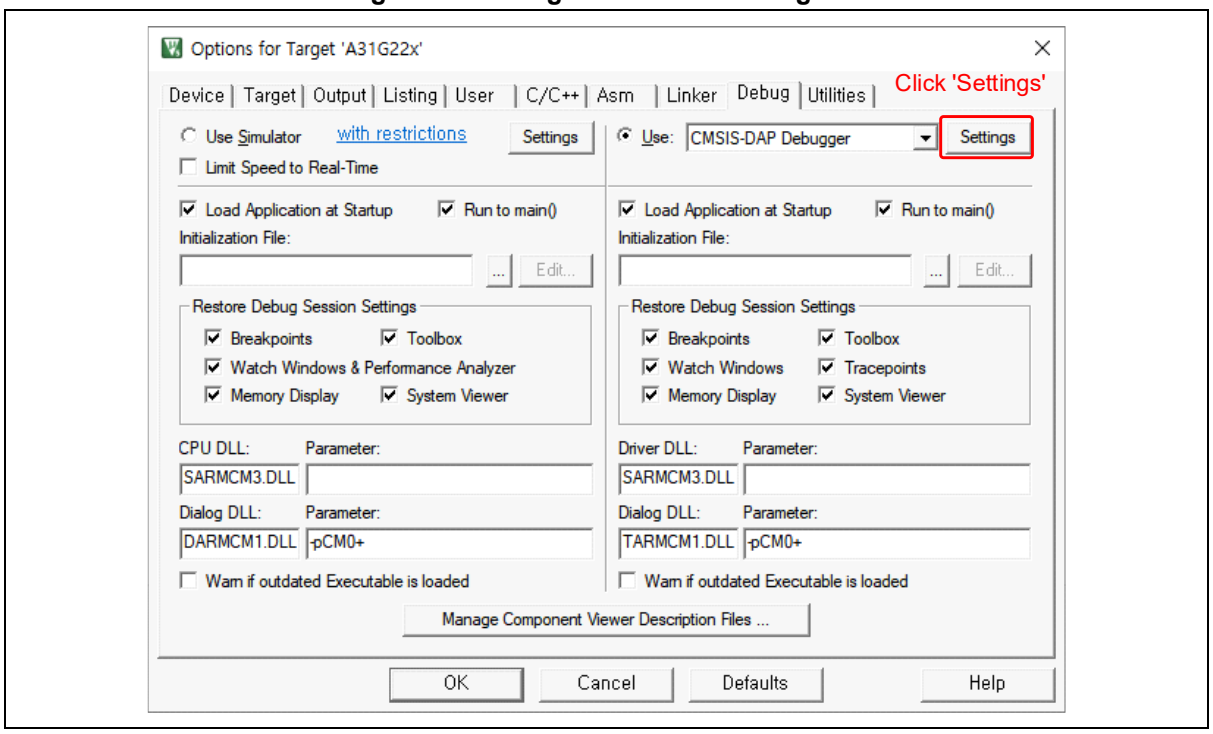
When the 'Options for Target ...' window appears, click the 'Debug' tab.

Figure 8. 'Debug' tab in 'Options for Target…' window



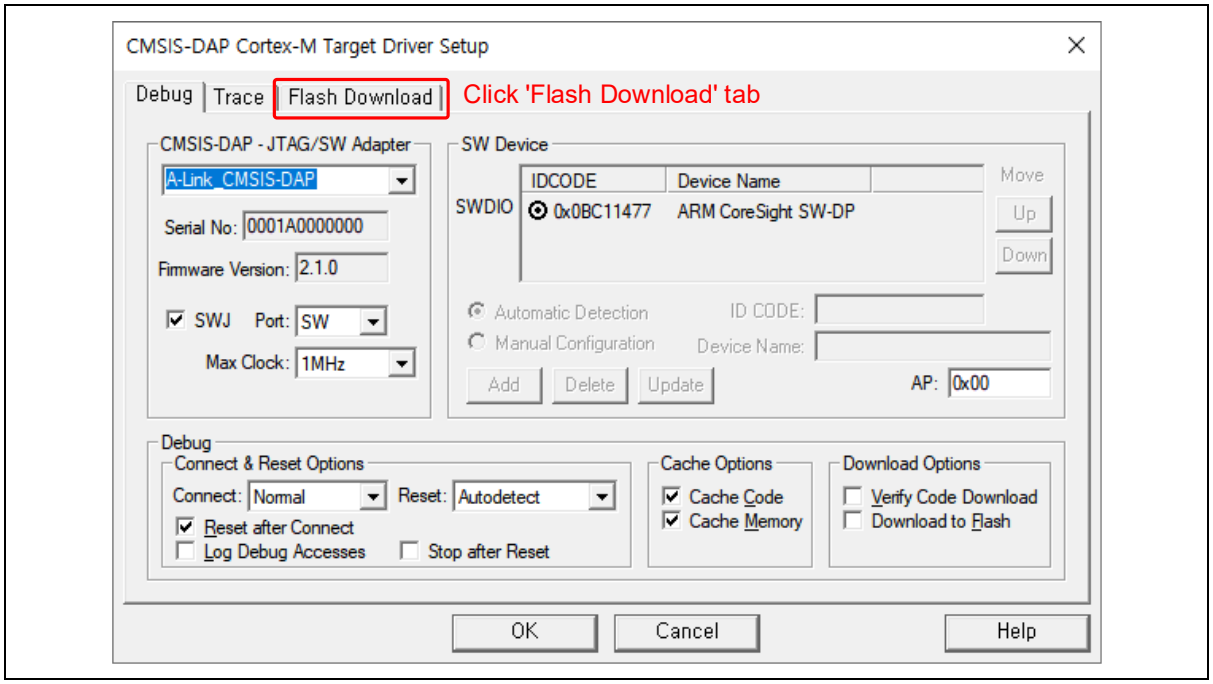
Click the ‘Settings’ button on the Debug tab.

Figure 9. ‘Settings’ button on ‘Debug’ tab



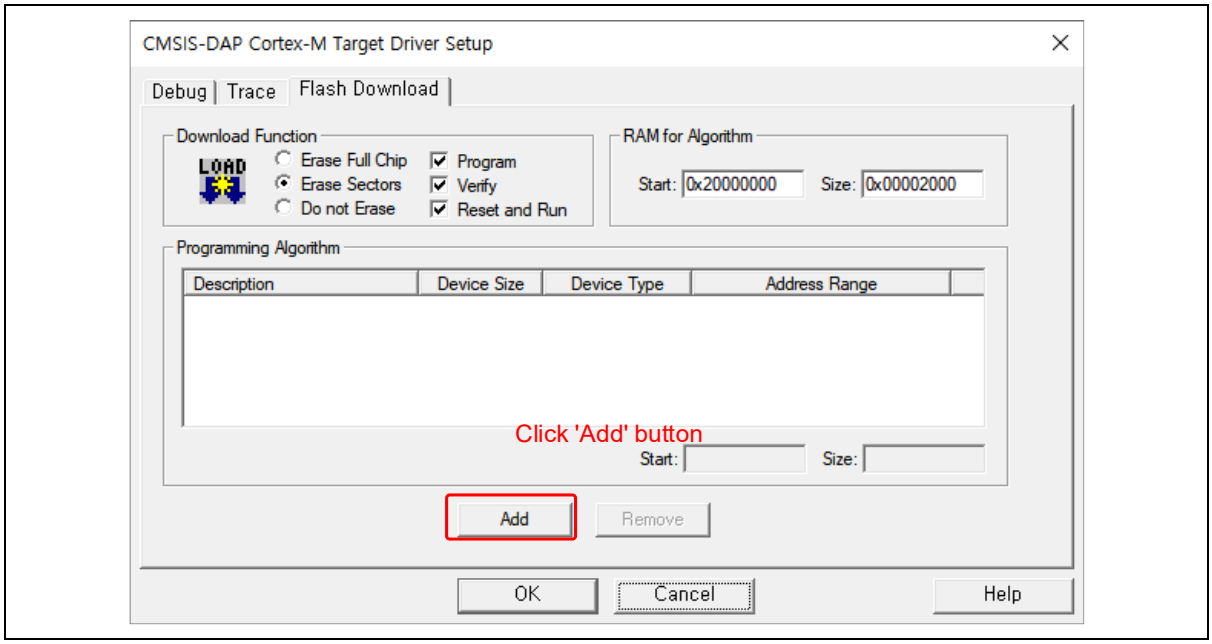
When the ‘CMSIS-DAP Cortex-M Target Driver Setup’ window pops up, click the ‘Flash Download’ tab.

Figure 10. ‘Flash Download’ tab in ‘CMSIS-DAP Cortex-M Target Driver Setup’ window



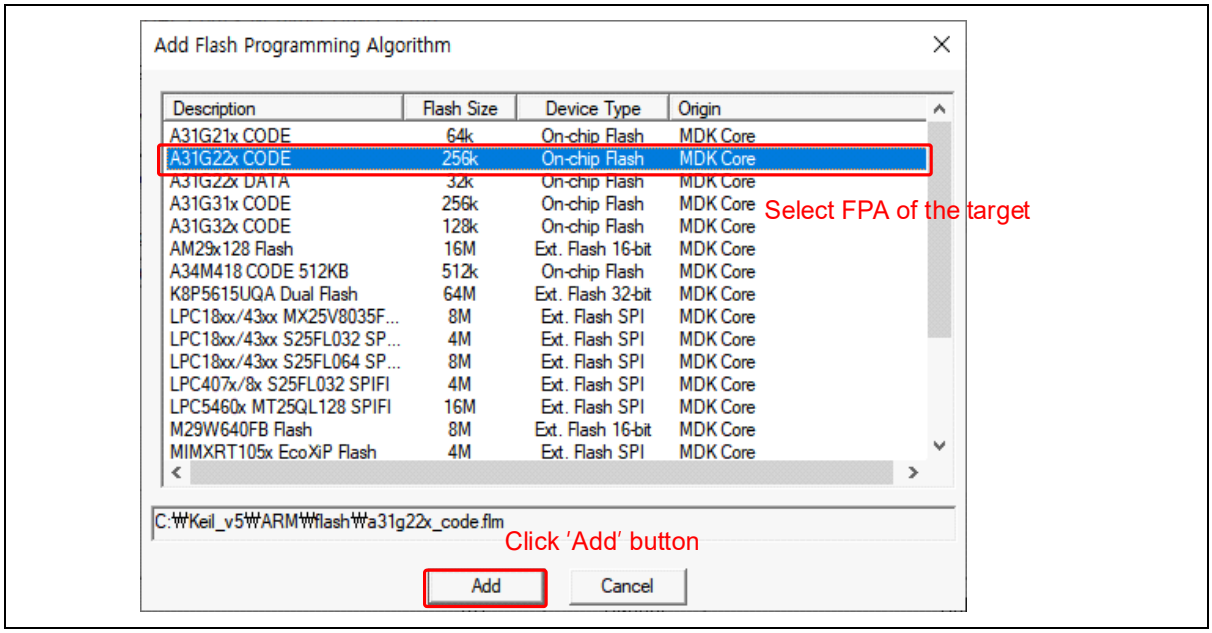
On the ‘Flash Download’ tab, click the ‘Add’ button at the bottom of the ‘Programming Algorithm’ to browse the FLM file copied before.

Figure 11. ‘Add’ button in ‘Flash Download’



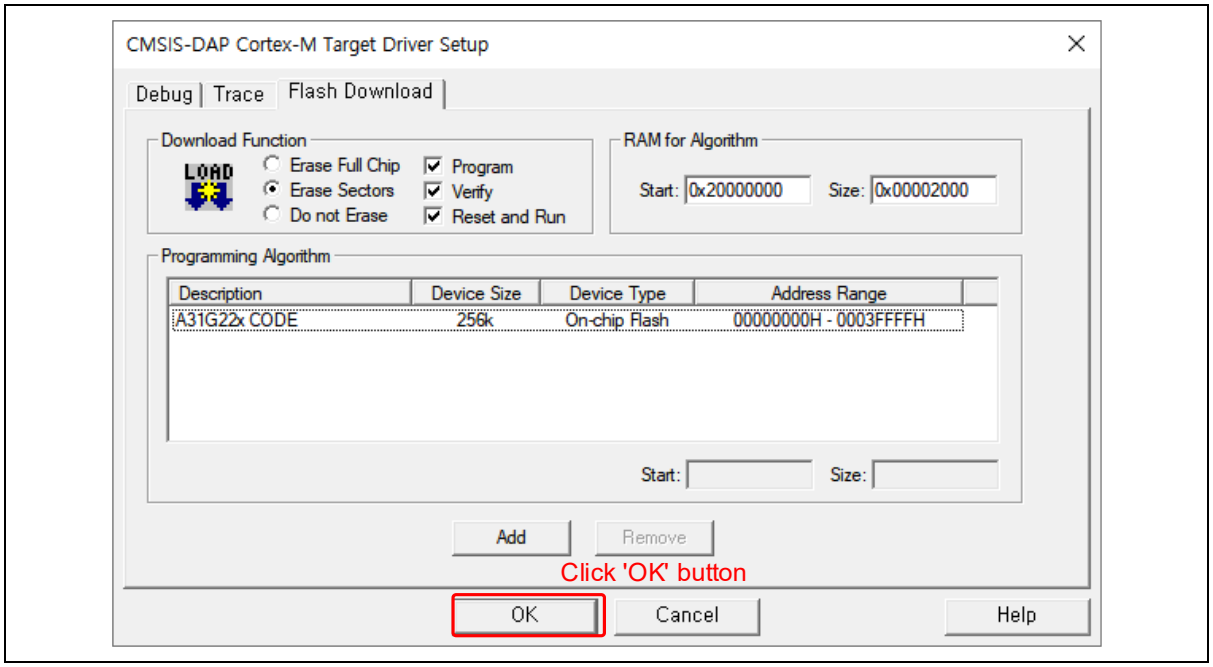
‘Add Flash Programming Algorithm’ window shows up and select the FPA files of the target that you set in the previous section, then click the ‘Add’ button.

Figure 12. Selection of FLM files in ‘Add Flash Programming Algorithm’ window



As see Figure 13 below, move your cursor to the ‘Programming Algorithm’ item and check if the information of the file selected is correct or not. Then click the ‘OK’ button to close the window.

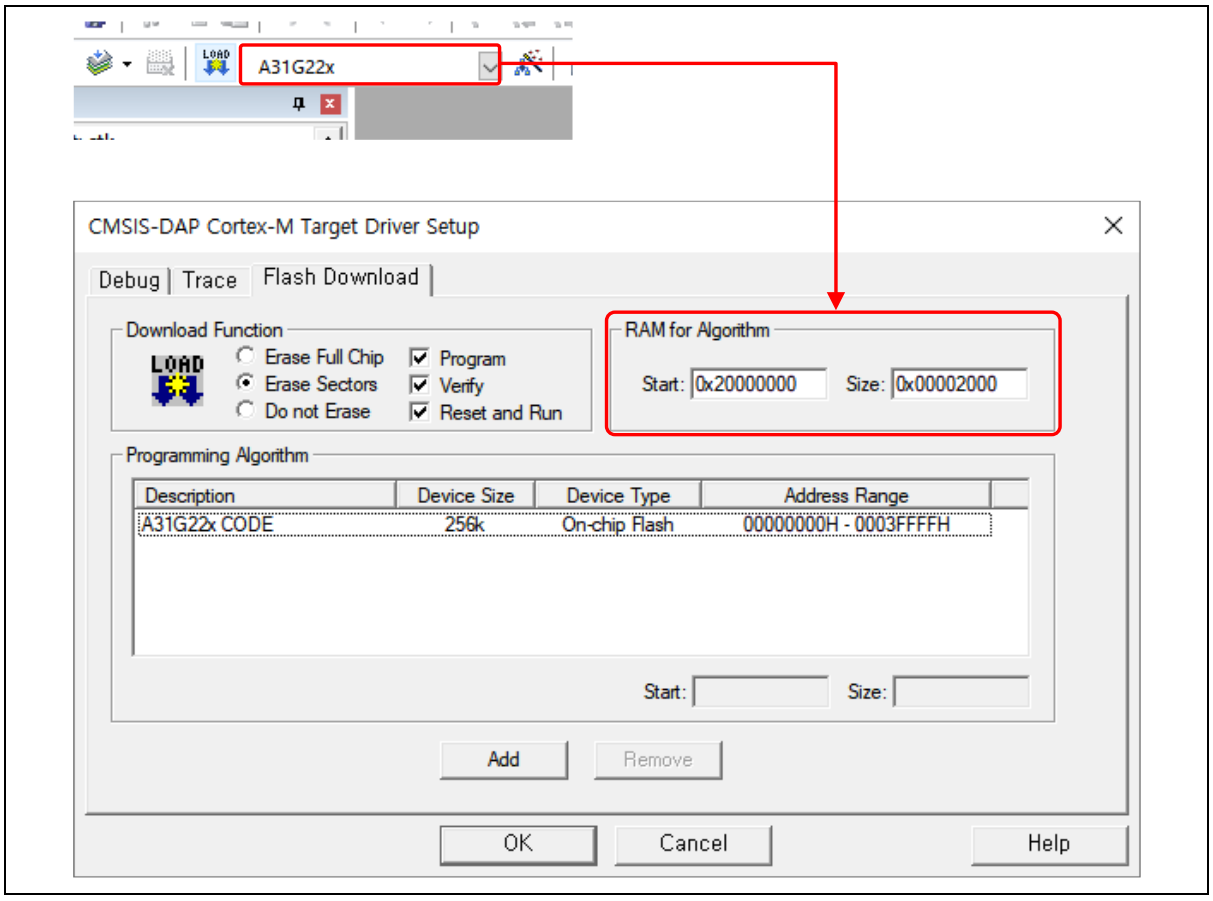
Figure 13. FLM register for target device in ‘Programming Algorithm’



The KEIL project provided by the AUDK32 automatically updates options of the ‘RAM for Algorithm’ item to their default values according to the ‘Select Target’ settings.

To change the options, you must refer to the user’s manual for that device before making any changes.

Figure 14. Auto-update of ‘RAM for Algorithm’ by ‘Select Target’ settings



4.4 Flashloader in IAR Embedded Workbench

The flashloader files of IAR Embedded Workbench are provided as described a file format shown in Table 7.

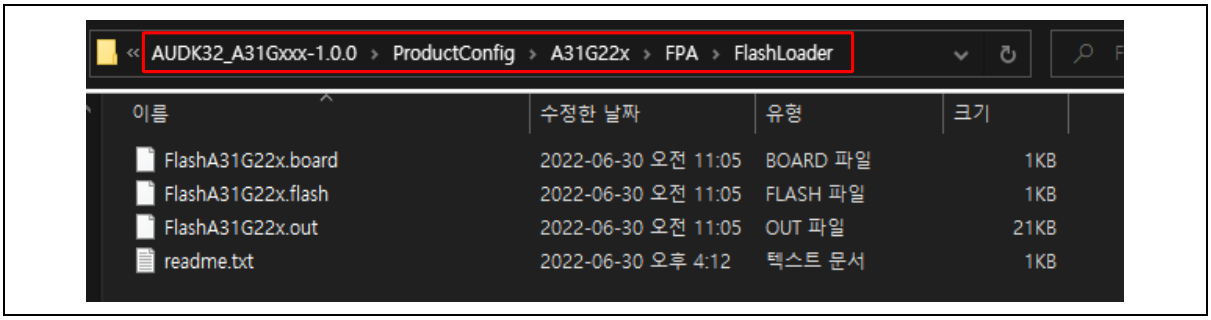
Table 7. Flashloader files in AUDK32

FlashLoader file	Description
Flash[Sub-Family].board	It is a top level configuration file and defines a file path of the flash file (*.flash) and a range of code flash.
Flash[Sub-Family].flash	It defines flash information as follows: • File path of the flashloader image (*.out) • Flash page size • Total number of flash blocks • Flash base address
Flash[Sub-Family].out	It is the flashloader binary file in ELF format is provided for the Sub-Family.

The flashloader files provided by the AUDK32 are located under the '~/ProductConfig/[Sub-Family]/FPA/FlashLoader' directory.

Figure 15 shows the A31G22x flashloader files referred by the IAR project in the location.

Figure 15. IAR flashloader file location of A31G22x



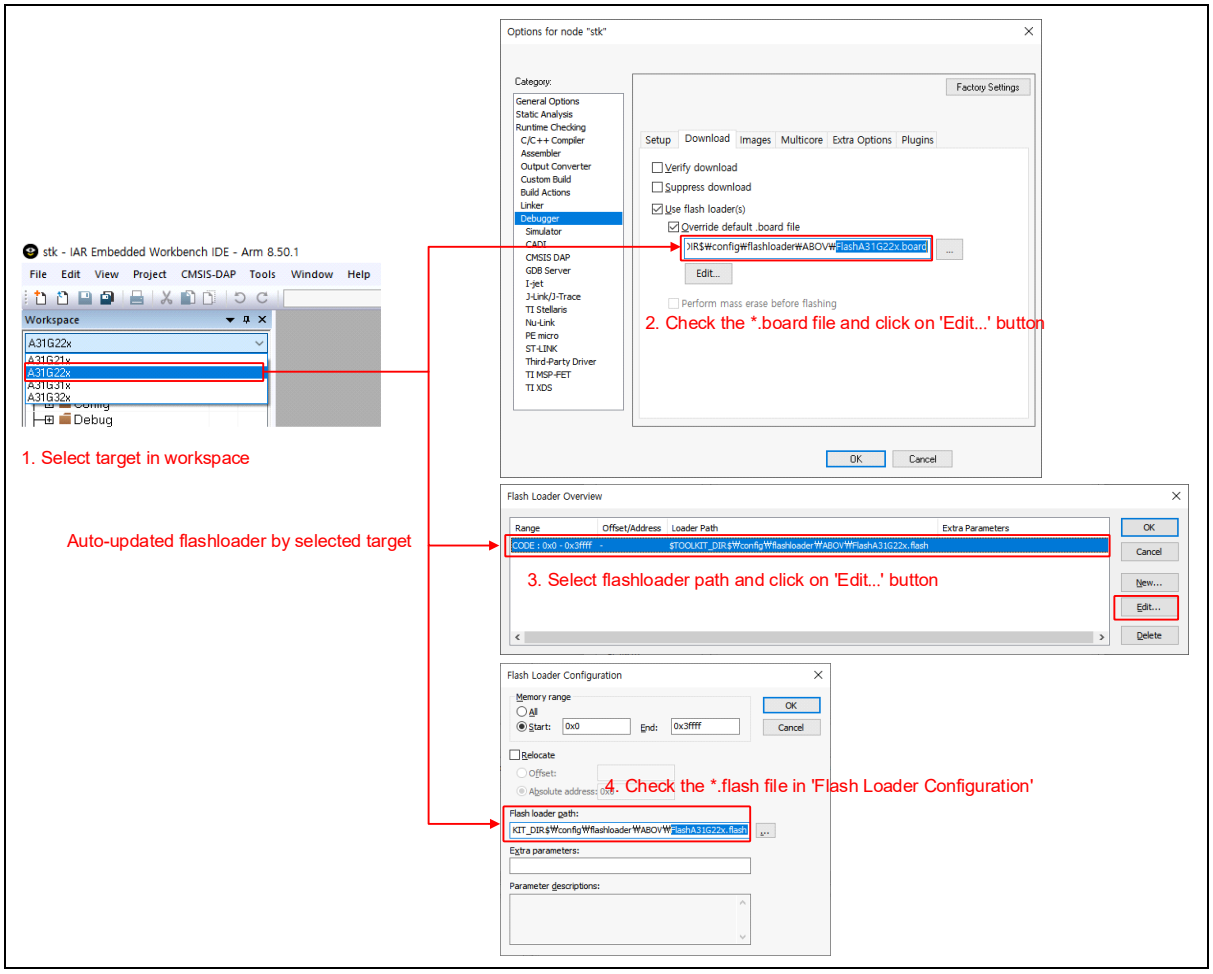
4.4.1 Flashloader by selection target in workspace

For the flash downloading or debugging, IAR project of AUDK32 is configured to refer to a relative path of flashloader provided by AUDK32 for each target device. Therefore, no additional settings are required.

When you select a target device in IAR workspace, flashloader configuration in the options window is automatically applied.

Figure 16 shows that flashloader configuration in the options window is automatically updated when the target is selected as A31G22x in IAR workspace.

Figure 16. Auto-update of flashloader by selected target



5. AUDK32 in KEIL uVision5

Chapter 5 describes how to build example code in KEIL IDE environment that is provided by the AUDK32 and how to run it on a target board. With the A31G22x as an example, we demonstrate how to set up the KEIL project using the Starter Kit and Bare board.

For more information on KEIL uVision5 (ARM-MDK), refer to the reference document below:

- URL: https://www.keil.com/support/man_arm.htm

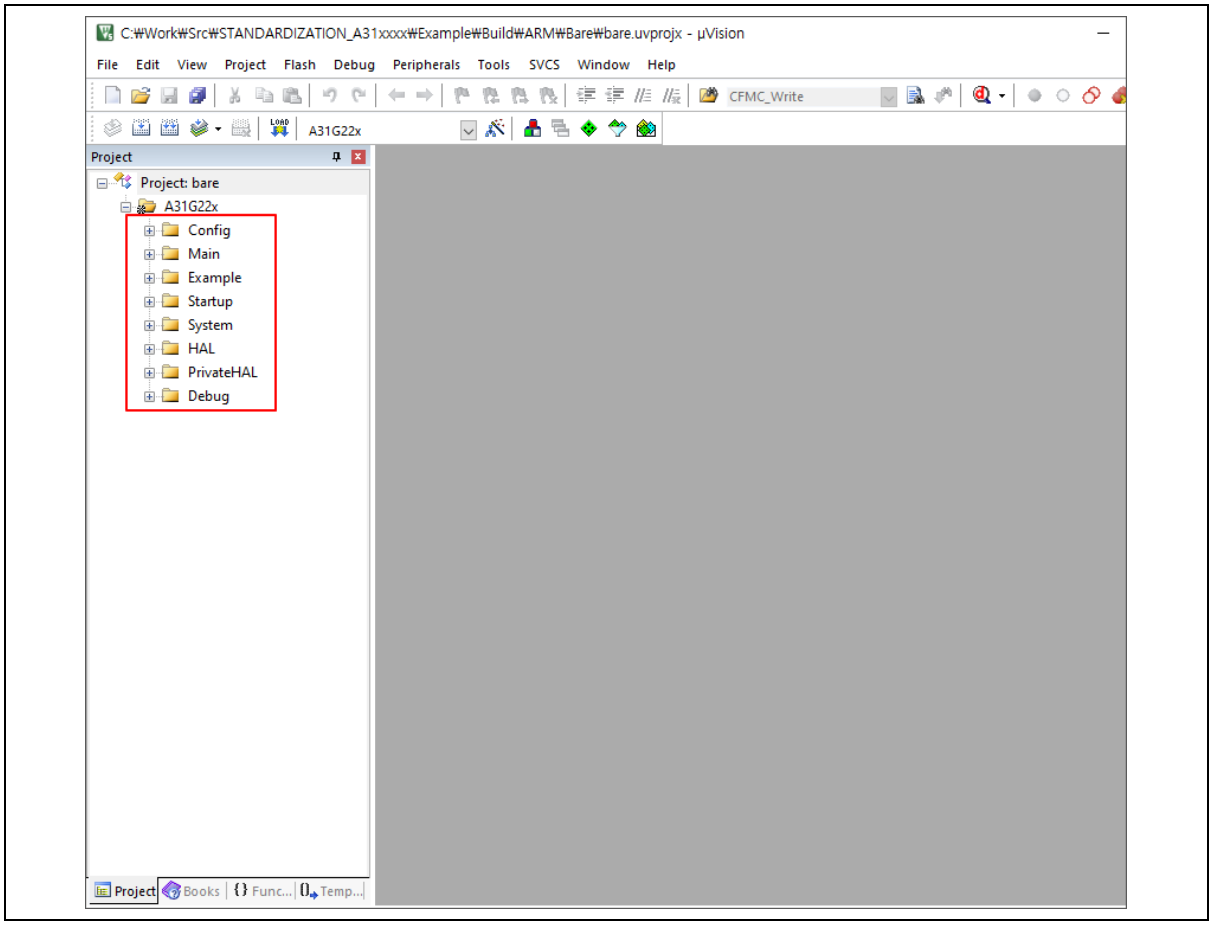
5.1 KEIL project

KEIL projects are provided on a board basis, and the location of each board project is as follows:

- Starter Kit board: ~/Example/Build/ARM/STK/stk.uvprojx
- Bare board: ~/Example/Build/ARM/Bare/bare.uvprojx

When you run the KEIL project in KEIL uVision5, you can see the group structure consisting of Config, Main, Example, Startup, System, HAL, PrivateHAL and Debug on the left pane of the screen, as shown in Figure 17.

Figure 17. Configuration of files in 'Config' group of KEIL IDE



Before starting compiling example, the development environment should be set up at first.
If you take a look at ‘Config’ group, you can find out some configuration files.
The configuration files need to be modified following the procedure below:

1. Select a target Sub-Family.
2. Modify the ‘abov_config.h’ file to select the target board and configure a debug channel.
3. Modify the ‘abov_example_config.h’ file to select example to build.
4. Check if the scatter loader files (*.sct) are proper for the target application.

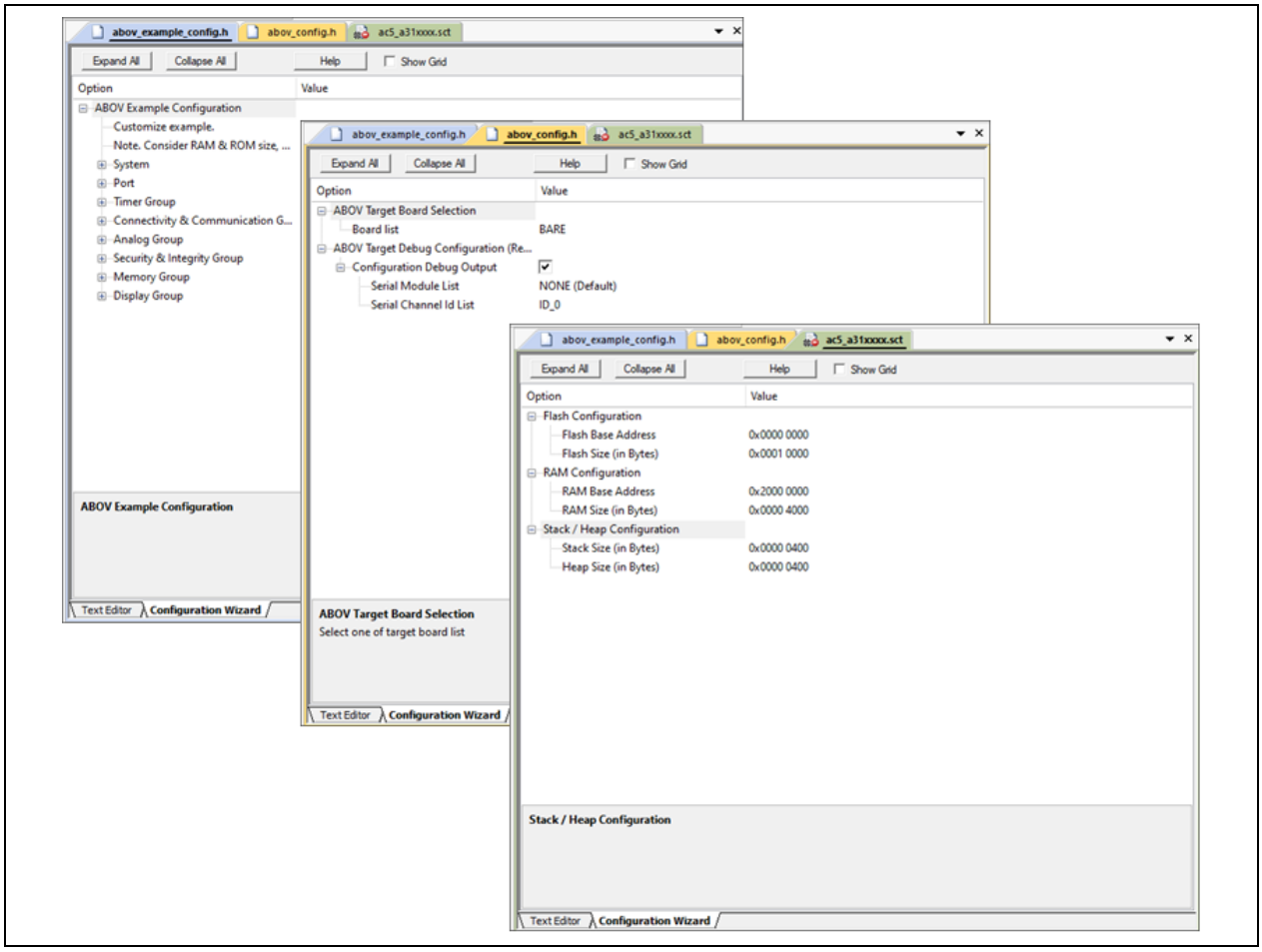
5.2 Configuration Wizard annotations

The AUDK32 uses ‘Configuration Wizard annotation’ based on the Open-CMSIS-Pack standard supported by KEIL IDE. Using this, it provides users with the ability to intuitively modify files in the Config group.

When the users click the configuration file, the corresponding annotation wizard tab appears allowing you to change the settings.

Figure 18 shows examples of annotation wizard associated with configuration files in the Config group.

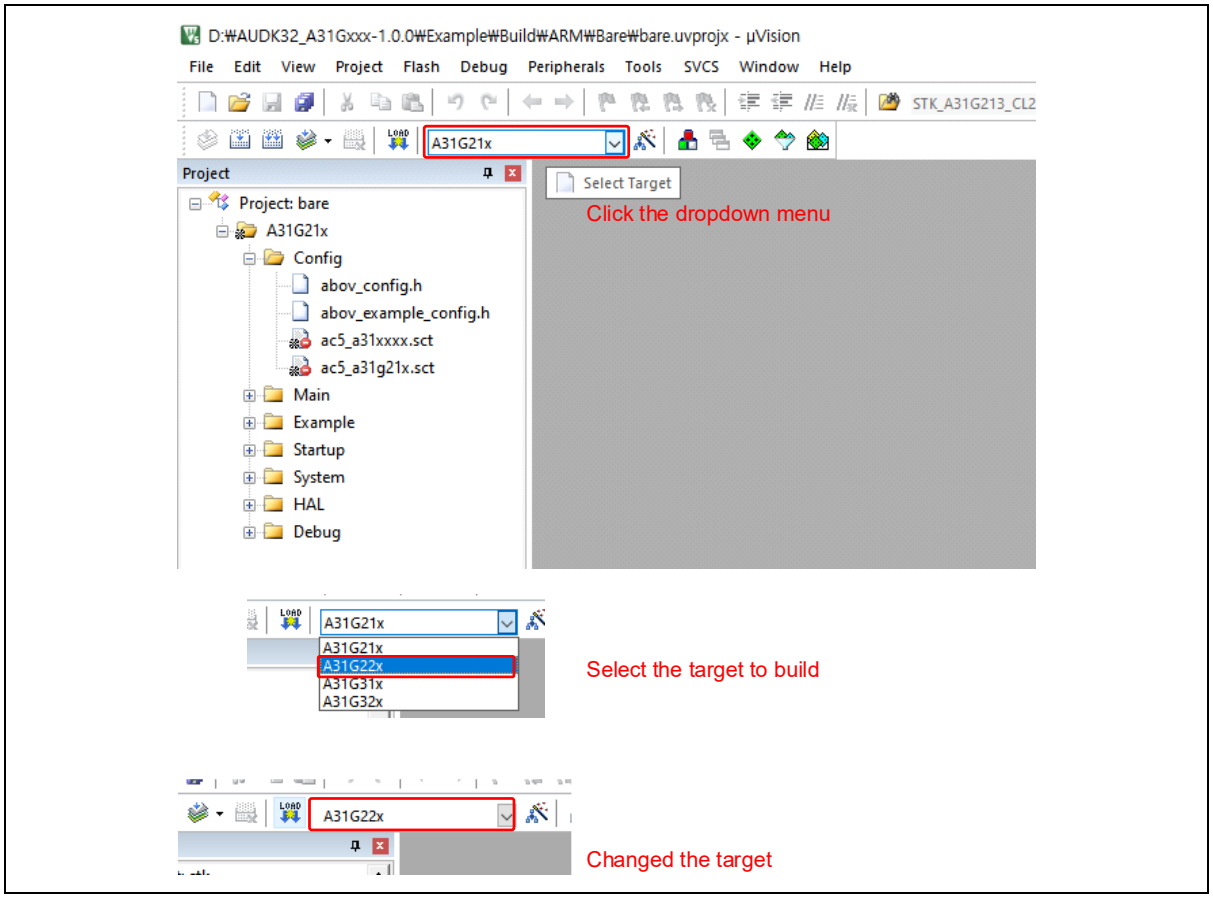
Figure 18. Configuration Wizard annotations in AUDK32



5.3 Select target

When the KEIL project is executed firstly, the Select Target is set to ‘A31G21x’ by default. You can change the target by selecting a different Sub-Family from the drop-down menu.

Figure 19. Changing target device using ‘Select Target’ function in KEIL IDE



5.4 abov_config.h

The ‘abov_config.h’ is a top-level configuration file and defines the configuration macros associated with the target board and debug channel. Once the configuration macros are defined, another configuration files related with the target board and debug channel are included automatically.

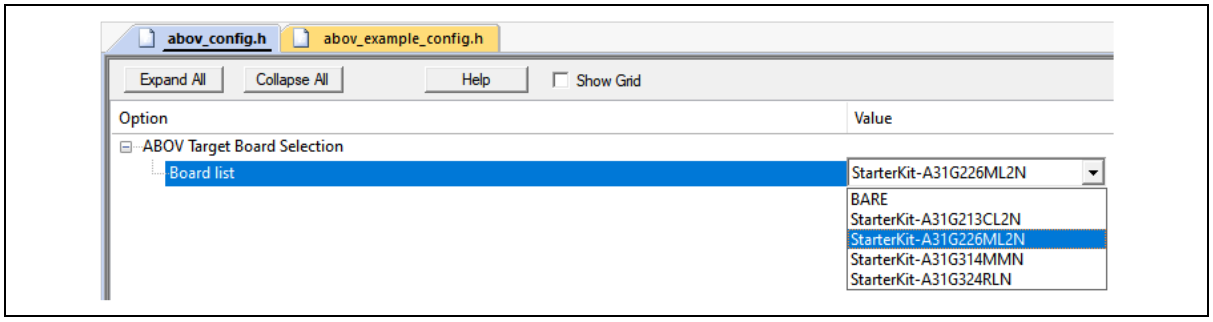
When you first open the ‘abov_config.h’ file using the annotation wizard in KEIL project file or Text editor, you will see the configuration macros in the file are set to default values already.
Users simply need to set up the target board and debug channel using the KEIL Configuration Wizard.

5.4.1 Select Starter Kit

The ‘stk.uvprojx’ file is supposed to configure and build the examples supported by Starter Kit.
However the default board in abov_config.h is ‘BARE’. Therefore, you must select the target Starter Kit from the drop-down menu shown in Figure 20.

The currently supported example is ‘GPIO_LED_BLINKING’.

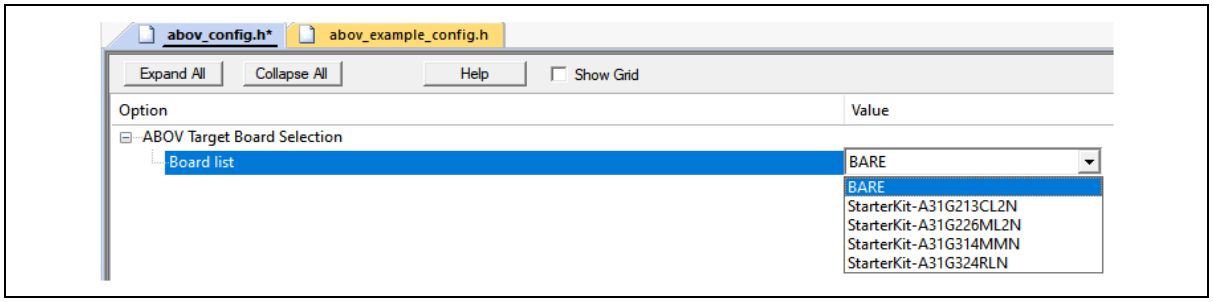
Figure 20. Selection of Starter Kit in KEIL IDE



5.4.2 Select BARE

The ‘bare.uvprojx’ file is supposed to verify on-chip peripherals which are provided by Bare board. Therefore, you must select ‘BARE’ from the ‘Board list’ drop-down menu, as shown in Figure 21.

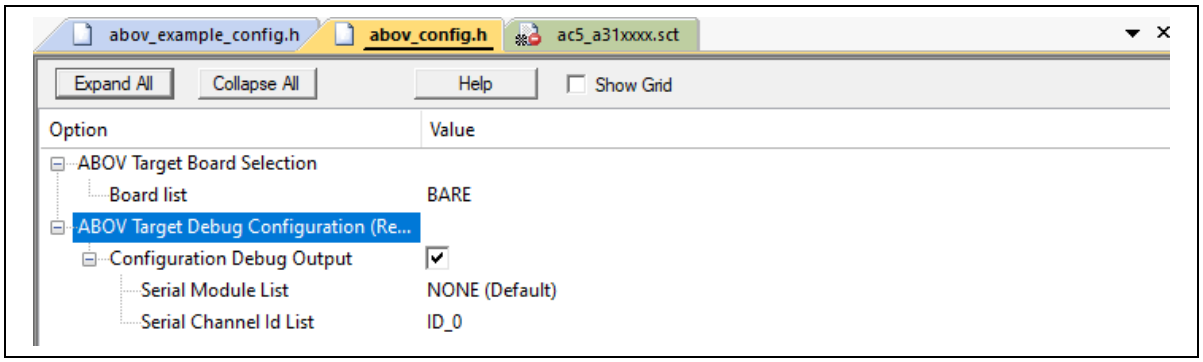
Figure 21. Selection of Bare board in KEIL IDE



5.4.3 Config debug channel

The ‘abov_config.h’ also shows settings of a current debug channel of the target device selected before.

Figure 22. Config debug channel in KEIL IDE



You can set which debug channel on which debug module as for serial communication referring to a layout on the target board and user’s manual of the target device.

5.5 abov_example_config.h

The ‘abov_example_config.h’ is used to select an example supported by the target board selected previously.

You can select the example to run on the target board by setting an identity value of example in this file. This change can be done by Text editor or the KEIL Configuration Wizard.

5.5.1 Example selection in Starter Kit

Figure 23 shows once user select Starter Kit as a target board, ‘GPIO_LED_BLINKING’ is taken as a target example to test automatically.
‘GPIO_LED_BLINKING’ is a representative example of Starter Kit and the others are not supported yet. Therefore, user should not change the target example.

Figure 23. GPIO_LED_BLINKING in case of Starter Kit board

```
/* Define customized target example */
#if defined(_RTE_) || (TARGET_BOARD & BD_STK_MASK)
/* Define target example */
#define GPIO_LED_BLINKING

#else

#if (EX_MULTI_SCU_LVD == 1)
#define SCU_LVD_TC
#endif
#if (EX_MULTI_SCU_PWR == 1)
#define SCU_PWR_TC
#endif
#if (EX_MULTI_PCU_GPIO == 1)
#define PCU_TC
#endif
#if (EX_MULTI_TIMER1 == 1)
#define TIMER1_TC
#endif
#if (EX_MULTI_TIMER2 == 1)
#define TIMER2_TC
#endif
#if (EX_MULTI_TIMER3 == 1)
#define TIMER3_TC
#endif
#if (EX_MULTI_TIMER4 == 1)
#define TIMER4_TC
#endif
#if (EX_MULTI_WDT == 1)
#define WDT_TC
#endif
#if (EX_MULTI_WT == 1)
#define WT_TC
#endif
#if (EX_MULTI_I2C == 1)
#define I2C_TC
#endif
```

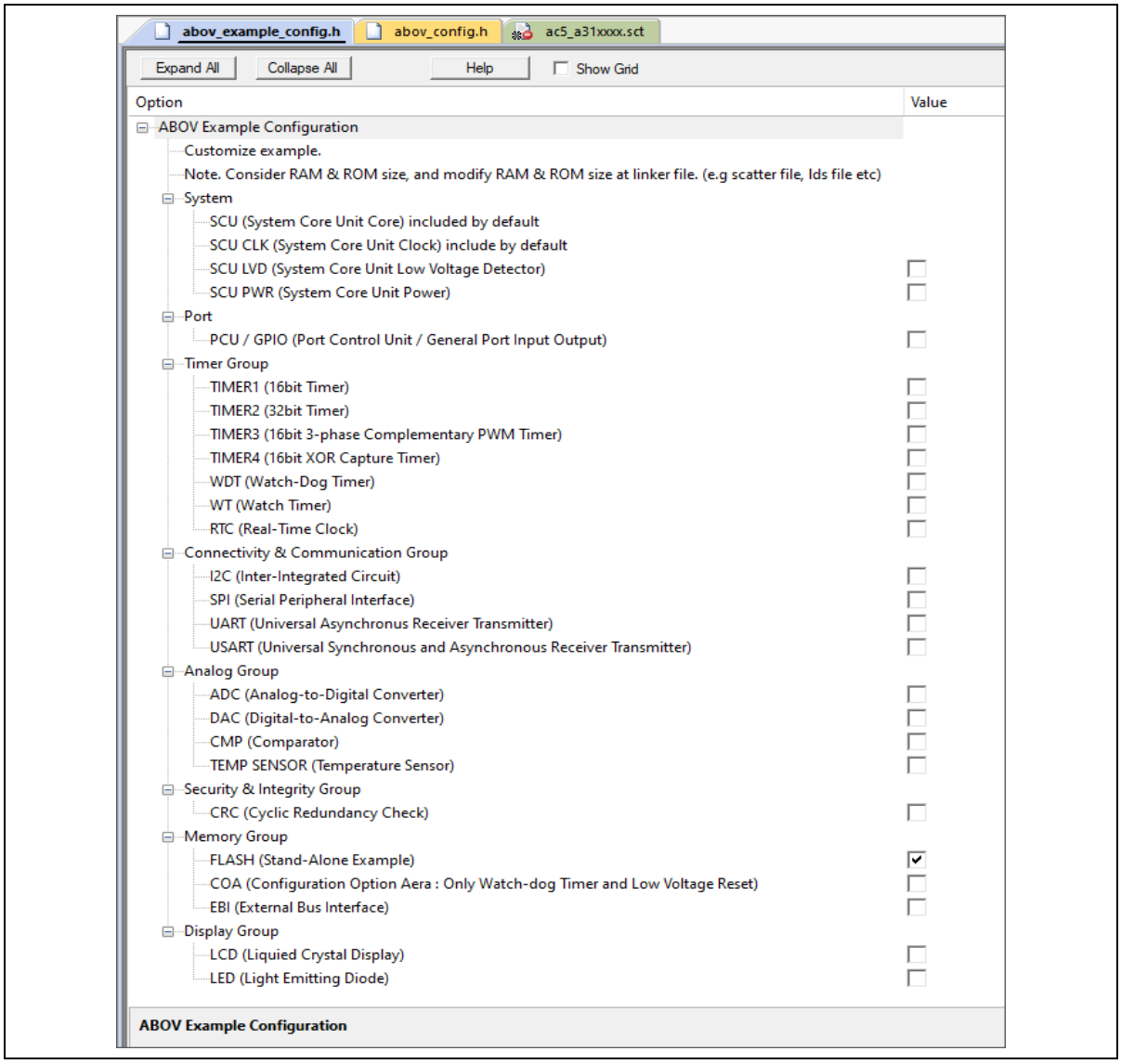
5.5.2 Multiple examples selection in Bare board

Selecting multiple examples is supported and the detailed procedure for it is shown below.

On the Configuration Wizard window, the two examples (SCU and SUC CLK) are embedded and selected by default 'abov_example_config.h'.
You must not exclude these examples because they are fundamental and essential for providing runtime environment on the target board with the other examples.

Examples are grouped by 'System', 'Port', 'Timer Group', 'Connectivity & Communication Group', 'Analog Group', 'Security & Integrity Group', 'Memory Group' and 'Display Group'.
If you tick a check box of a group, examples belonging to it are expanded and displayed as Figure 24 below. You only need to choose each example by ticking check box of it.
If you click over an example for each peripheral, main features and summarized descriptions for the peripheral are usually displayed in the annotation field at the bottom of the Configuration Wizard.

Figure 24. Selection of multiple examples in KEIL IDE

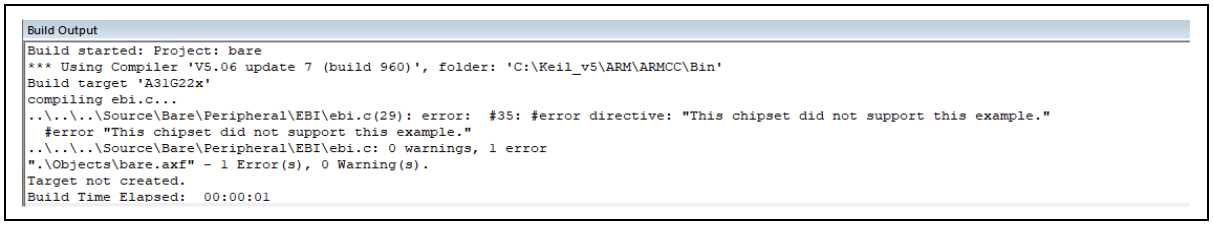


5.5.3 Error messages

When you build examples, an error occurs because the peripherals to test are not supported by the target board or device and the error message is displayed on 'Build Output' windows as Figure 25 below.

For more information, refer to user's manual of the target device.

Figure 25. Build error message with improper example

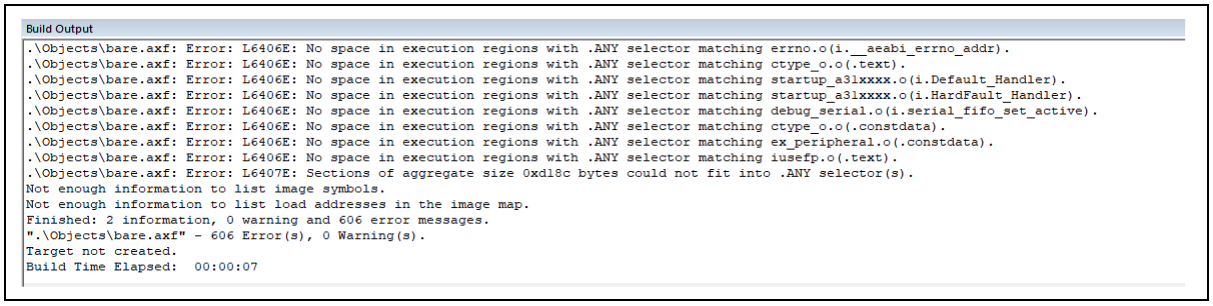


In addition to the unsupported peripherals, another error message is generated if memory resource is not enough to execute the examples as Figure 26 below.

In this case, you can modify the '*.sct' file to ensure that sufficient memory resources are available for running the examples, by referring to memory specification described in user's manual.

In other way you can select an appropriate number of examples not to exceed the maximum memory size of the target board.

Figure 26. Build error message with insufficient resources



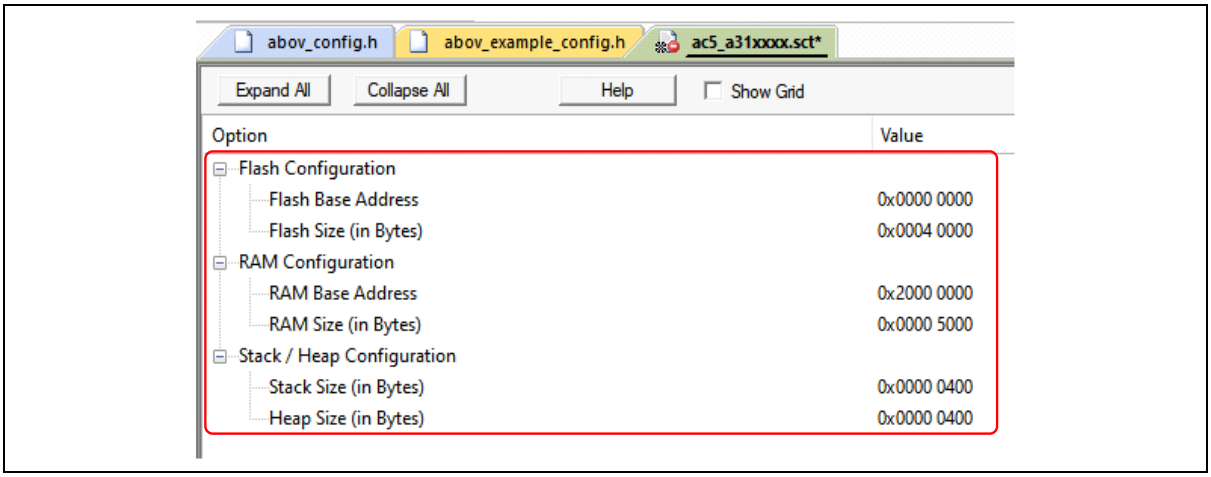
5.6 Scatter loader (*.sct) file

The scatter file is used to config the location and size of ROM, RAM, Stack, and Heap, and relocate the final outputs generated by the linker. This file in the AUDK32 has default values for each device Sub-Family.

You can change the default values in the scatter file but be sure that the changes you make should comply with the specifications of flash memory and SRAM in the user's manual of the target device.

Figure 27 shows the configurable options in the scatter file and you can change them by any text editor or annotation facility of the Configuration Wizard in KEIL IDE.

Figure 27. Configuration of memory information by ac5_a31xxxx.sct



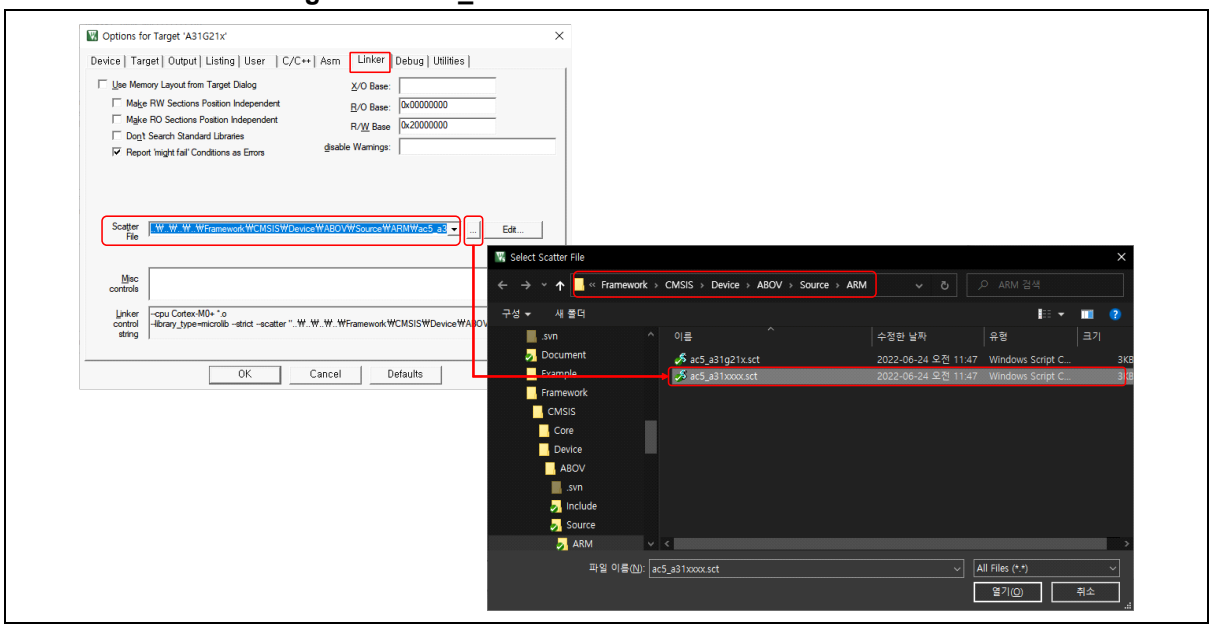
The KEIL scatter files are located in '~/Framework/CMSIS/Device/ABOV/Source/ARM/'.
In most case, AUDK32 provides a single scatter file for each ARM core but in some cases, there could be a special scatter file dedicated to a certain device. See Table 8.

Table 8. Scatter files

ABOV Device Family/ARM Core	ABOV Device Sub-Family	Scatter file
A31xxxx/CM0 plus	Except for A31G21x and A31G12x	ac5_a31xxxx.sct
	A31G21x	ac5_a31g21x.sct
	A31G12x	ac5_a31g12x.sct
A33xxxx/CM3	N/A	ac5_a33xxxx.sct
A34xxxx/CM4	N/A	ac5_a34xxxx.sct

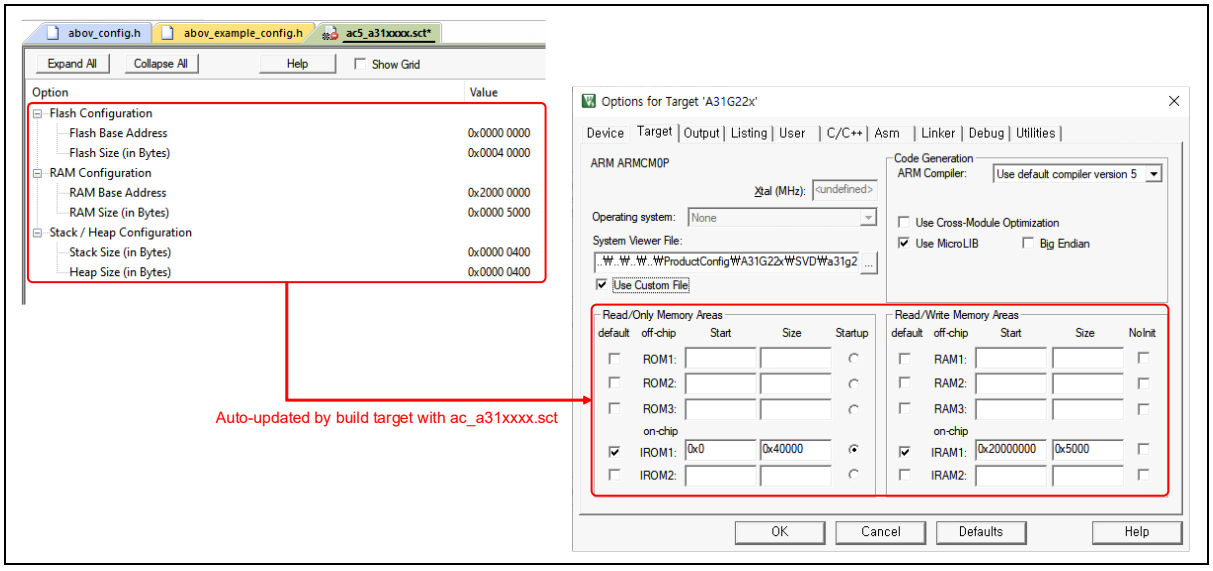
Figure 28 shows how to set the scatter file of the A31G22x in target option window of KEIL project file.

Figure 28. ac5_a31xxxx.sct scatter file of Linker tab



The memory settings selected in the scatter file is automatically applied when the target is built, as shown in Figure 29. (1)(2)(3)

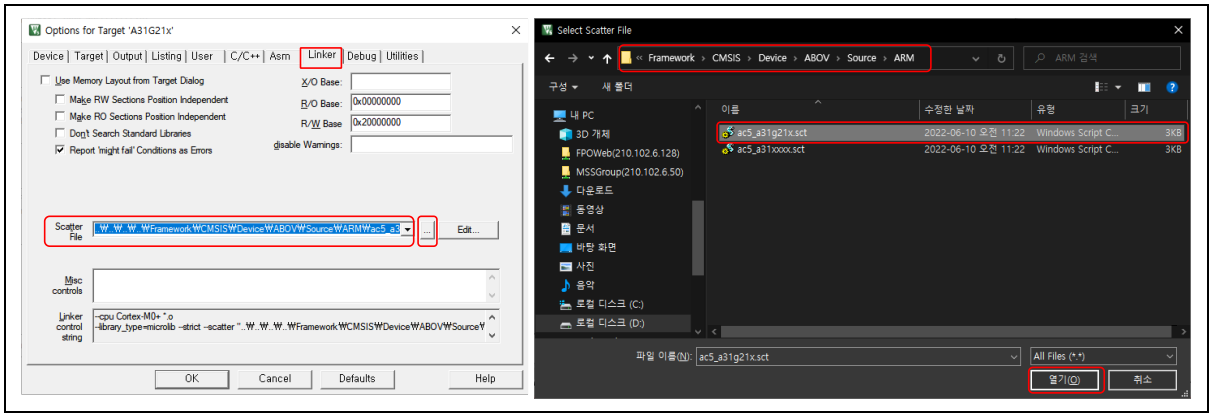
Figure 29. Auto-update memory configuration from selected scatter file



NOTE:

1. For the case using the target A31Gxxx other than A31G21x or A31G12x, use the 'ac5_a31xxxx.sct' file.
2. Because the memory size of A31G21x or A31G12x is too small for the 'ac5_a31xxxx.sct' file to operate normally, additional scatter files for A31G21x and A31G12x only are provided. Therefore, when using A31G21x or A31G12x as a target sub family, move on the 'Linker' tab of the 'Options for Target' window and use the scatter file 'ac5_a31g21x.sct' or 'ac5_a31g12x.sct' located under the directory '~/Framework/CMSIS/Device/ABOV/Source/ARM/'. Refer to Figure 30.
3. AUDK32 configures KEIL project file in advance so that the 'ac5_a31g21x.sct' for A31G21x, 'ac5_a31g12x.sct' for A31G12x and the 'ac5_a31xxxx.sct' for the other devices are used.

Figure 30. Selection of ac5_a31g21x.sct in linker tab for A31G21x

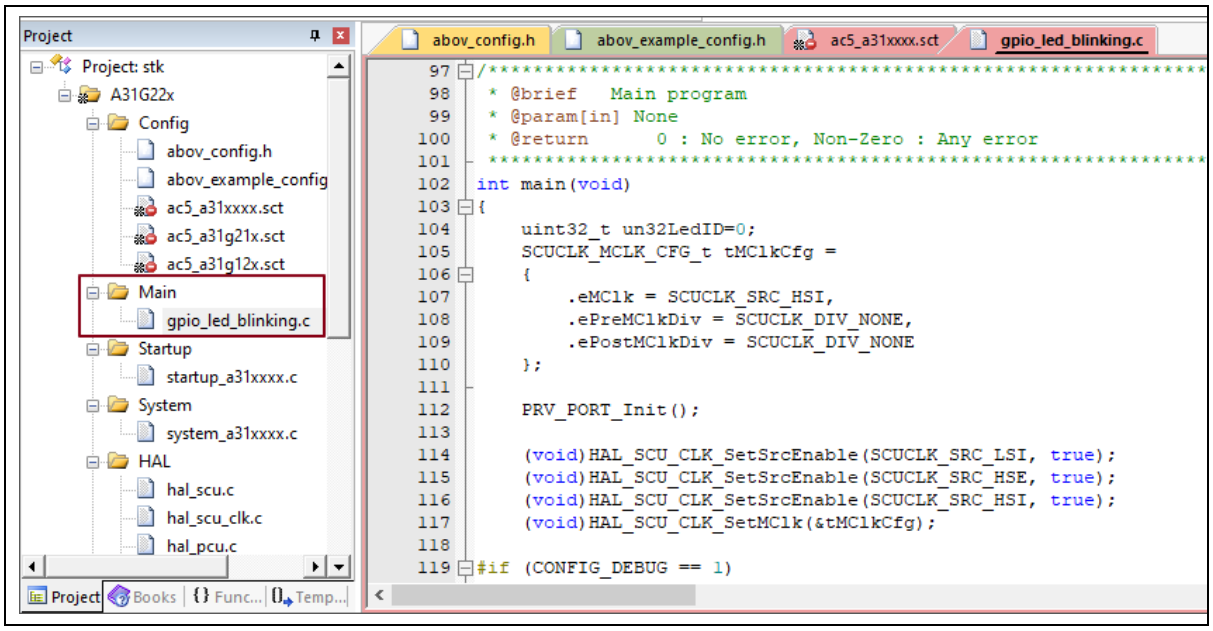


5.7 Building and run example

The AUDK32 provides examples for each target board. Users must check if which examples are supported by the target board first, and then select examples following the description in the previous chapters. There is an example list in the 'abov_example_config.h' file.

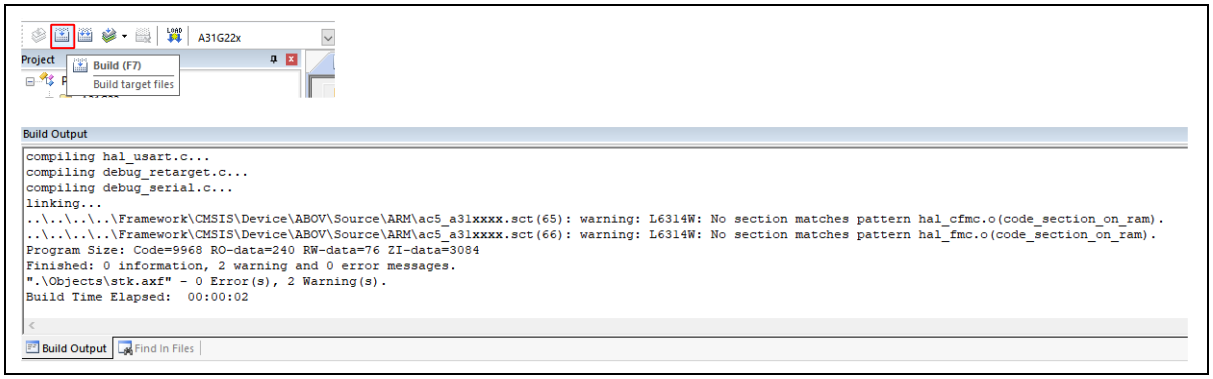
Figure 31 shows that the 'gpio_led_blinking' example in the 'stk.uvprojx' project operating on the Starter Kit.

Figure 31. gpio_led_blinking.c example in KEIL IDE



You can build a project by selecting the 'Build target files' from the Project menu or by clicking the Build (F7) icon button as shown in Figure 32. You can check the results of the project build, on the 'Build Output' window.

Figure 32. Build target in KEIL IDE



Once source codes have been completely compiled without errors, now you can download the binary image file onto the target board by clicking the 'Download' from the Flash menu or by clicking the 'Download (F8)' button as shown in Figure 33.

You can check if the file has been successfully downloaded, in the Build Output window.

Figure 33. Download image to flash memory in KEIL IDE

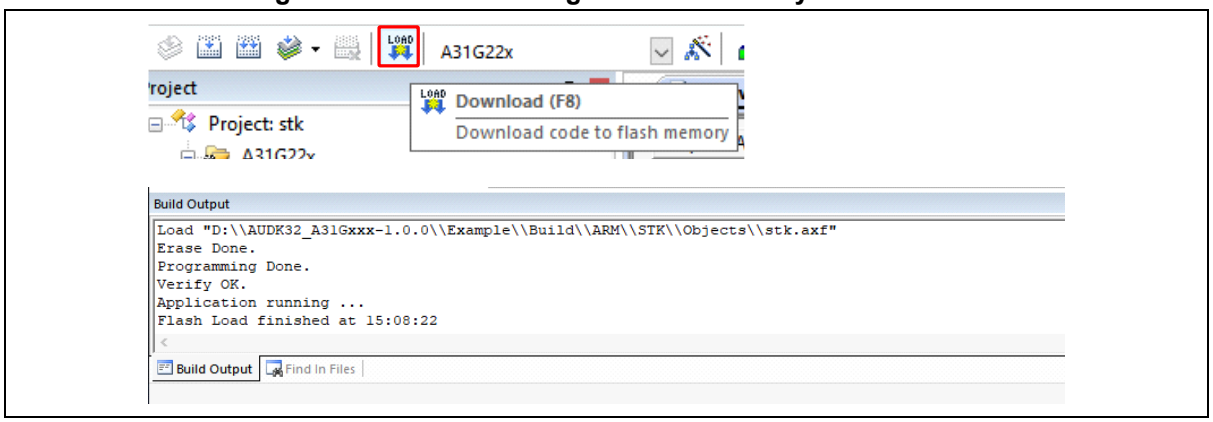
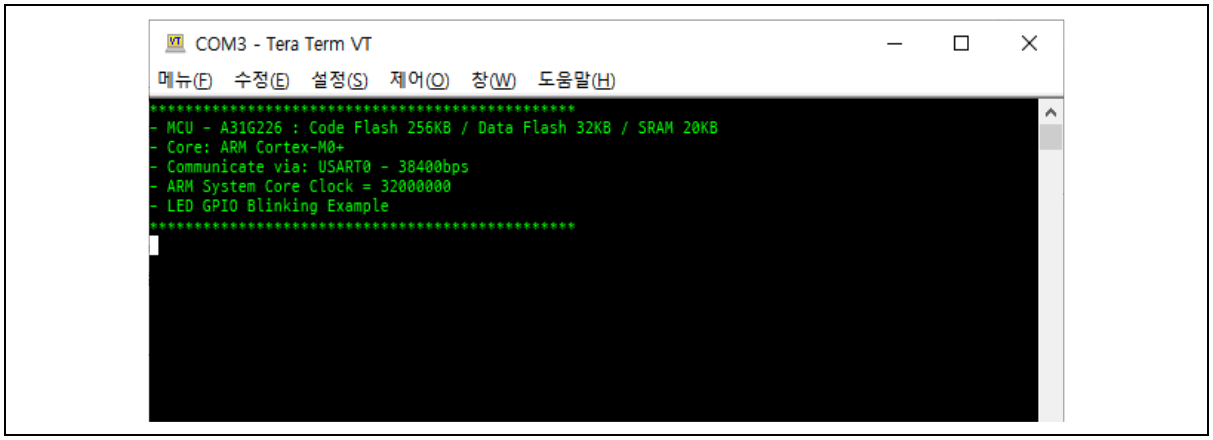


Figure 34 shows that the output of 'gpio_led_blinking' example is downloaded and running on the Starter Kit.

Figure 34. State of gpio_led_blinking example

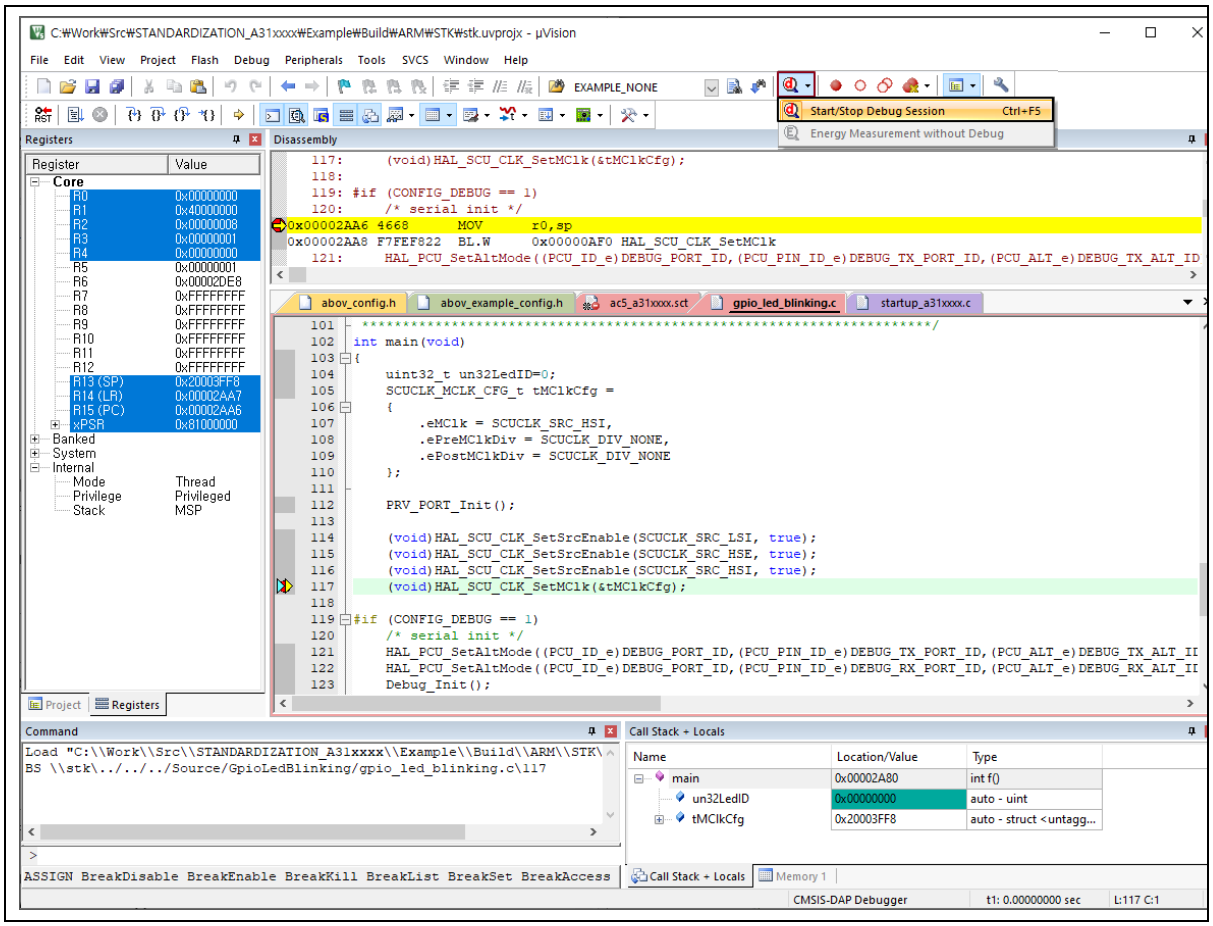


5.8 Debugging with A-Link

The AUDK32 supports A-Link debugger that is based on the CMSIS-DAP interface by default.
The A-Link debugger allows source level debugging like other ARM debugger tools.

To enter debugging session, select the ‘Start/Stop debug session’ from the ‘Debug’ menu or press the icon button shown in Figure 35.

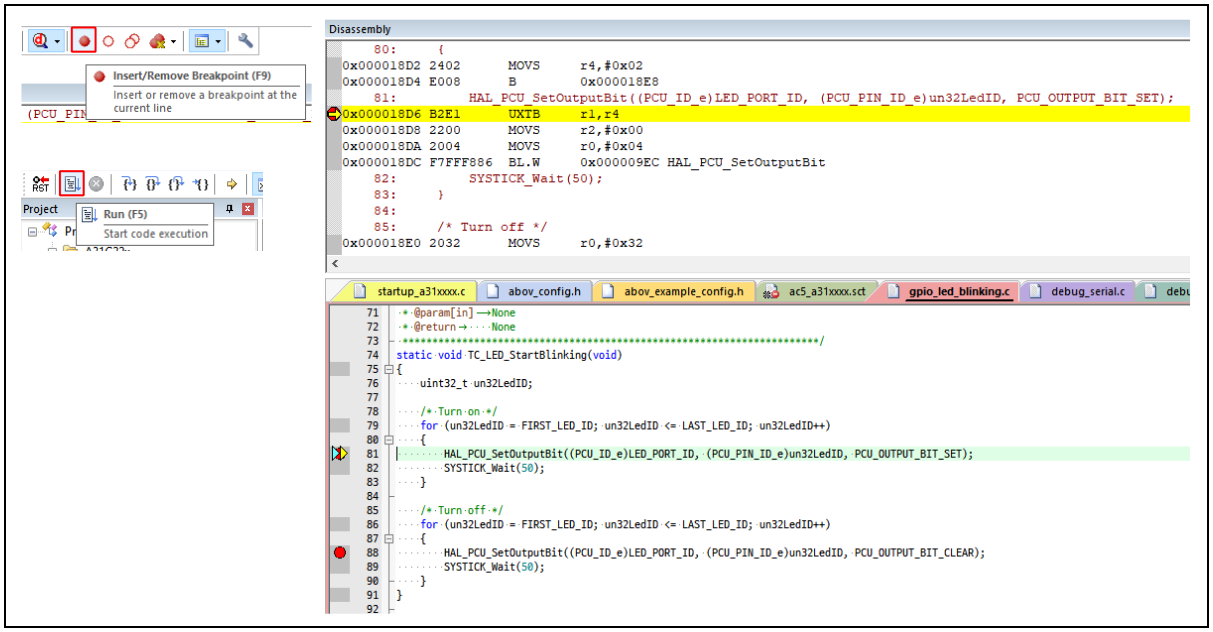
Figure 35. Debug session in KEIL IDE



If you set a break point at the front of the line you want to pause and click the Run icon button, the target device runs up to the line where the break point is set, and then stops as shown in Figure 36.

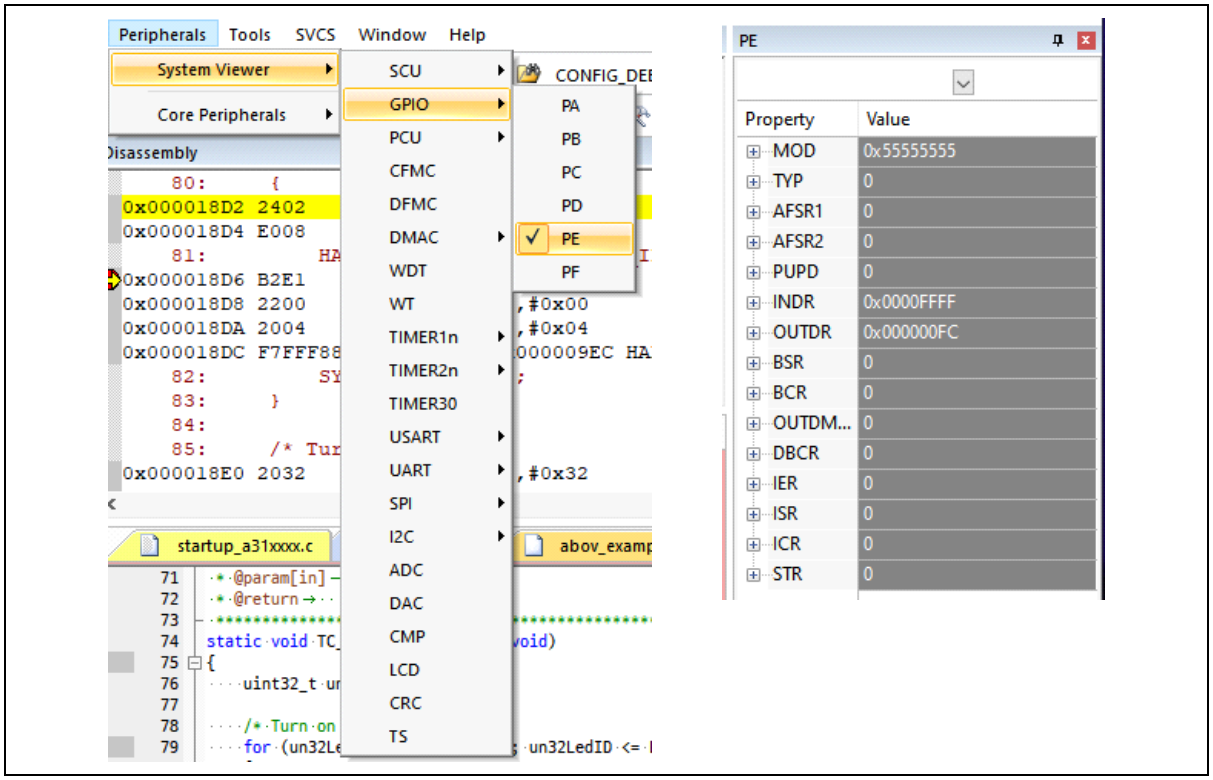
Therefore, you can use a debugger to check and modify specific code such as variables, Special Function Registers (SFRs), and memory for your purpose.

Figure 36. Break point in KEIL IDE



As shown in Figure 37, you can check or control the register values in real time by loading the System Viewer Description (SVD) file of the target device. The SVD file is provided by the AUDK32.

Figure 37. System viewer in KEIL IDE



6. AUDK32 in IAR Embedded Workbench

Chapter 6 describes how to build example code in IAR IDE environment that is provided by the AUDK32 and run it on a target board. The following sections describe how to set up the IAR workspace and AUDK32 configuration files, and procedure for building and running a target example on the Starter Kit or Bare board, respectively.

For more information about the IAR Embedded Workbench for ARM, refer to the reference document available at the following web site:

- URL: https://www.iar.com/kr/products/architectures/arm/iar-embedded-workbench-for-arm/

6.1 IAR Workspace

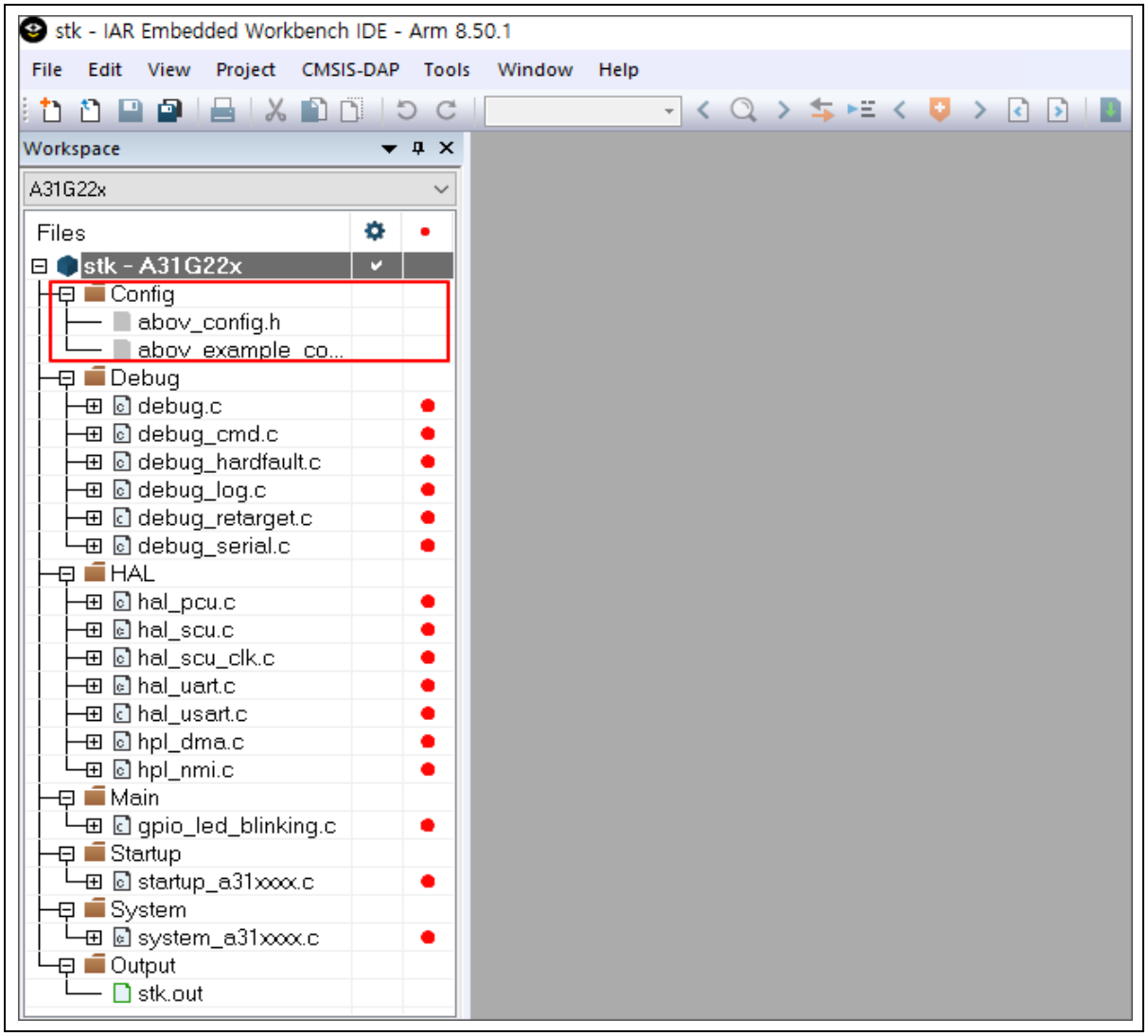
Workspace is a container that holds one or more projects. A workspace file typically has the extension '.eww'. The workspace provides a convenient way to organize and manage multiple projects within a single environment.
However, KEIL IDE does not discriminate workspace and project and has a project file only.

IAR workspaces are provided on a board basis, and the location of them is as follows:

- Starter Kit board: ~/Example/Build/IAR/STK/stk.eww
- Bare board: ~/Example/Build/IAR/Bare/bare.eww

If you run IAR workspace file in the IAR Embedded Workbench, the IDE shows all projects belongs to the IAR workspace you run. A workspace file contains only a single project in AUDK32.
The project consists of six groups: Config, Debug, HAL, Main, Startup and System as shown in Figure 38.

Figure 38. Configuration files in 'Config' group of the IAR IDE



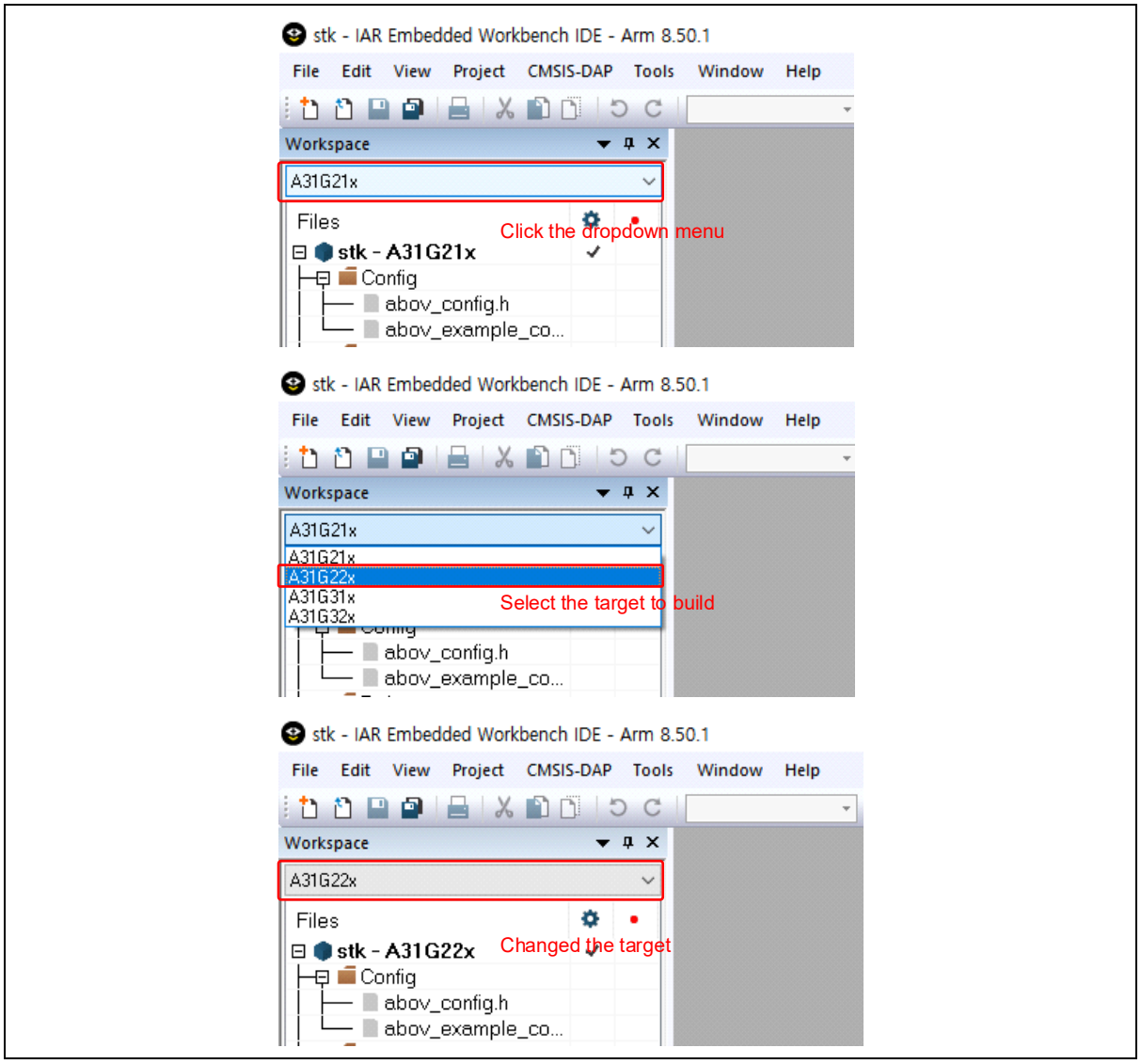
Before selecting and building examples, you must select the target Sub-Family and modify files in the Config group appropriately. Follow the procedures below for this purpose:

1. Select a target Sub-Family.
2. Modify the ‘abov_config.h’ file to select the target board and configure a debug channel.
3. Modify the ‘abov_example_config.h’ file to select example to build.
4. Check if the scatter loader files (*.icf) are proper for the target application.

6.2 Select target

When you first execute the stk.eww workspace, the target Sub-Family in the workspace is set to ‘A31G21x’ by default. You can change the target Sub-Family from the drop-down menu shown in Figure 39.

Figure 39. Changing target Sub-Family using ‘Select Target’ function in IAR IDE



6.3 abov_config.h

The ‘abov_config.h’ is a top-level configuration file and defines the configuration macros associated with the target board and debug channel. Once the configuration macros are defined, another configuration files related with the target board and debug channel are included automatically.

Since the annotation facility is not available in IAR 8.50 for now, Text editor should be used to change a configuration macro.

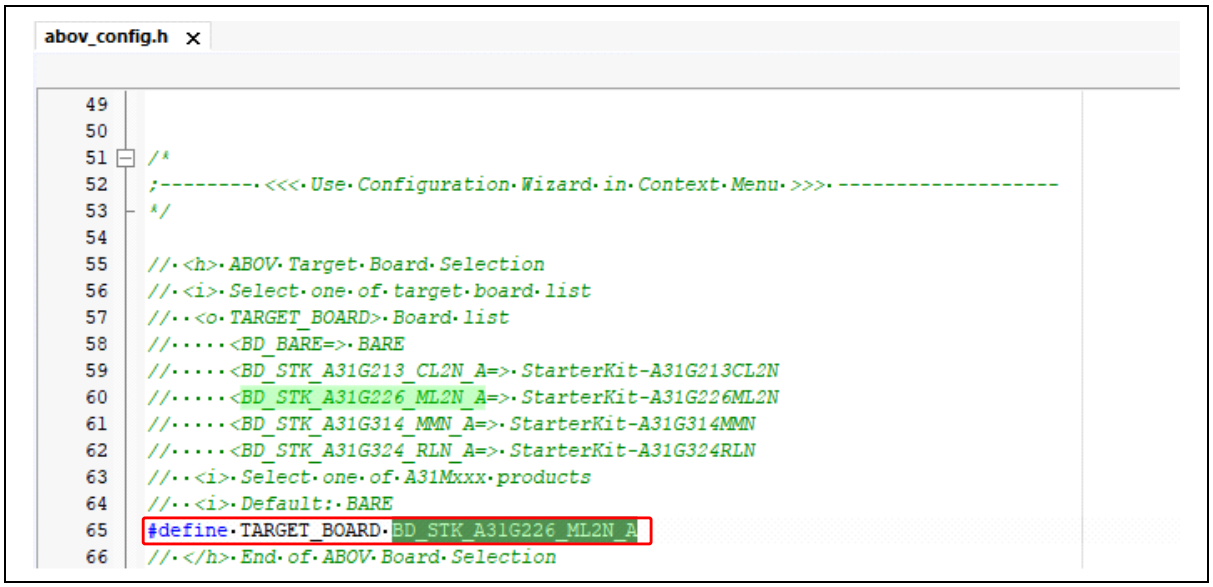
Users simply need to set up the target board and debug channel using the annotation wizard. When you first open the ‘abov_config.h’ file using IAR or external text editor, you will see the configuration macros in the file are set to default values already.

6.3.1 Select Starter Kit

The ‘stk.eww’ file is supposed to configure and build the examples supported by Starter Kit.

Since the default board in abov_config.h is ‘BARE’, you must replace BARE by the target Starter Kit as Figure 40 below.
For now, only ‘GPIO_LED_BLINKING’ is supported by Starter Kit.

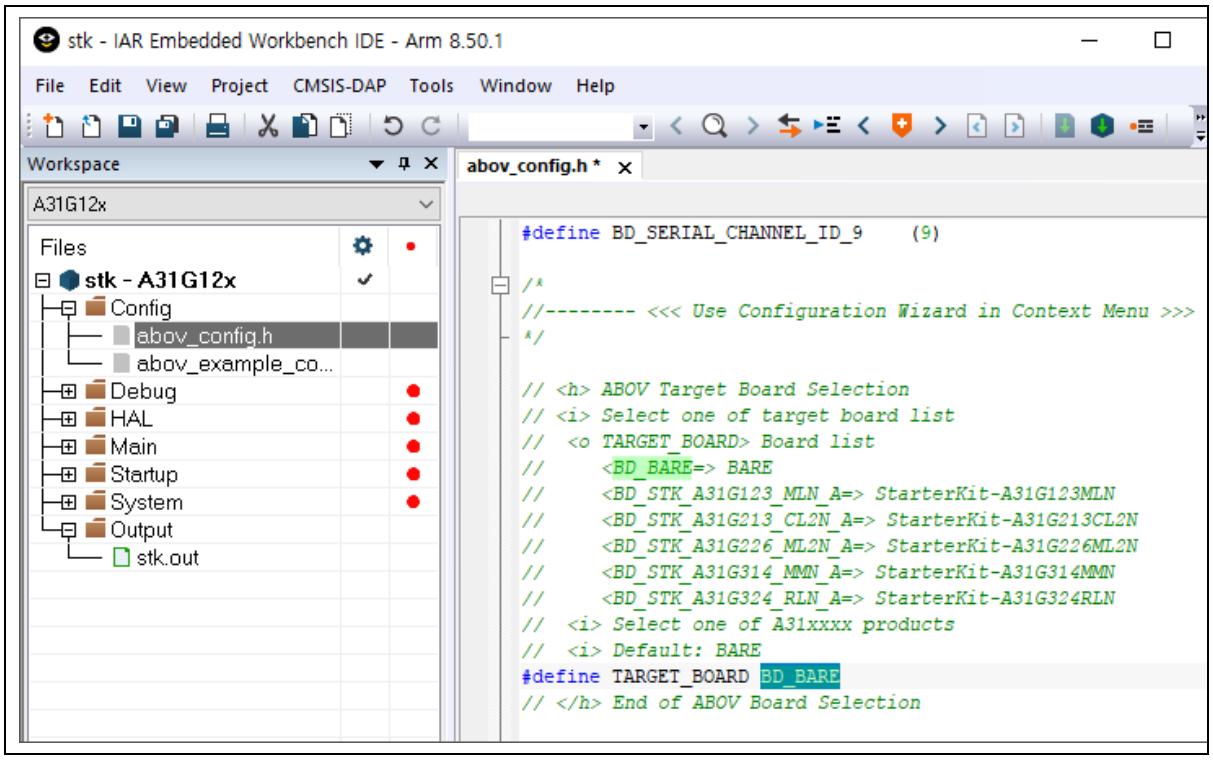
Figure 40. Selection of Starter Kit in IAR IDE



6.3.2 Select BARE

The ‘bare.eww’ file is supposed to verify on-chip peripherals which are provided by Bare board. Therefore, the target board should always be BD_BARE as shown in Figure 41.

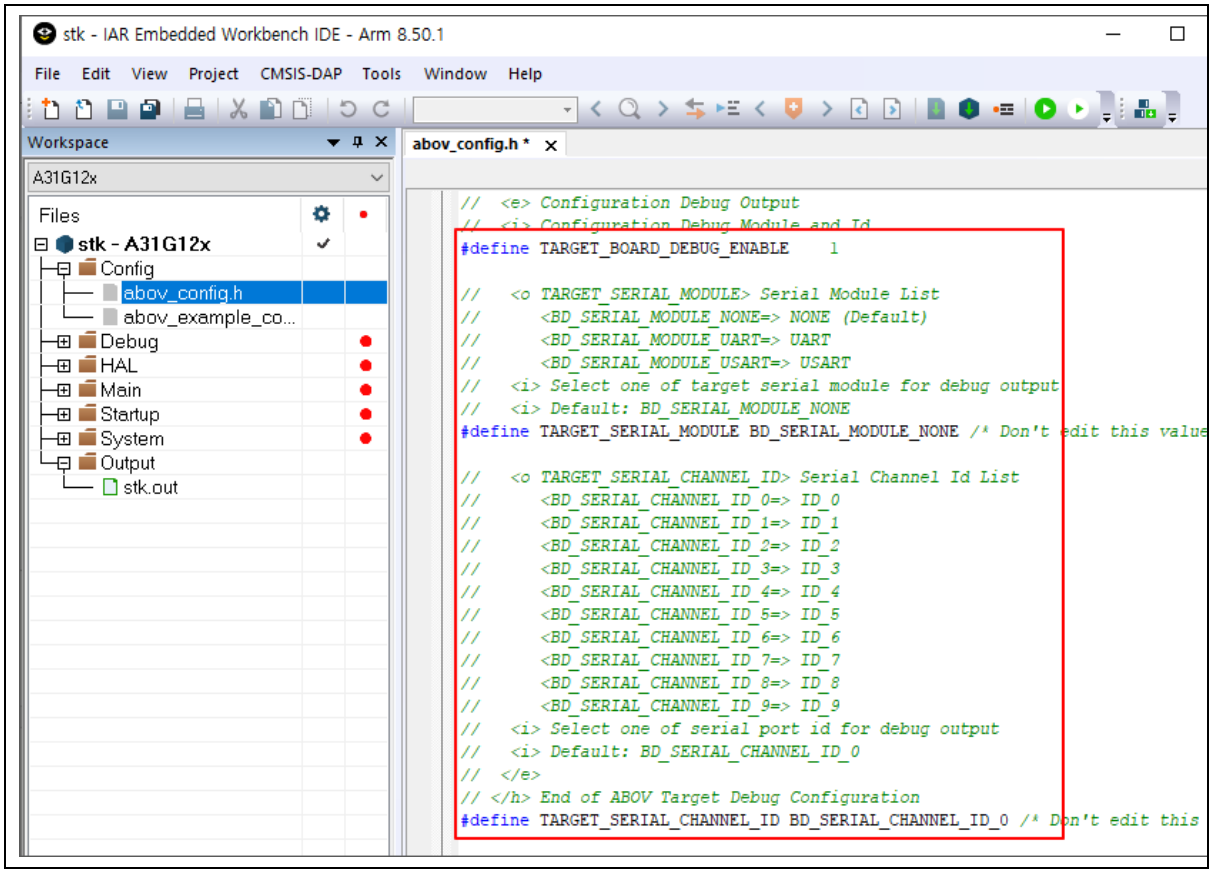
Figure 41. Selection of Bare board in IAR IDE



6.3.3 Config debug channel

The ‘abov_config.h’ also shows settings of a current debug channel of the target device selected before. If you take a look at abov_config.h, you can find out macros related to debug channel as shown Figure 42.

Figure 42. Configure Debug channel



You can set which debug channel on which debug module as for serial communication referring to a layout on the target board and user’s manual of the target device.

6.4 abov_example_config.h

The ‘abov_example_config.h’ is used to select an example supported by the target board selected previously.

You can select the example to run on the target board by setting an identity value of example in this file. This change can be done by Text Editor only because annotation feature is not available in IAR 8.50 for now.

6.4.1 Example selection of Starter Kit

Figure 43 shows once user select Starter Kit as a target board, ‘GPIO_LED_BLINKING’ is taken as a target example to test automatically.
‘GPIO_LED_BLINKING’ is a representative example of Starter Kit and the others are not supported yet. Therefore, user should not change the target example.

Figure 43. GPIO_LED_BLINKING in case of Starter Kit board

```
/* Define customized target example */
#if defined(_RTE_) || (TARGET_BOARD & BD_STK_MASK)
/* Define target example */
#define GPIO_LED_BLINKING

#else

#if (EX_MULTI_SCU_LVD == 1)
#define SCU_LVD_TC
#endif
#if (EX_MULTI_SCU_PWR == 1)
#define SCU_PWR_TC
#endif
#if (EX_MULTI_PCU_GPIO == 1)
#define PCU_TC
#endif
#if (EX_MULTI_TIMER1 == 1)
#define TIMER1_TC
#endif
#if (EX_MULTI_TIMER2 == 1)
#define TIMER2_TC
#endif
#if (EX_MULTI_TIMER3 == 1)
#define TIMER3_TC
#endif
#if (EX_MULTI_TIMER4 == 1)
#define TIMER4_TC
#endif
#if (EX_MULTI_WDT == 1)
#define WDT_TC
#endif
#if (EX_MULTI_WT == 1)
#define WT_TC
#endif
#if (EX_MULTI_I2C == 1)
#define I2C_TC
#endif
```

6.4.2 Multiple examples selection in Bare board

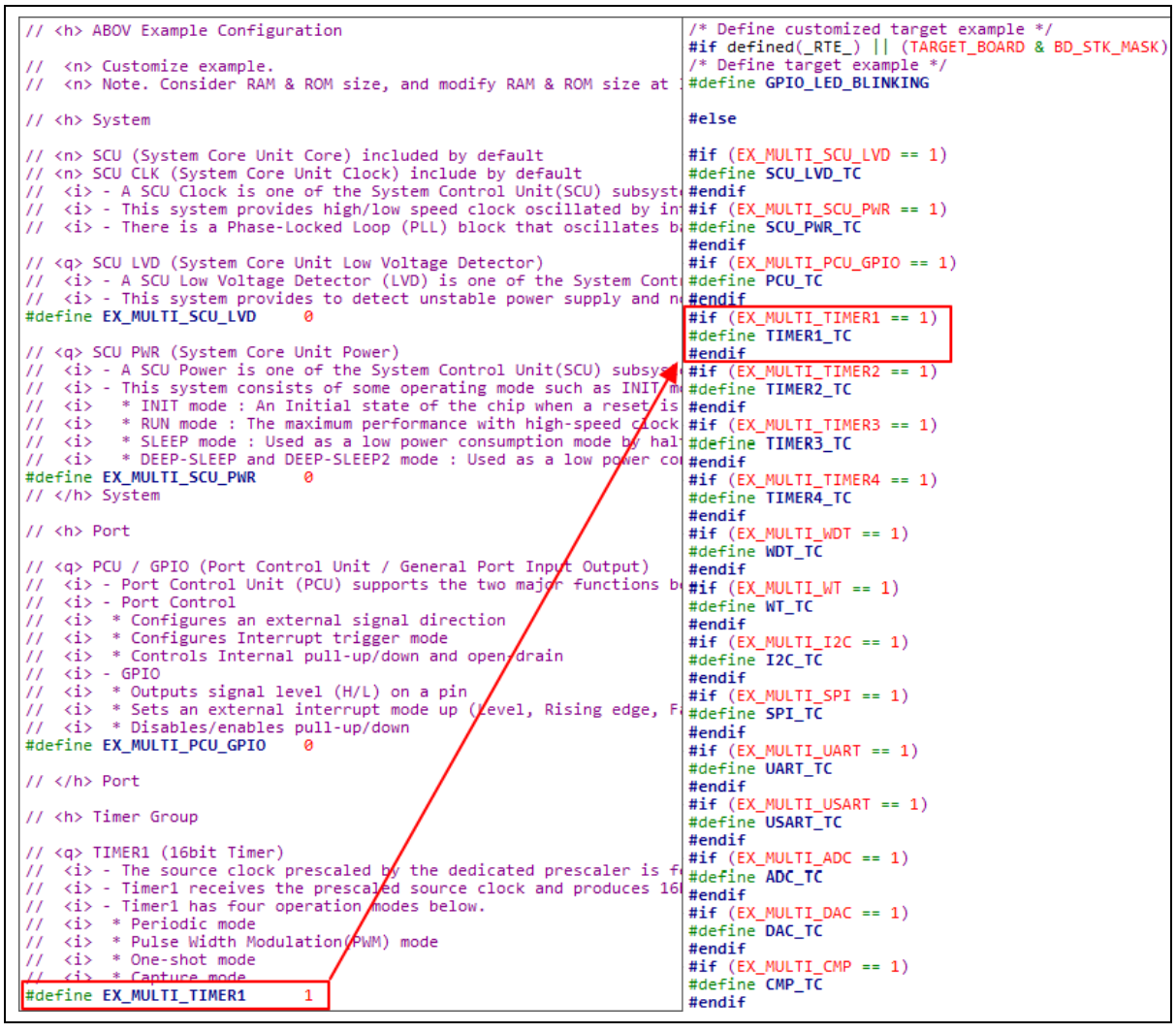
Selecting multiple examples is supported and the detailed procedure for it is shown below.

When you first open the IAR workspace file, you will see that the two examples (SCU and SUC CLK) are embedded and selected by default.
You must not exclude these examples because they are fundamental and essential for providing the other examples with a runtime environment on the target board.

Examples are grouped by ‘System’, ‘Port’, ‘Timer Group’, ‘Connectivity & Communication Group’, ‘Analog Group’, ‘Security & Integrity Group’, ‘Memory Group’ and ‘Display Group’.

Since the annotation wizard is not available in IAR 8.50, there is no way but to use text editor to change any example macro.
If you change a macro value then it will decide which test case will be included which means which example will be compiled.
Figure 44 shows the relation between example identity and test case (example) to build and run.

Figure 44. Selection of multiple examples



6.5 Linker (*.icf) file

Linker configuration file (*.icf) is used to config the location and size of ROM, RAM, Stack, and Heap, and relocate the final outputs generated by the linker. This file in the AUDK32 has default values for each device Sub-Family.

You can change the default values in the linker configuration file but be sure that the changes you make should comply with the specifications of flash memory and SRAM in the user's manual of the target device.

The linker configuration file can be modified using text editor or the linker configuration file editor of IAR IDE. Figure 45 and Figure 46 shows modifying memory options of A31G22x.

Figure 45. Configuration of memory in a31xxxx.icf by text editor

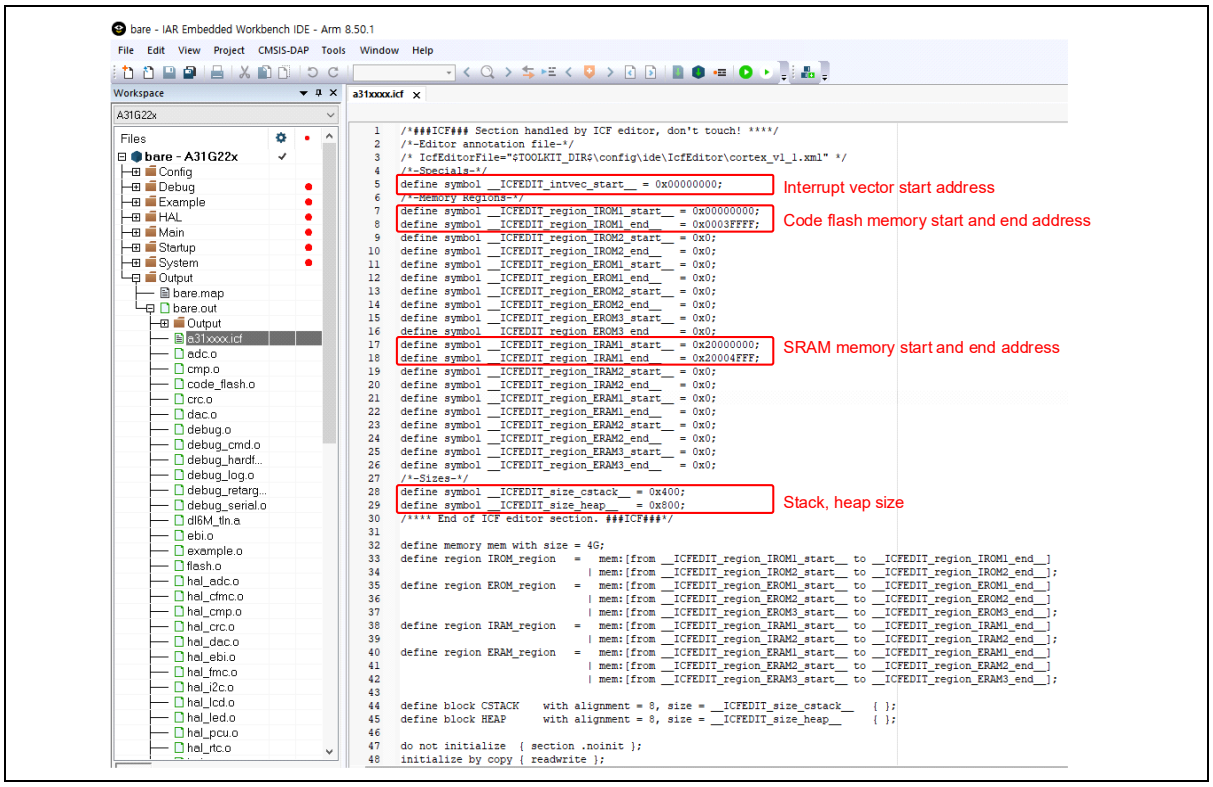
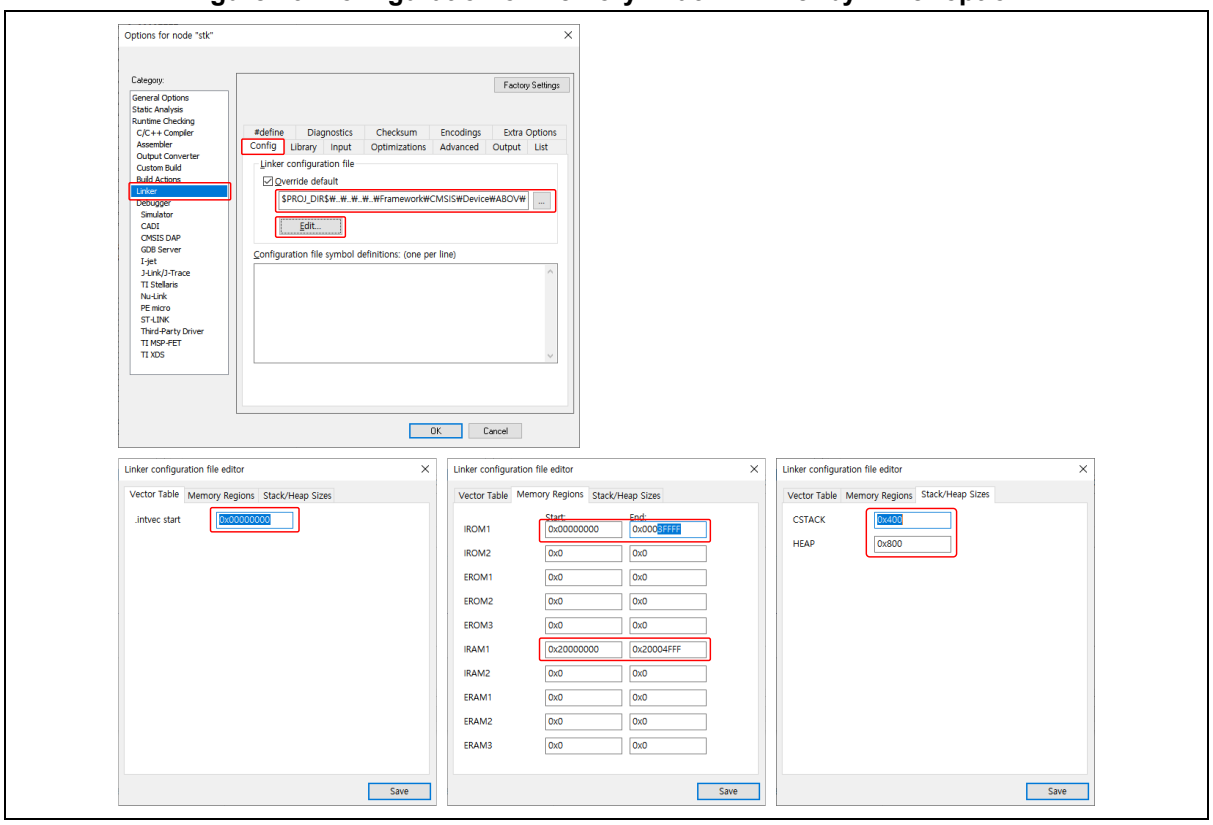


Figure 46. Configuration of memory in a31xxxx.icf by linker option

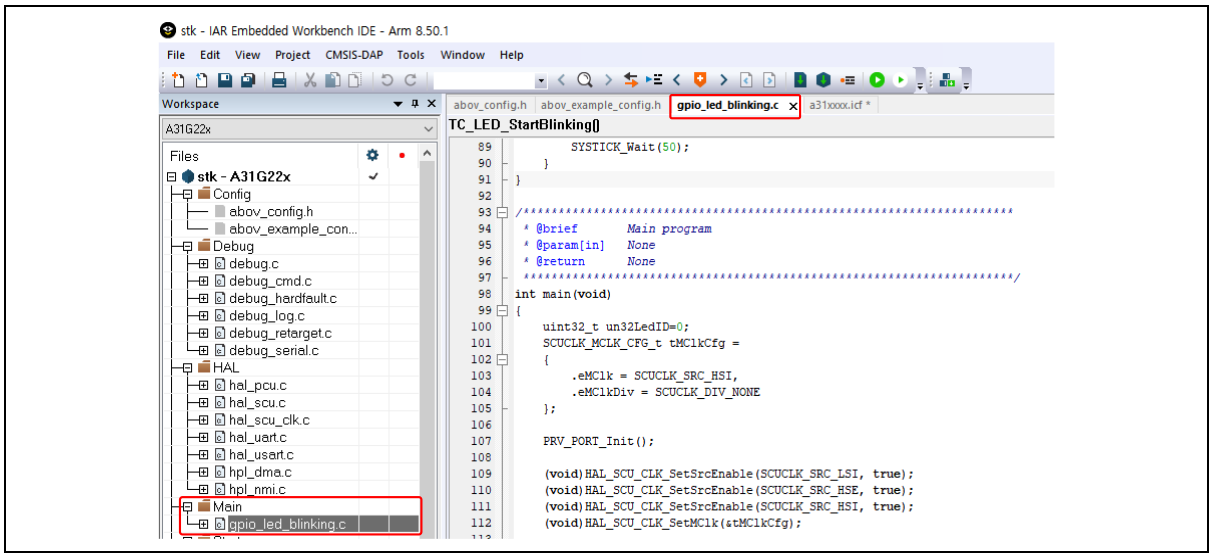


6.6 Build and run example

The AUDK32 provides examples for each target board. Users must check if which examples are supported by the target board first, and then select examples following the description in the previous chapters. There is an example list in the 'abov_example_config.h' file.

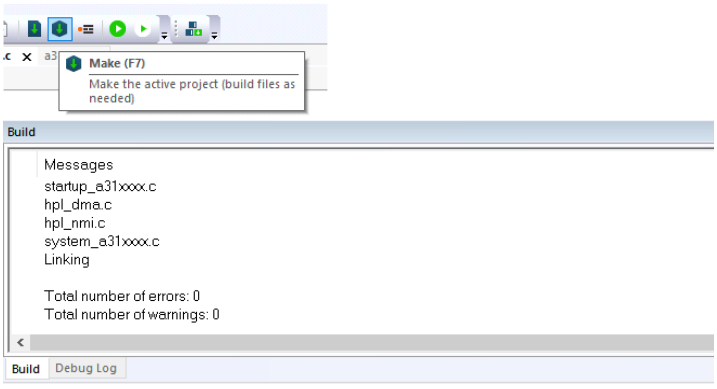
Figure 47 shows that the 'gpio_led_blinking' example in the 'stk.eww' workspace operating on the Starter Kit.

Figure 47. gpio_led_blinking.c example in IAR IDE



If you 'Make' from the Project menu or press 'F7' as shown in Figure 48 in order to start building a project and then you can check the results of the project build in the Build window.

Figure 48. Build target in IAR IDE



Once source codes have been completely compiled without errors, you can now click the menus Project > Download > Download active application, as shown in Figure 49. then, download the binary image file from the build to the target board connected to your PC.

You can check if the file has been successfully downloaded, in the Build Output window.

Figure 49. Download image to flash memory in IAR IDE

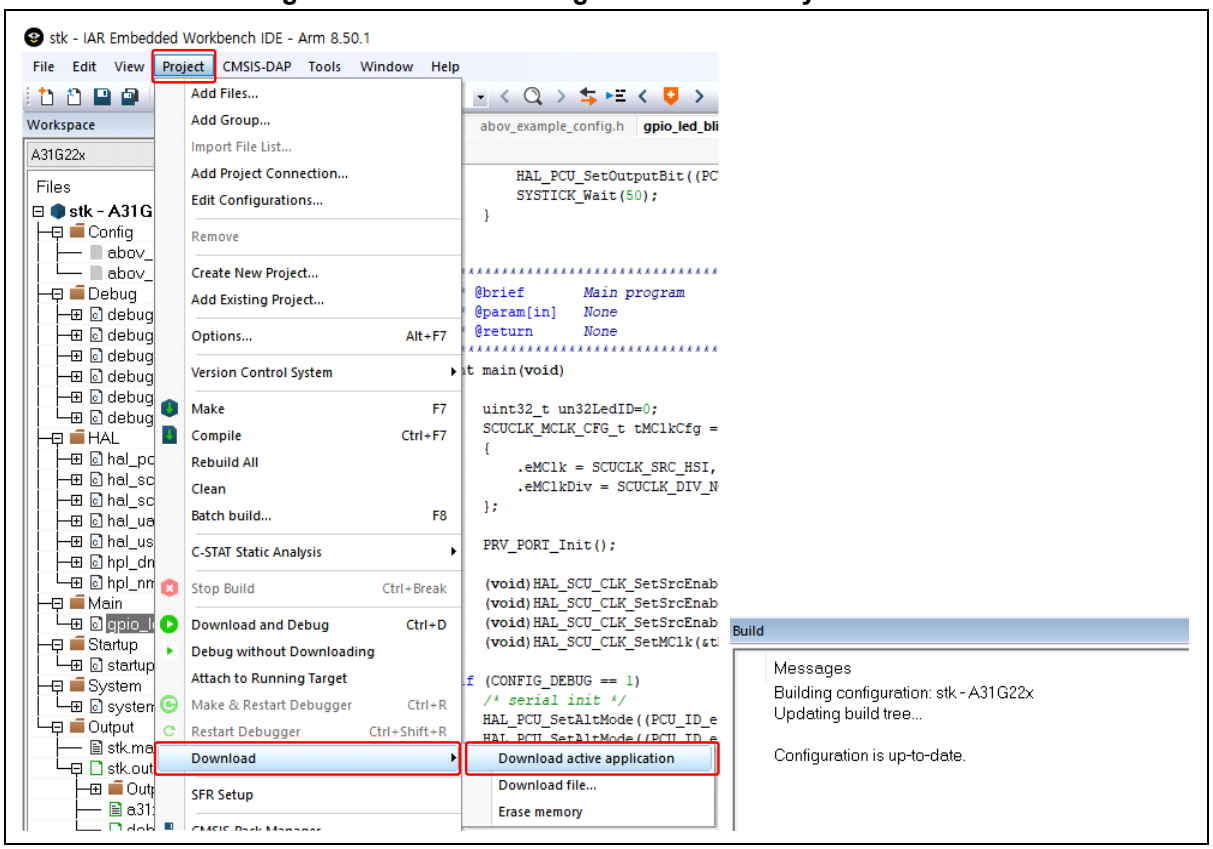
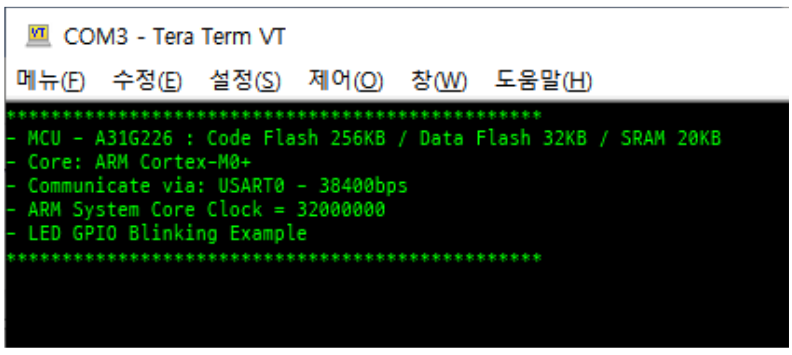


Figure 50 shows that the output of 'gpio_led_blinking' example is downloaded and running on the Starter Kit.

Figure 50. State of gpio_led_blinking example

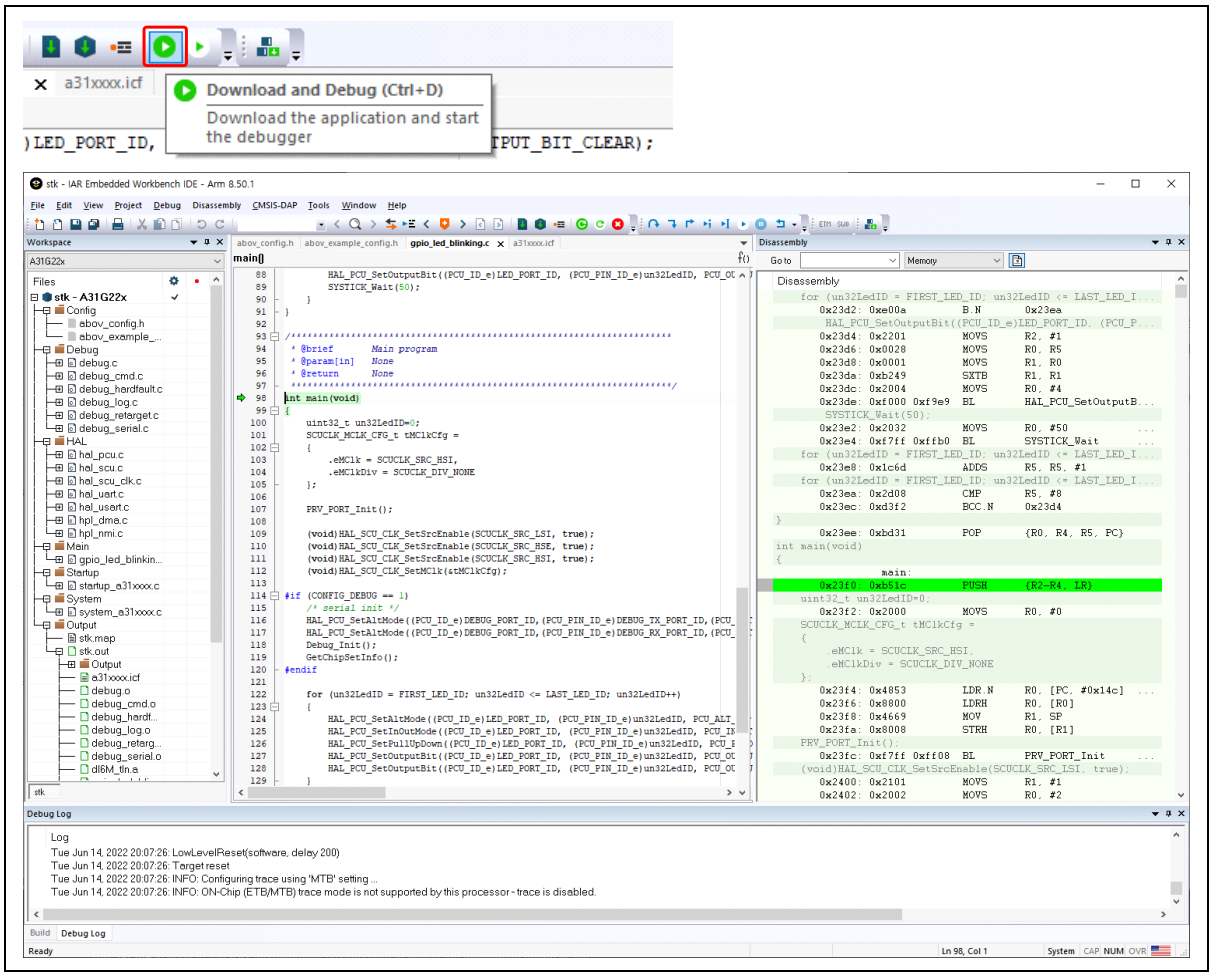


6.7 Debugging with A-Link

The AUDK32 supports the A-Link debugger based on the CMSIS-DAP interface by default.
The A-Link debugger allows source level debugging like other ARM debugger tools.

To enter debugging session, select Project > Download and Debug or click the icon button shown in Figure 51.

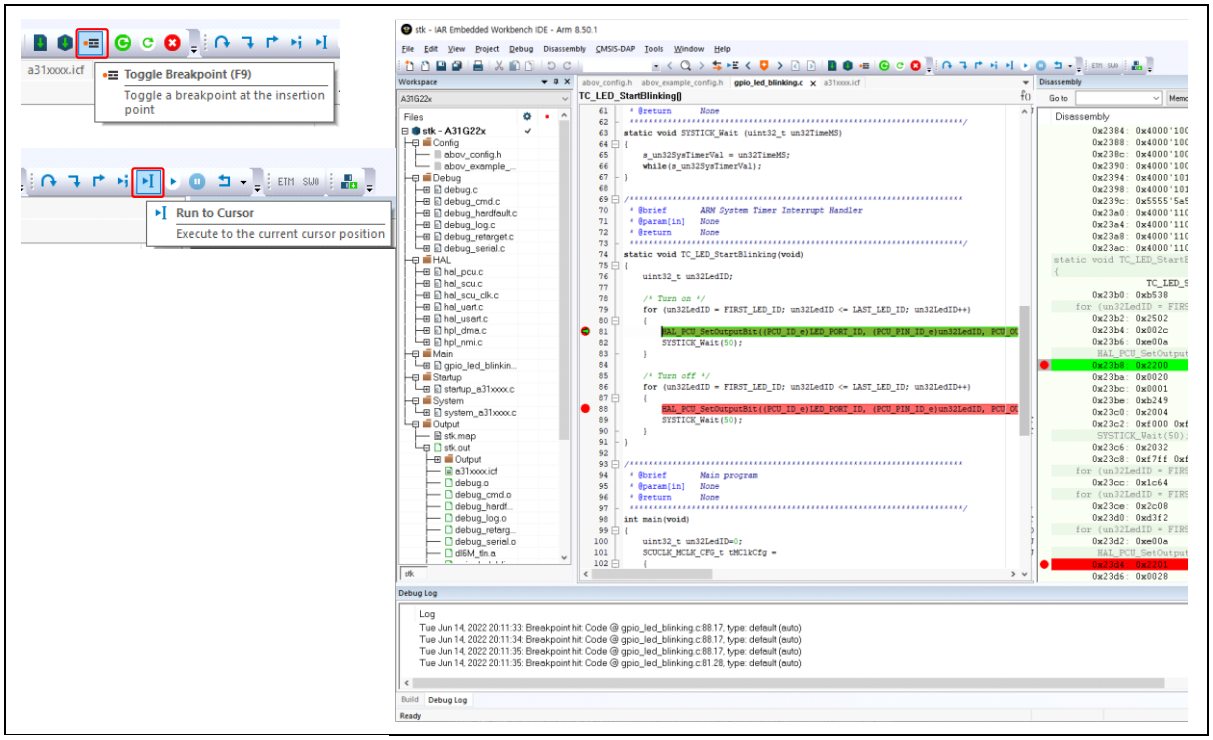
Figure 51. Debug session of IAR IDE



If you set a break point at the front of the line you want to pause and click the Run icon button, the target device runs up to the line where the break point is set, and then stops as shown in Figure 52.

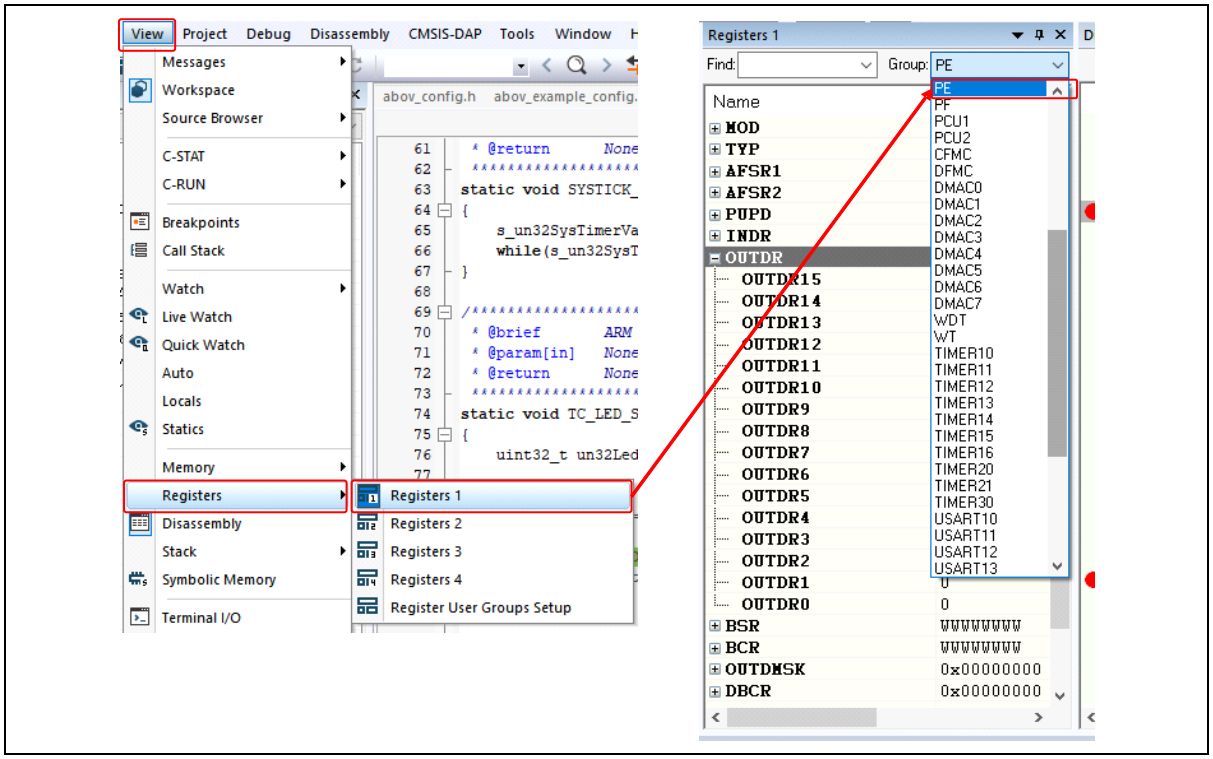
Therefore, you can use a debugger to check and modify specific code such as variables, Special Function Registers (SFRs), and memory for your purpose.

Figure 52. Break point in IAR IDE



As shown in Figure 53, you can check or control register values in real time by loading the System Viewer Description (SVD) file of the target device. The SVD file is provided by the AUDK32.

Figure 53. System viewer in IAR IDE



7. AUDK32 in Eclipse Embedded C/C++

Chapter 7 describes how to build an example code in Eclipse IDE environment that is provided by the AUDK32 and run it on a target board. The following sections describe how to set up the workspace and AUDK32 configuration files, and procedure for building and running a target example on the Starter Kit or Bare board, respectively.

For more information about Eclipse Embedded C/C++ Developers, refer to the web site below.

- URL: https://projects.eclipse.org/projects/iot.embed-cdt

7.1 Eclipse Workspace

Even if IAR IDE provides a single file as a workspace, eclipse IDE provides meta files in '.metadata' directory instead of a single file and the workspace in AUDK32 is supposed to define or have a single project only.

The workspace directories are provided on a board basis, and the location of them is as follows:

- Starter Kit board: ~/Example/Build/Eclipse/STK/Workspace
- Bare board: ~/Example/Build/Eclipse/Bare/Workspace

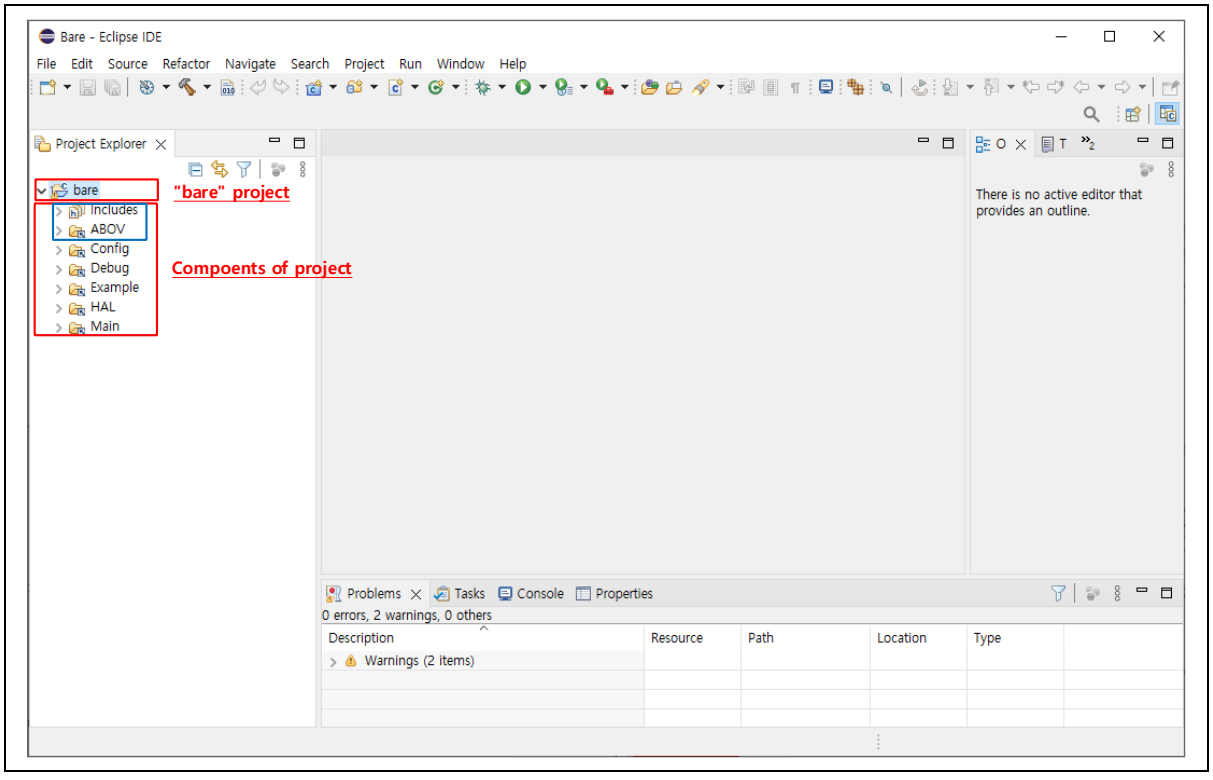
When it comes to installing and setting up a workspace and project by default, please refer to the Eclipse IDE for Embedded C/C++ Developers in the appendix.

7.1.1 Open workspace and project

A workspace used to have multiple projects inside generally, but it is supposed to have a single project only in AUDK32.

As described, there are only two eclipse workspaces in AUDK32 currently. It is SKT and Bare.
For more workspace and project details in eclipse IDE, refer to 'Setup workspace' in the appendix.

Figure 54. Bare workspace and bare project in Eclipse IDE



If AUDK32 eclipse project is opened, the IDE shows that the project consists of seven groups: Includes, ABOV, Config, Debug, Example, HAL and Main.
The most modules are same with other IDEs (KEIL and IAR) except for 'includes and ABOV' that are made up according to the policy of eclipse in AUDK32.

Before selecting and building examples, you must modify files in the Config group appropriately and then select the target Sub-Family which starts build codes automatically.

Follow the procedures below for this purpose:

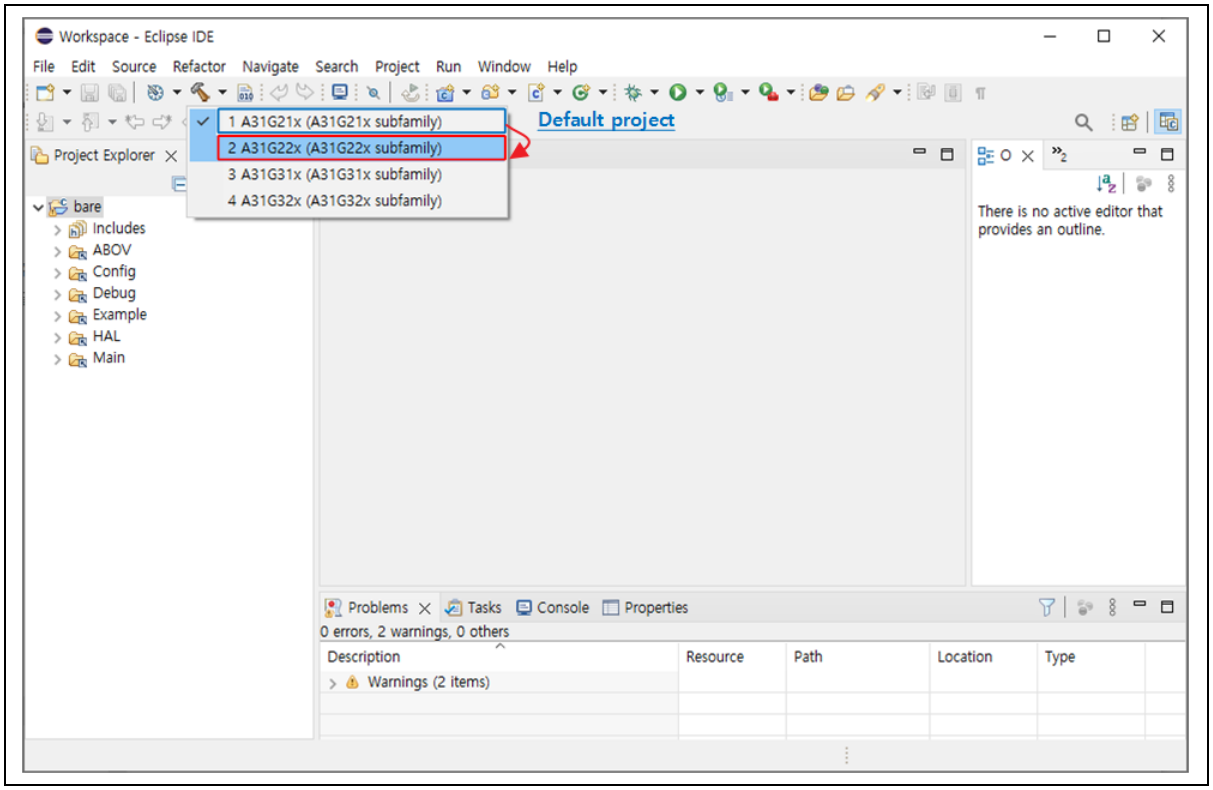
1. Modify the ‘abov_config.h’ file to select the target board and configure a debug channel.
2. Modify the ‘abov_example_config.h’ file to select example to build.
3. Check if the scatter loader files (*.ld) are proper for the target application.
4. Build code by selecting a target Sub-Family, ‘Build All’ from Project menu or press ‘Ctrl + B’.

7.2 Select target

Selecting a target Sub-Family and building codes does work separately in other IDEs but Eclipse IDE does not discriminate them. It means, if a target Sub-Family is selected, building codes starts automatically.
Therefore selecting target must be started after all required configurations for a target example and board are done.

When you first open bare project in Bare workspace, the target Sub-Family in the workspace is set to 'A31G21x' by default. You can change the target Sub-Family from the drop-down menu shown in Figure 55. But be sure, before selecting or changing the target, all required configuration should be done because selecting or changing a target means, building codes.

Figure 55. Changing target Sub-Family



7.3 abov_config.h

The ‘abov_config.h’ is not a normal header file but a top-level configuration file written in CMSIS annotation syntax and defines the configuration macros associated with the target board and debug channel. Once the configuration macros are defined, another configuration files related with the target board and debug channel are included automatically.

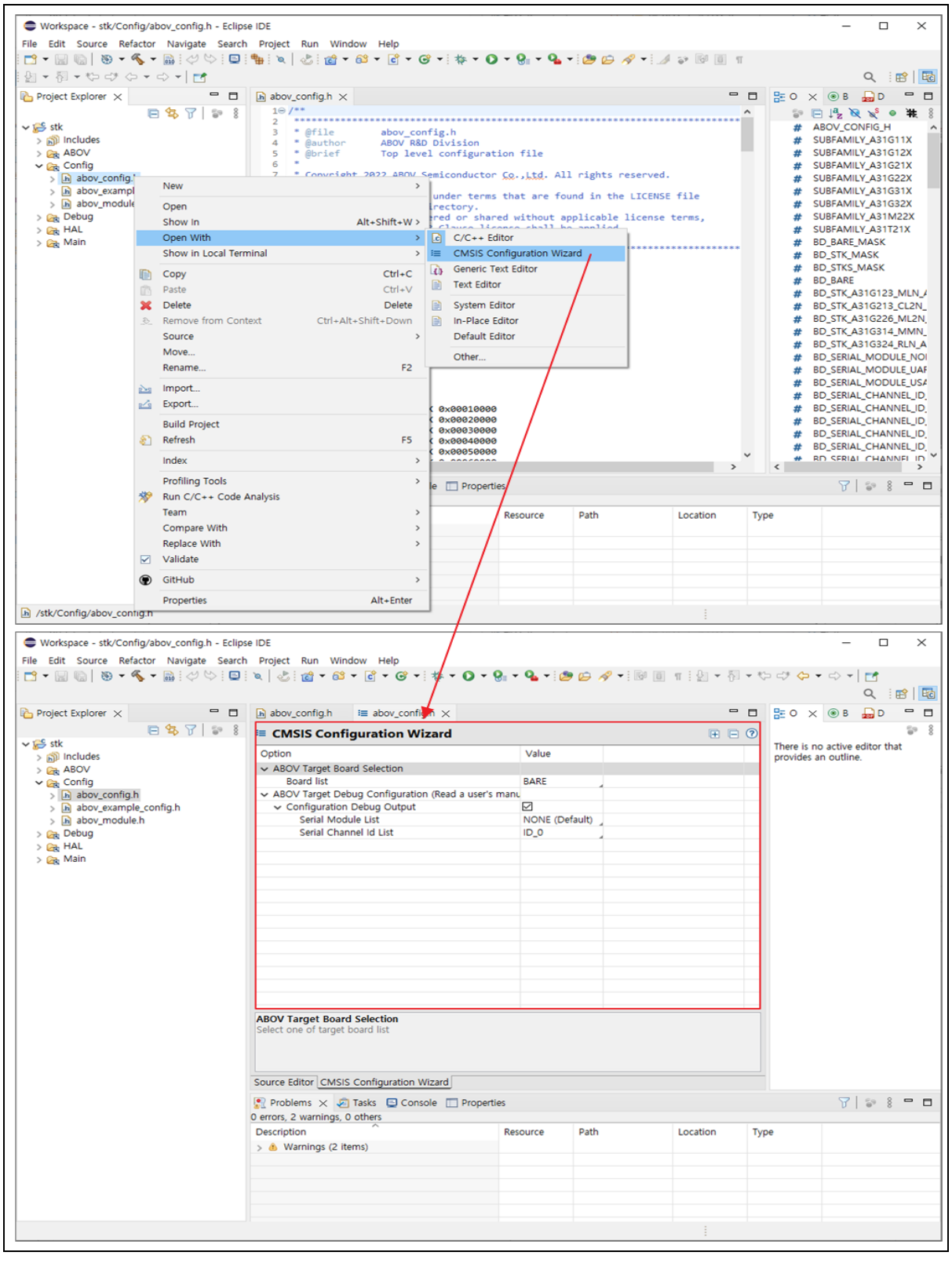
When you first open the ‘abov_config.h’ file by using ‘CMSIS Configuration Wizard’ or Text editor, you will see the configuration macros in the file are set to default values already.
‘CMSIS Configuration Wizard’ is a sort of configuration tool like a ‘KEIL Annotation Wizard’.
Users simply need to set up the target board and debug channel by using the CMSIS Configuration Wizard.

‘abov_config.h’ can be edited by Text editor typically but also changed by CMSIS Configuration Wizard with more easier approach. See Figure 56.

7.3.1 CMSIS Configuration Wizard

CMSIS Configuration Wizard decodes ‘abov_config.h’ and displays it in better readable format.

Figure 56. CMSIS Configuration Wizard



7.3.2 Select Starter Kit

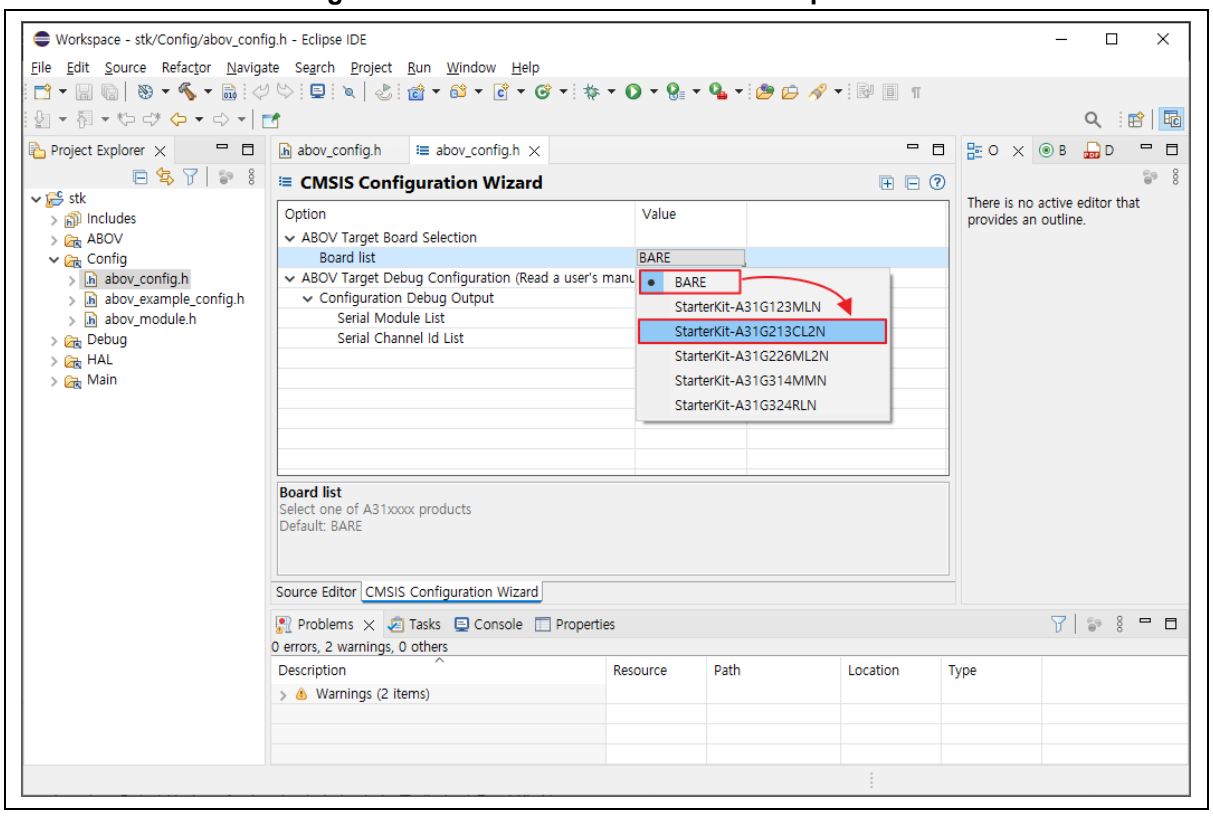
The ‘stk’ project in ‘STK’ workspace is supposed to configure and build the examples supported by Starter Kit.

However the default board in abov_config.h is ‘BARE’. Therefore, you must select the target Starter Kit from the drop-down menu shown in For now, only ‘GPIO_LED_BLINKING’ is supported by Starter Kit.

Figure 57.

For now, only ‘GPIO_LED_BLINKING’ is supported by Starter Kit.

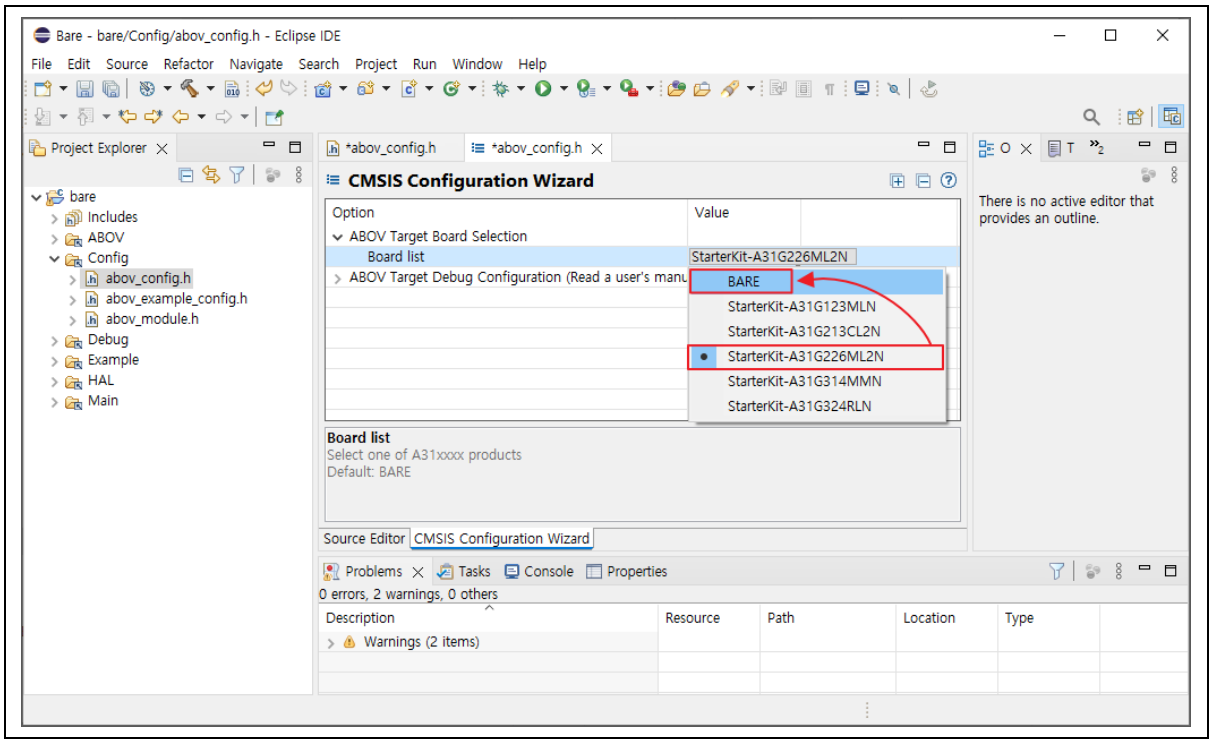
Figure 57. Selection of Starter Kit in Eclipse IDE



7.3.3 Select BARE

The ‘bare’ project in ‘Bare’ workspace is supposed to verify on-chip peripherals which are provided by Bare board. Therefore, you must select ‘BARE’ from the ‘Board list’ drop-down menu, as shown in Figure 58.

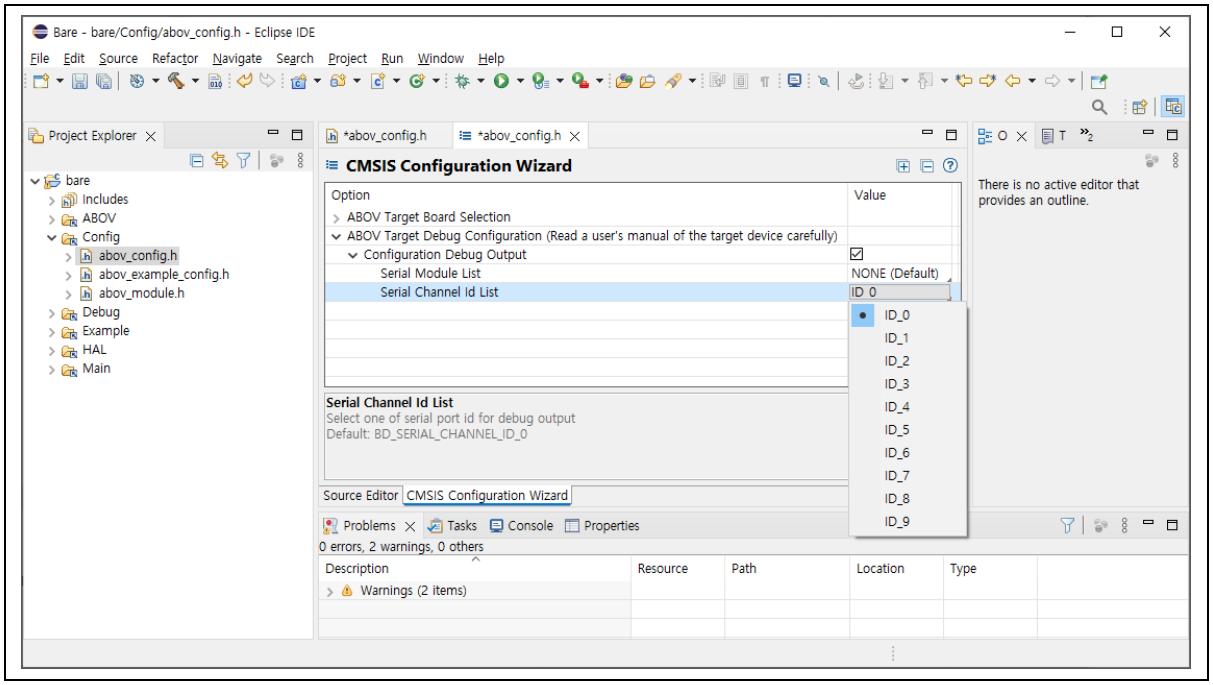
Figure 58. Selection of Bare board in Eclipse IDE



7.3.4 Config debug channel

The ‘abov_config.h’ also shows settings of a current debug channel of the target device selected before. If you take a look at ‘abov_config.h’, you can find out macros related to debug channel as shown Figure 59.

Figure 59. Configure Debug channel



You can set which debug channel on which debug module as for serial communication referring to a layout on the target board and user’s manual of the target device.

7.4 abov_example_config.h

The 'abov_example_config.h' is also written in CMSIS annotation syntax, similar to 'abov_config.h,' and is used to select an example supported by the previously chosen target board.

You can designate the target example to run on the selected board by setting the identity value of an example in this file.This change can be done by Text editor or the CMSIS Configuration Wizard.

7.4.1 Example selection of Starter Kit

Figure 60 shows once user select Starter Kit as a target board, ‘GPIO_LED_BLINKING’ is taken as a target example to test automatically.
‘GPIO_LED_BLINKING’ is a representative example of Starter Kit and the others are not supported yet. Therefore, user should not change the target example.

Figure 60. GPIO_LED_BLINKING in case of Starter Kit board

```
/* Define customized target example */
#if defined(_RTE_) || (TARGET_BOARD & BD_STK_MASK)
/* Define target example */
#define GPIO_LED_BLINKING

#else

#if (EX_MULTI_SCU_LVD == 1)
#define SCU_LVD_TC
#endif
#if (EX_MULTI_SCU_PWR == 1)
#define SCU_PWR_TC
#endif
#if (EX_MULTI_PCU_GPIO == 1)
#define PCU_TC
#endif
#if (EX_MULTI_TIMER1 == 1)
#define TIMER1_TC
#endif
#if (EX_MULTI_TIMER2 == 1)
#define TIMER2_TC
#endif
#if (EX_MULTI_TIMER3 == 1)
#define TIMER3_TC
#endif
#if (EX_MULTI_TIMER4 == 1)
#define TIMER4_TC
#endif
#if (EX_MULTI_WDT == 1)
#define WDT_TC
#endif
#if (EX_MULTI_WT == 1)
#define WT_TC
#endif
#if (EX_MULTI_I2C == 1)
#define I2C_TC
#endif
```

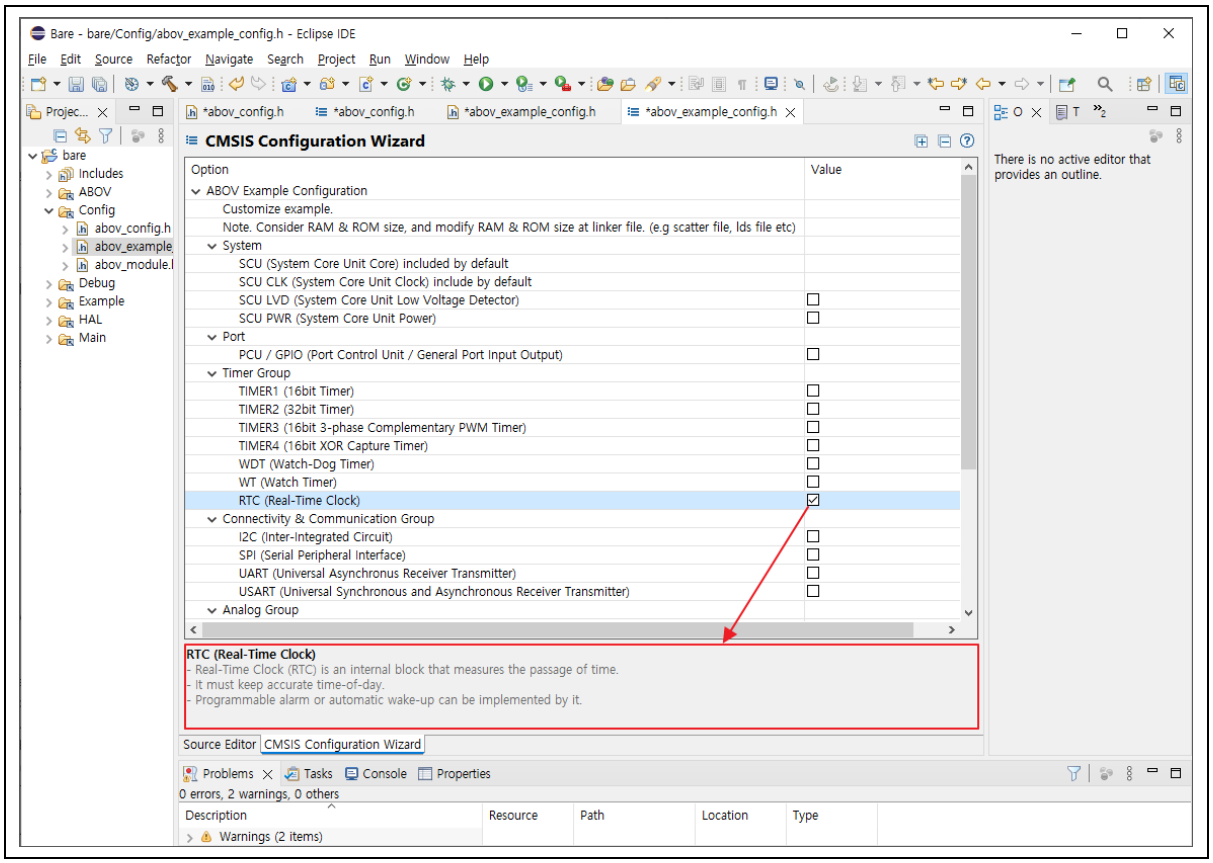
7.4.2 Multiple examples selection in Bare board

Bare board supports multiple examples running on itself and selecting multiple examples is also supported by AUDK32.
The detailed procedure for selecting examples is shown below.

When you open the 'bare' project in 'Bare' workspace, the two examples (SCU and SUC CLK) are embedded and selected by default in 'abov_example_config.h'.
You must not exclude these examples because they are fundamental and essential for providing runtime environment on the target board with the other examples.

Examples are grouped by 'System', 'Port', 'Timer Group', 'Connectivity & Communication Group', 'Analog Group', 'Security & Integrity Group', 'Memory Group' and 'Display Group'.
If you tick a check box of a group, examples belonging to it are expanded and displayed as Figure 61 below. You only need to choose each example by ticking check box of it.
If you click over an example for each peripheral, main features and summarized descriptions for the peripheral are usually displayed in the annotation field at the bottom of the CMSIS Configuration Wizard.

Figure 61. Selection of multiple examples



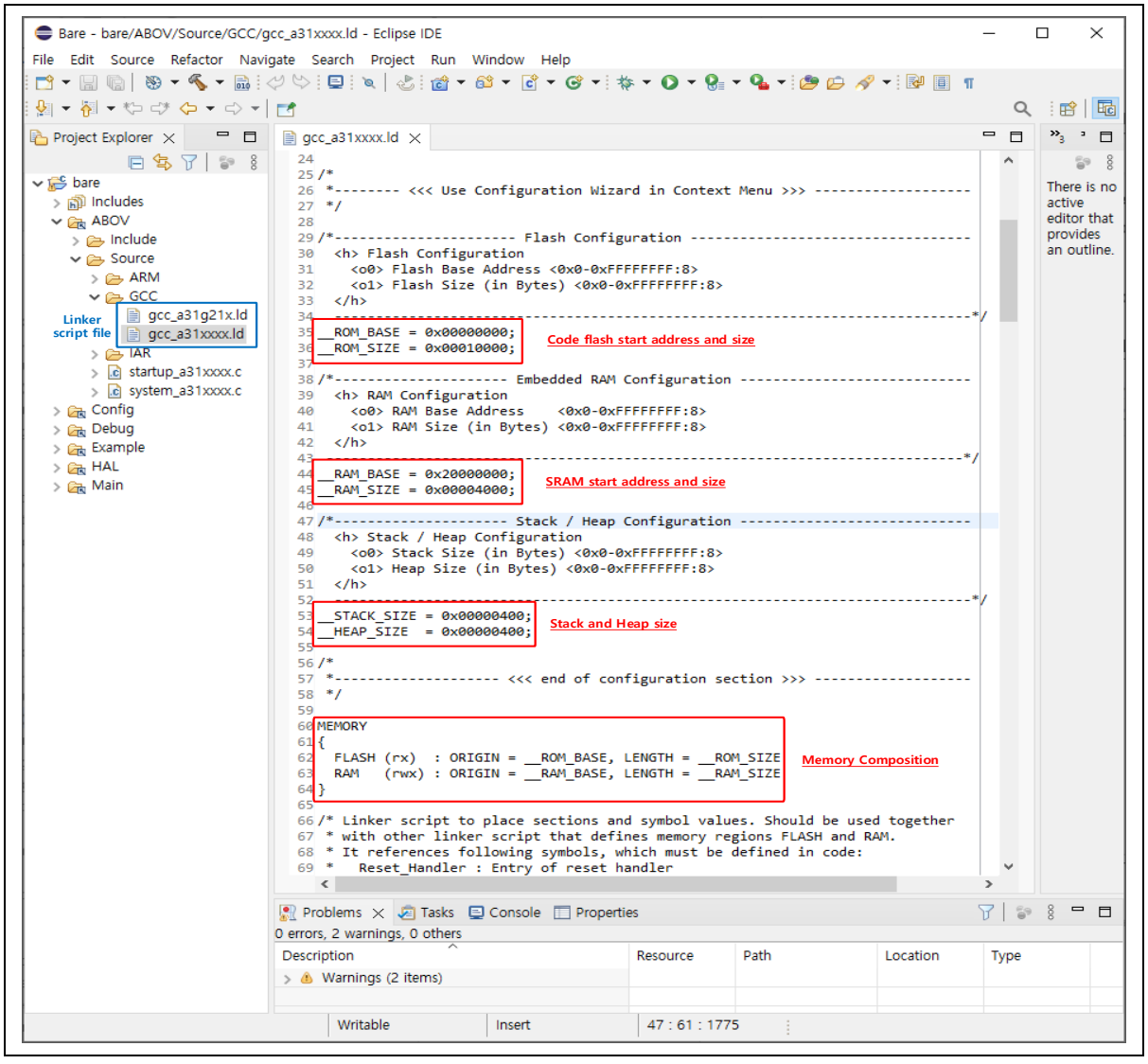
7.5 Linker (*.ld) file

GNU Linker script file (*.ld) is used to config the location and size of ROM, RAM, Stack, and Heap, and relocate the final outputs generated by the linker. This file in the AUDK32 has default values for each device Sub-Family.

You can change the default values in the linker script file by using an integrated text editor but be sure that the changes you make should comply with the specifications of flash memory and SRAM in the user's manual of the target device.

Figure 62 shows the multiple GNU linker script files. Normally a single common linker script file is supposed to cover all devices in its Family but a special linker script file dedicated to a certain device could be needed due to very small size of SRAM or Flash.
For example, there are 'gcc_a31xxxx.ld' that covers all devices in A31xxxx Family except for A31G21x, and 'gcc_a31g21x.ld' dedicated to A31G21x.

Figure 62. Configuration of memory in gcc_a31xxxx.ld

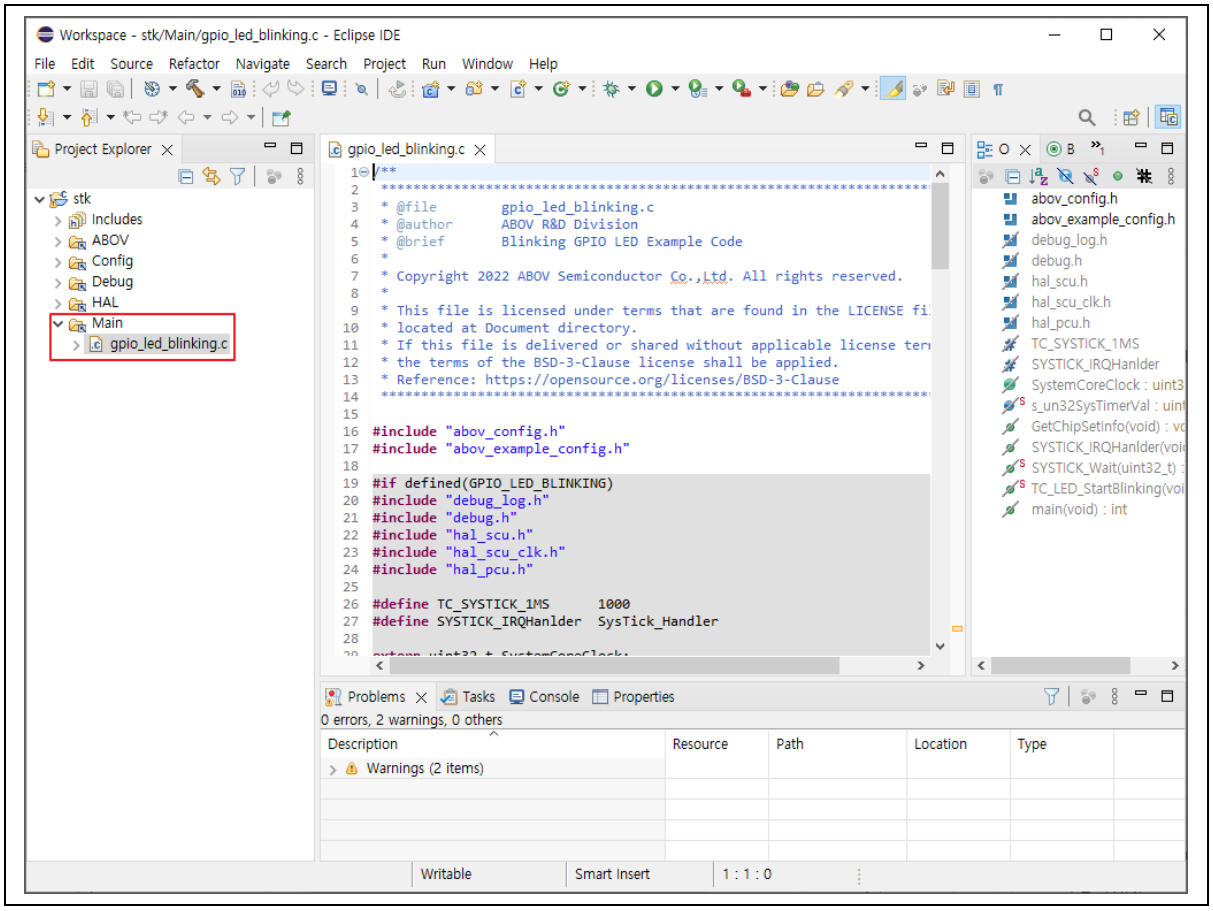


7.6 Build and run example

The AUDK32 provides examples for each target board. Users must check if which examples are supported by the target board first, and then select examples following the description in the previous chapters. There is an example list in the 'abov_example_config.h' file.

Figure 63 shows that the 'gpio_led_blinking' example in the 'stk.eww' project operating on the Starter Kit.

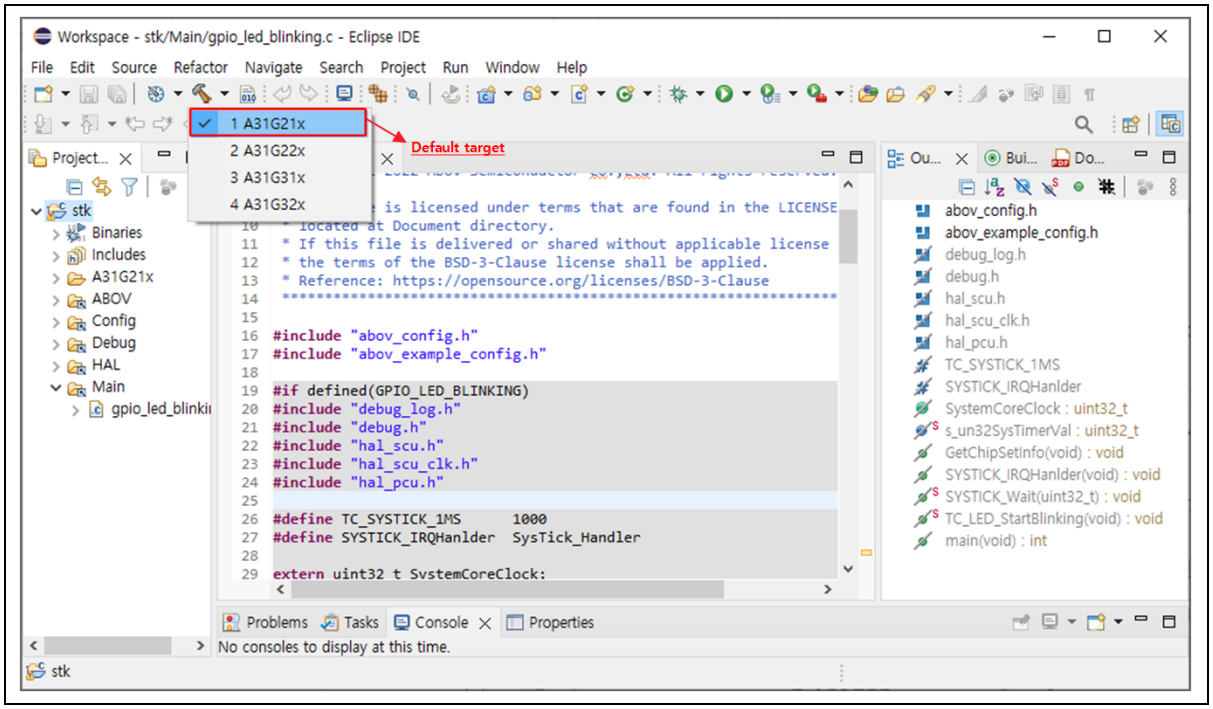
Figure 63. gpio_led_blinking.c example in Eclipse IDE



7.6.1 Build target

Once the proper changes on ‘abov_config.h’ and ‘abov_example_config.h’ are done, you are ready to build codes. If you click the arrow icon beside of ‘build’ button, the available targets are listed up and you can see the default target in each device Family. For example, A31G21x is a default target in A31xxxx Family.

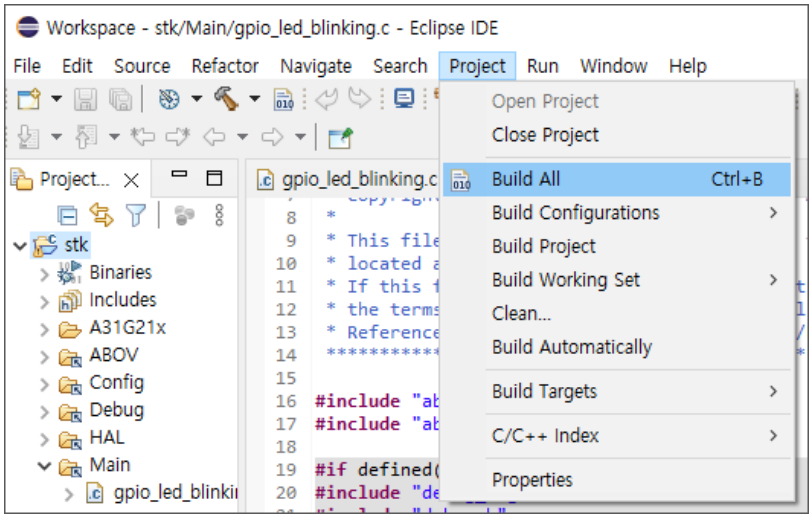
Figure 64. Target list in Eclipse IDE



Now, if you select one of the targets, building the target will be started.

If you ‘Build All’ from the Project menu or press ‘Ctrl + B’ as shown in Figure 65, Building code starts with the selected target lastly. However, building code also starts as soon as the target is changed in Figure 64.

Figure 65. Build target in Eclipse IDE



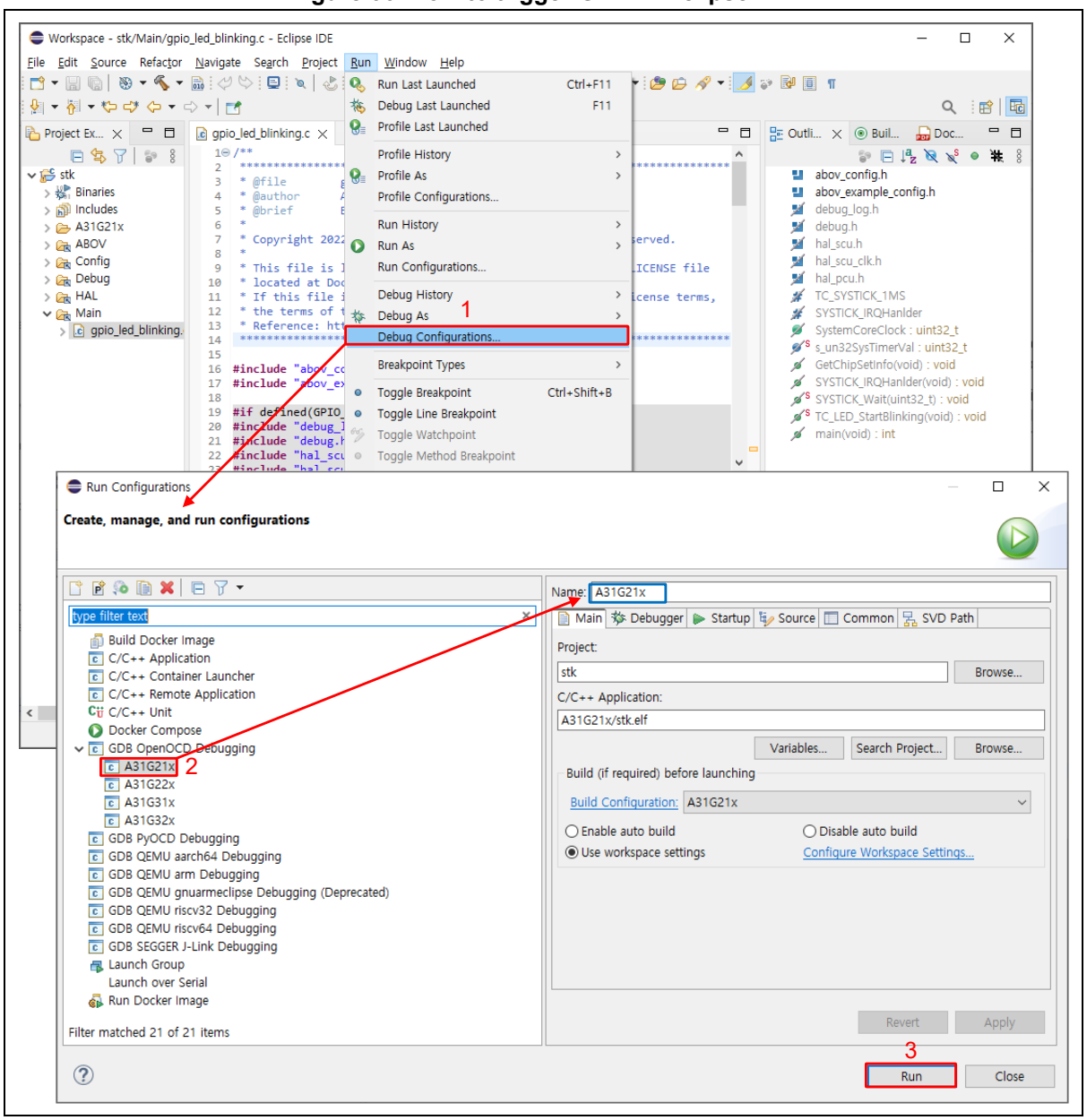
7.6.2 Program binary into device

There is no 'Flash Programmer IDE' in eclipse by default. For this, a special programming host tool must be embedded into eclipse IDE. For example, 'CodeWarrior IDE'.

However, we can flash a user binary into a target device by using 'gdb' and its extension.
Therefore, if source codes have been completely compiled without errors, you can program the compiled binary output into a target device by the flash programming feature of 'gdb' without IDE.
In other words, programming user binary starts as soon as 'gdb' is triggered to run.

Figure 66 shows how to trigger 'gdb' in eclipse IDE.

Figure 66. How to trigger GDB in Eclipse IDE



All detailed information about the triggering is displayed on ‘Console’ window at bottom during ‘gdb’ debug session is established. If any issue happens, you need to check the information at first.

Figure 67. Debug session information

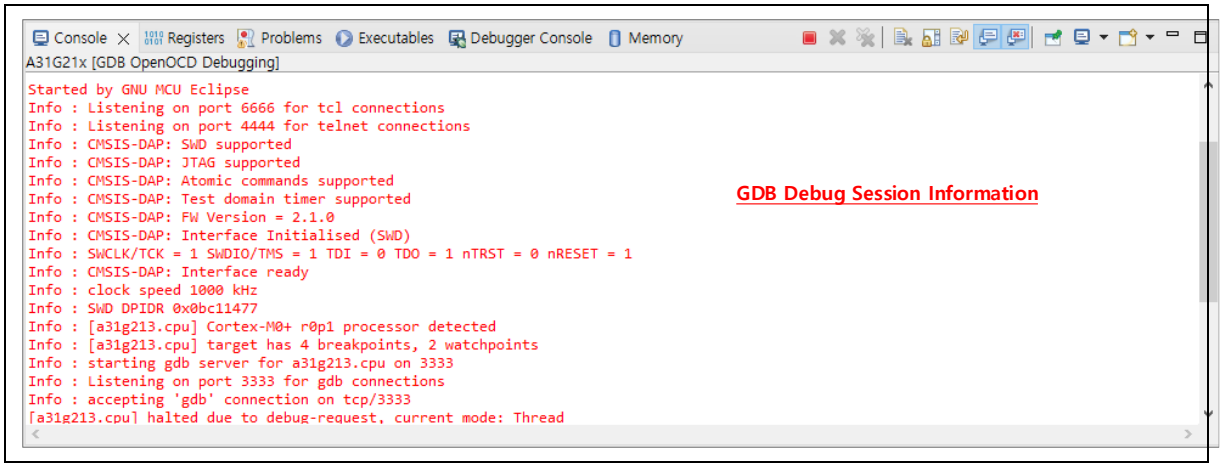
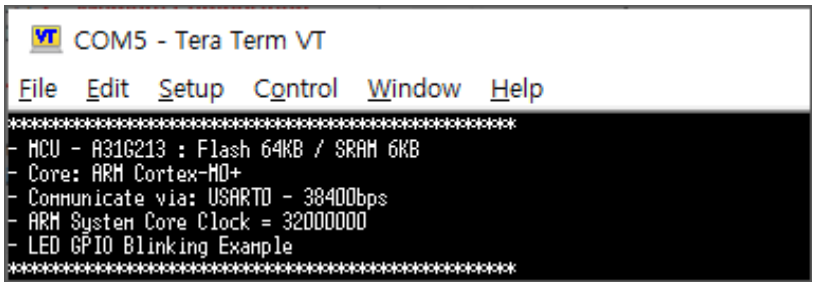


Figure 68 shows that the output of ‘gpio_led_blinking’ example is downloaded and running on the Starter Kit by GDB and you can also see if all LEDs on Starter Kit is blinking sequentially or not.

Figure 68. State of gpio_led_blinking example



7.7 Debugging with A-Link

AUDK32 supports the A-Link debugger based on the CMSIS-DAP interface by default.
The A-Link debugger allows source level debugging like other ARM debugger tools.
To enter debugging session, refer to Figure 66 above.

Once ‘gdb’ debug session operates completely, you can do source level debugging and check values of peripheral registers.

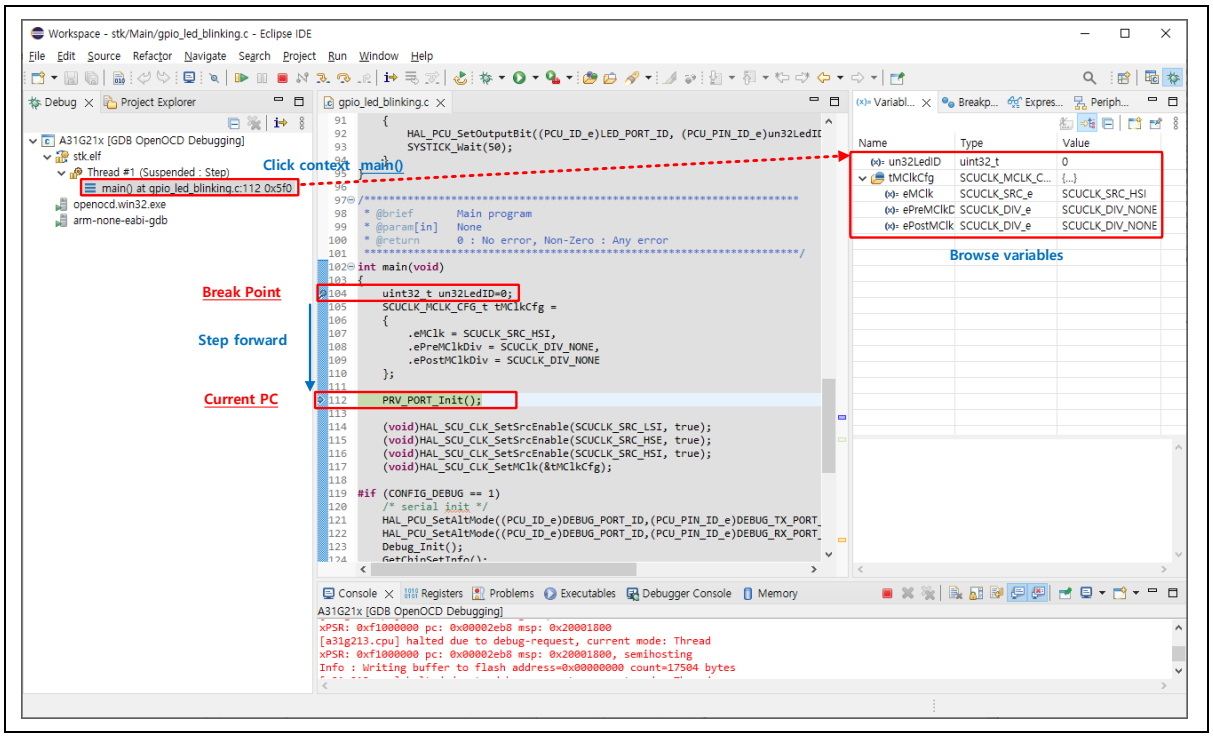
7.7.1 Source level debugging

All required debug features are supported like setting ‘Break point’ up, disabling ‘Break point’, Step into and so on including browsing variables.

Debug commands are available from the context menu on the blue area in Figure 69 below or Run menu.
For example, if you want to set a break point at a certain line, you just need to click a right button of your mouse at the same line of your targeted line in the blue area to display the context menu and then click ‘Add breakpoints’ (Ctrl + Double click). New break point will be appeared.

If you click the context to debug in ‘Debug’ tab on the left-side panel, you can also check the value of all variables related to the context in ‘Variables’ tab on the right-side panel.

Figure 69. Source level debugging in Eclipse IDE



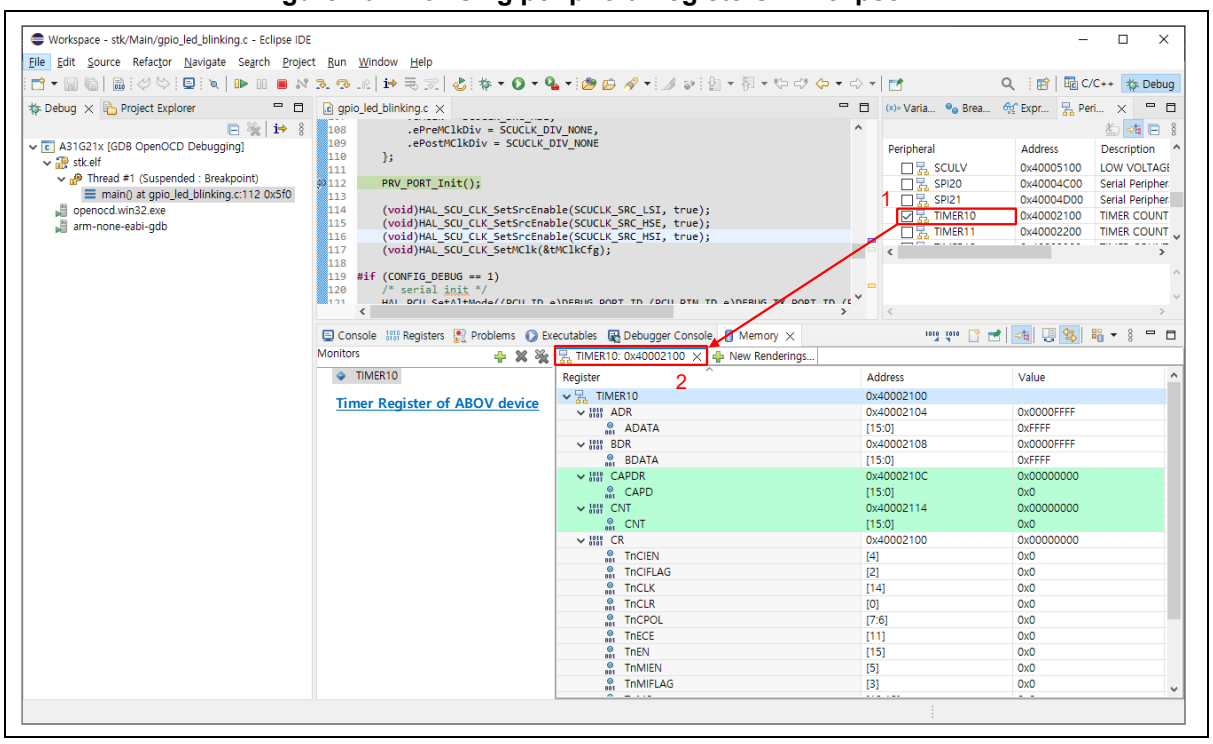
7.7.2 Browsing peripherals registers

As described, if you set a break point at the front of the line you want to pause and click the Resume icon button, the target device runs up to the line where the break point is set, and then stops.

Therefore, you can use a debugger to check and modify specific code such as variables, Special Function Registers (SFRs), and memory for your purpose.

Just in case of peripherals registers, if peripheral H/W modules are selected in 'Peripherals' tab, the corresponding register set will be shown as Figure 70.

Figure 70. Browsing peripheral registers in Eclipse IDE



8. Serial debug configuration

Debug messages from the applications running on the target board are output to the PC terminal program through a serial debug channel.
The serial debug channel on the target board can be either UART or USART, depending on the board's characteristics. It is configured with interrupt mode as the default setting.

Chapter 8 describes how to set a debug channel.

8.1 Default serial debug channel of Starter Kit

The serial debug channel on the Starter Kit is connected to the Serial-to-USB of the A-Link and works with the terminal program of the PC.

As describe above, the serial debug channel on the Starter Kit can use either UART or USART as a default channel, depending on the board characteristics.
For example, A31G22x Starter Kit uses USART as its default channel.

For pins assigned to serial debugging on the Starter Kit, refer to the Starter Kit Manual for each device.

Figure 71. Serial debug interface of A31G22x Starter Kit

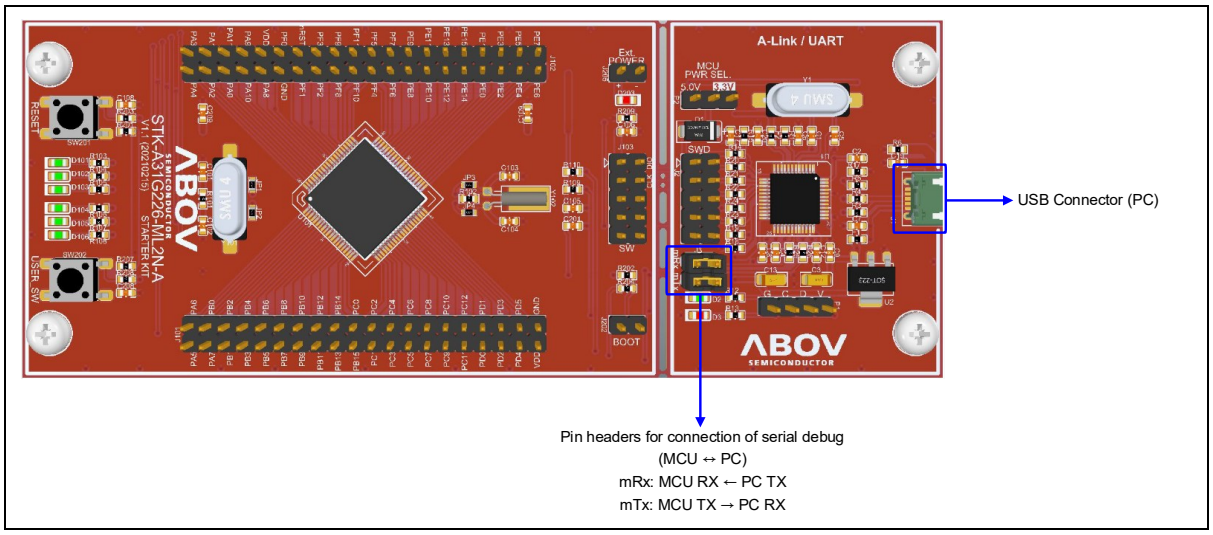
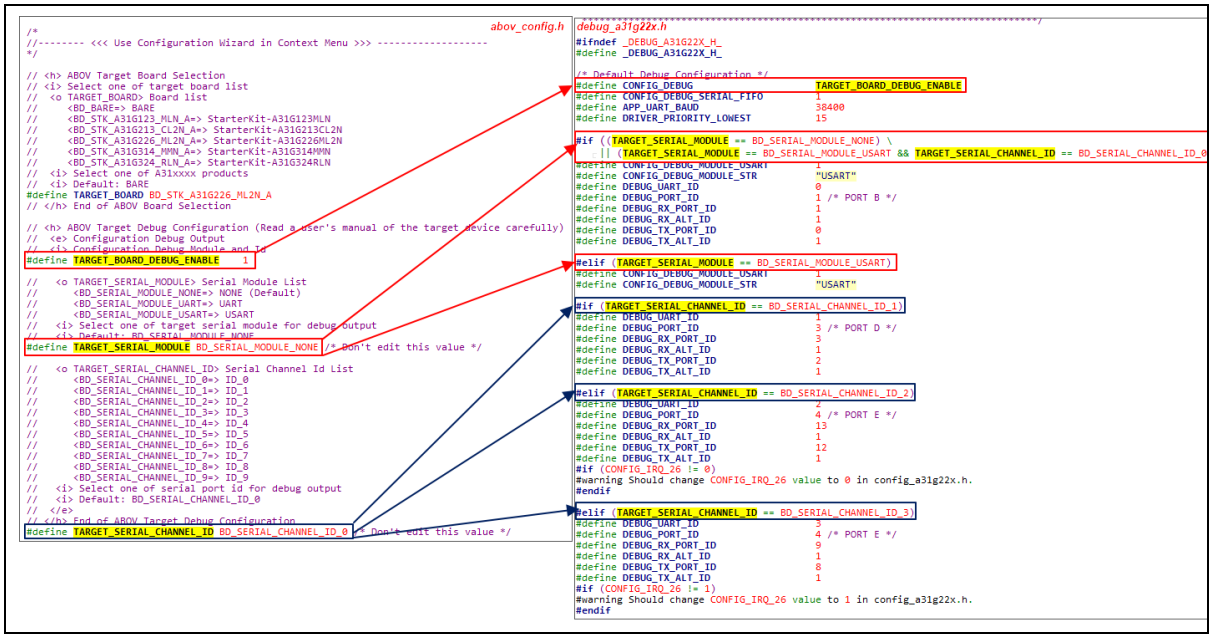


Figure 72 shows the settings in the header file ‘debug_ a31g22x.h’ that holds configuration macros of which values are defined by the debug configuration macros in ‘abov_config.h’.
It means, the top-level configuration file ‘abov_config.h’ provides users with debug configuration macros that impact on actual debug configuration file ‘debug_a31g22x.h’.
Therefore users need to focus on the top-level configuration file only.

Figure 72. Serial debug configuration in AUDK32



8.2 Changing serial debug channel

To use another UART or USART as a serial debug channel on a target board, additional equipment such as UART-to-USB converter is required, and the header file 'debug_[Sub-Family].h' located at the '~\ProductConfig\[Sub-Family]\Config\' directory must be modified for the serial debug channel you want to. but as described, user has to change the top-level configuration file 'abov_config.h' instead of 'debug_[Sub-Family].h'.

For example, If the debug channel on the StarterKit-A31G226ML2N board is changed to USART of port D as a new debug channel, you must configure the hardware as shown in Figure 73 and change the 'abov_configh' configuration file as shown in Figure 74.

Figure 73. H/W connection to change debug channel

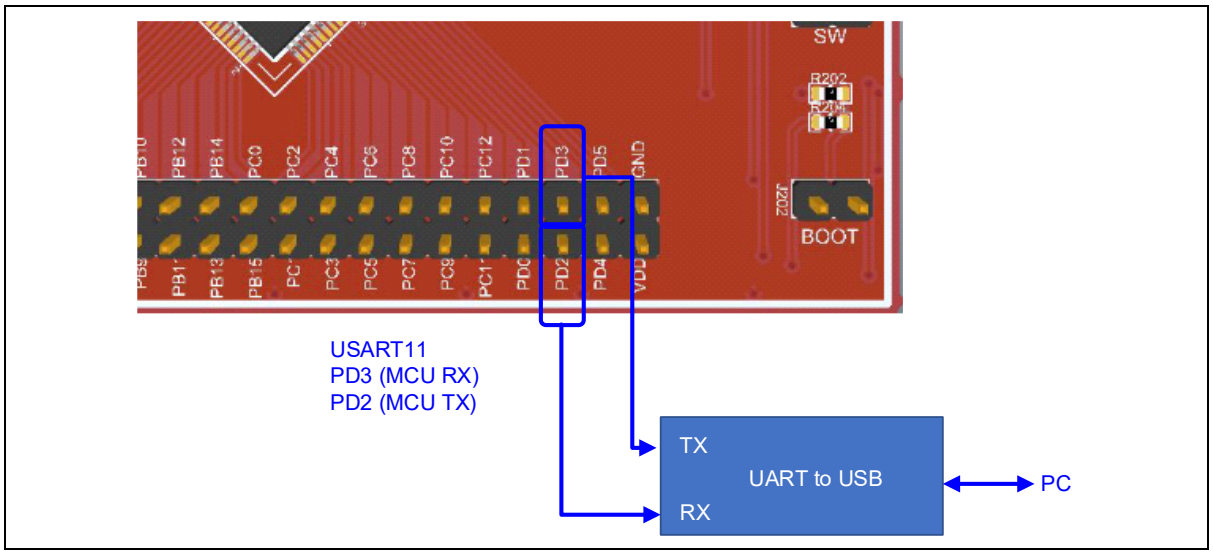


Figure 74. Modify 'abov_config.h' to switch a new debug channel

```
//    <o TARGET_SERIAL_MODULE> Serial Module List
//       <BD_SERIAL_MODULE_NONE=> NONE (Default)
//       <BD_SERIAL_MODULE_UART=> UART
//       <BD_SERIAL_MODULE_USART=> USART
//    <i> Select one of target serial module for debug output
//    <i> Default: BD_SERIAL_MODULE_NONE
#define TARGET_SERIAL_MODULE BD_SERIAL_MODULE_USART /* Don't edit this value */

//    <o TARGET_SERIAL_CHANNEL_ID> Serial Channel Id List
//       <BD_SERIAL_CHANNEL_ID_0=> ID_0
//       <BD_SERIAL_CHANNEL_ID_1=> ID_1
//       <BD_SERIAL_CHANNEL_ID_2=> ID_2
//       <BD_SERIAL_CHANNEL_ID_3=> ID_3
//       <BD_SERIAL_CHANNEL_ID_4=> ID_4
//       <BD_SERIAL_CHANNEL_ID_5=> ID_5
//       <BD_SERIAL_CHANNEL_ID_6=> ID_6
//       <BD_SERIAL_CHANNEL_ID_7=> ID_7
//       <BD_SERIAL_CHANNEL_ID_8=> ID_8
//       <BD_SERIAL_CHANNEL_ID_9=> ID_9
//    <i> Select one of serial port id for debug output
//    <i> Default: BD_SERIAL_CHANNEL_ID_0
//   </e>
// </h> End of ABOV Target Debug Configuration
#define TARGET_SERIAL_CHANNEL_ID BD_SERIAL_CHANNEL_ID_1 /* Don't edit this value */
```

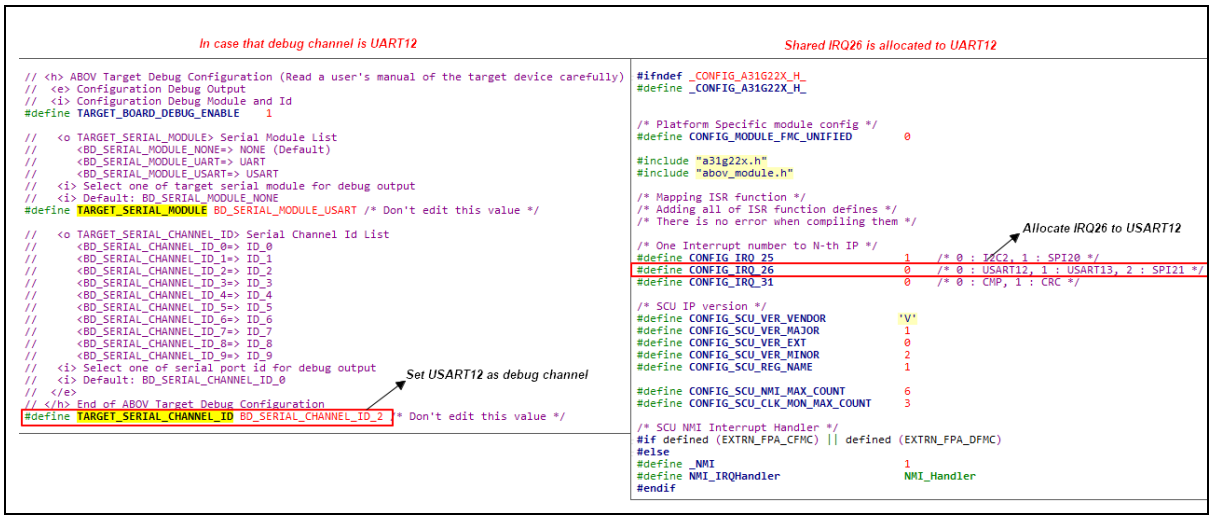
8.2.1 Shared interrupt vector of serial debug channels

Some ABOV devices are designed to share IRQ with their on-chip peripherals because of minimal resource, cost or whatever.
For more information about the shared IRQ, it is strongly recommended that you need to be familiarized with the user's manual of ABOV devices.

Let's take an example, IRQ26 is shared with USART12, USART13 and SPI21 on A31G22x.
If you decide to use UART12 as a debug channel then you have no choice but to allocate IRQ26 to USART12.

Figure 75 shows two steps to config the debug channel to USART12.
At first step, set UART12 as a debug channel to use in 'abov_config.h' and at last step, allocate IRQ26 to UART12 in 'config_a31g22x.h'

Figure 75. To edit serial debug configuration file when sharing interrupt vector




```
// <h> ABOV Target Debug Configuration (Read a user's manual of the target device carefully)
//   <e> Configuration Debug Output
//   <i> Configuration Debug Module and Id
#define TARGET_BOARD_DEBUG_ENABLE    1

//    <o TARGET_SERIAL_MODULE> Serial Module List
//       <BD_SERIAL_MODULE_NONE=> NONE (Default)
//       <BD_SERIAL_MODULE_UART=> UART
//       <BD_SERIAL_MODULE_USART=> USART
//    <i> Select one of target serial module for debug output
//    <i> Default: BD_SERIAL_MODULE_NONE
#define TARGET_SERIAL_MODULE BD_SERIAL_MODULE_USART /* Don't edit this value */

//    <o TARGET_SERIAL_CHANNEL_ID> Serial Channel Id List
//       <BD_SERIAL_CHANNEL_ID_0=> ID_0
//       <BD_SERIAL_CHANNEL_ID_1=> ID_1
//       <BD_SERIAL_CHANNEL_ID_2=> ID_2
//       <BD_SERIAL_CHANNEL_ID_3=> ID_3
//       <BD_SERIAL_CHANNEL_ID_4=> ID_4
//       <BD_SERIAL_CHANNEL_ID_5=> ID_5
//       <BD_SERIAL_CHANNEL_ID_6=> ID_6
//       <BD_SERIAL_CHANNEL_ID_7=> ID_7
//       <BD_SERIAL_CHANNEL_ID_8=> ID_8
//       <BD_SERIAL_CHANNEL_ID_9=> ID_9
//    <i> Select one of serial port id for debug output
//    <i> Default: BD_SERIAL_CHANNEL_ID_0
//   </e>
// </h> End of ABOV Target Debug Configuration
#define TARGET_SERIAL_CHANNEL_ID BD_SERIAL_CHANNEL_ID_2 /* Don't edit this value */
```

```
#ifndef _CONFIG_A31G22X_H_
#define _CONFIG_A31G22X_H_


/* Platform Specific module config */
#define CONFIG_MODULE_FMC_UNIFIED       0

#include "a31g22x.h"
#include "abov_module.h"

/* Mapping ISR function */
/* Adding all of ISR function defines */
/* There is no error when compiling them */

/* One Interrupt number to N-th IP */
#define CONFIG_IRQ_25                   1    /* 0 : I2C2, 1 : SPI20 */
#define CONFIG_IRQ_26                   0    /* 0 : USART12, 1 : USART13, 2 : SPI21 */
#define CONFIG_IRQ_31                   0    /* 0 : CMP, 1 : CRC */

/* SCU IP version */
#define CONFIG_SCU_VER_VENDOR           'V'
#define CONFIG_SCU_VER_MAJOR            1
#define CONFIG_SCU_VER_EXT              0
#define CONFIG_SCU_VER_MINOR            2
#define CONFIG_SCU_REG_NAME             1

#define CONFIG_SCU_NMI_MAX_COUNT        6
#define CONFIG_SCU_CLK_MON_MAX_COUNT    3

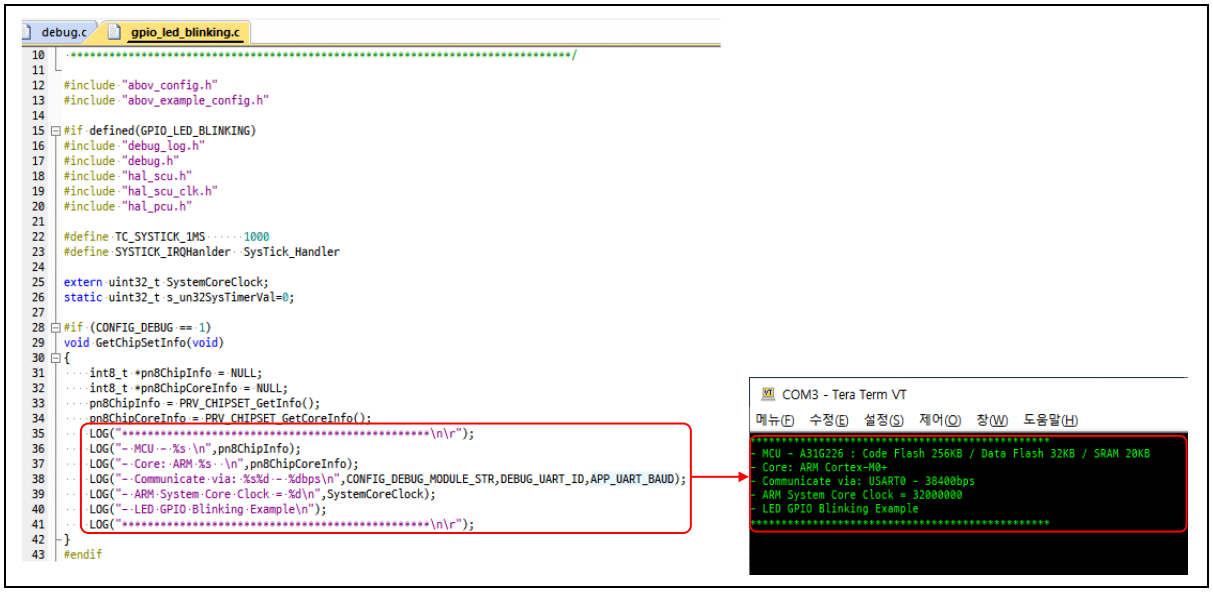
/* SCU NMI Interrupt Handler */
#if defined (EXTRN_FPA_CFMC) || defined (EXTRN_FPA_DFMC)
#else
#define _NMI                            1
#define NMI_IRQHandler                  NMI_Handler
#endif
```

8.3 Debug message in terminal

Debug log messages in an example code can be output to a terminal, using the LOG macro function defined in the ‘debug_log.h’ file.

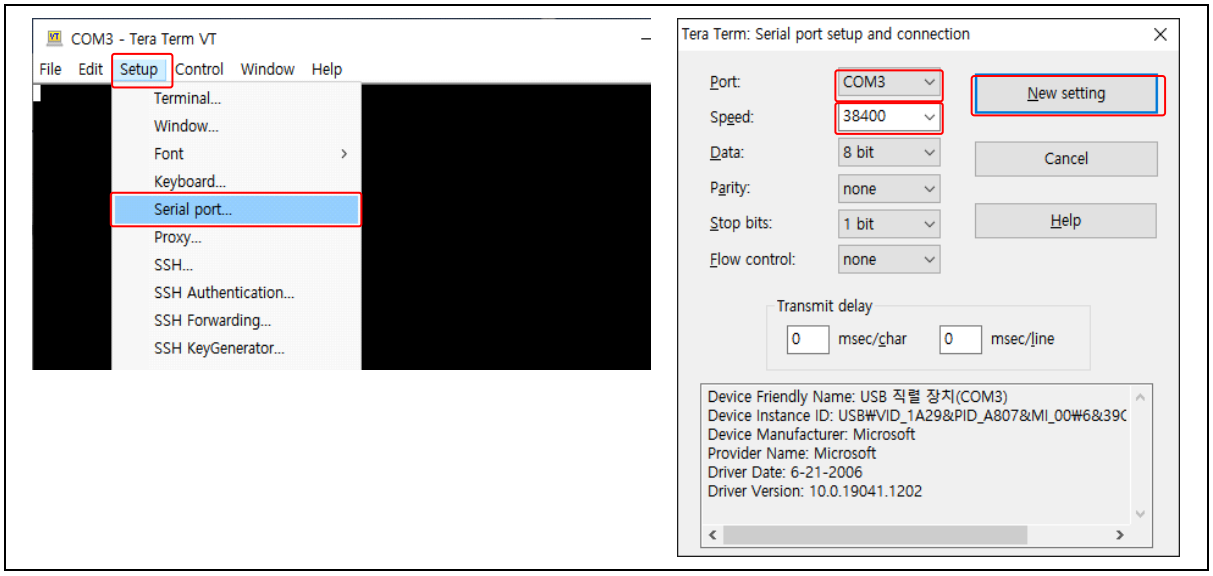
Figure 76 shows that the ‘GetChipSetInfo()’ function in the example code below uses the LOG macros to output the example description to a terminal.

Figure 76. Result of AUDK32 debug log API



If you set up a PC terminal program through the Serial Port with a Baud Rate defined in the example code, a log message is displayed that is output to the terminal.

Figure 77. Serial port configuration in terminal program

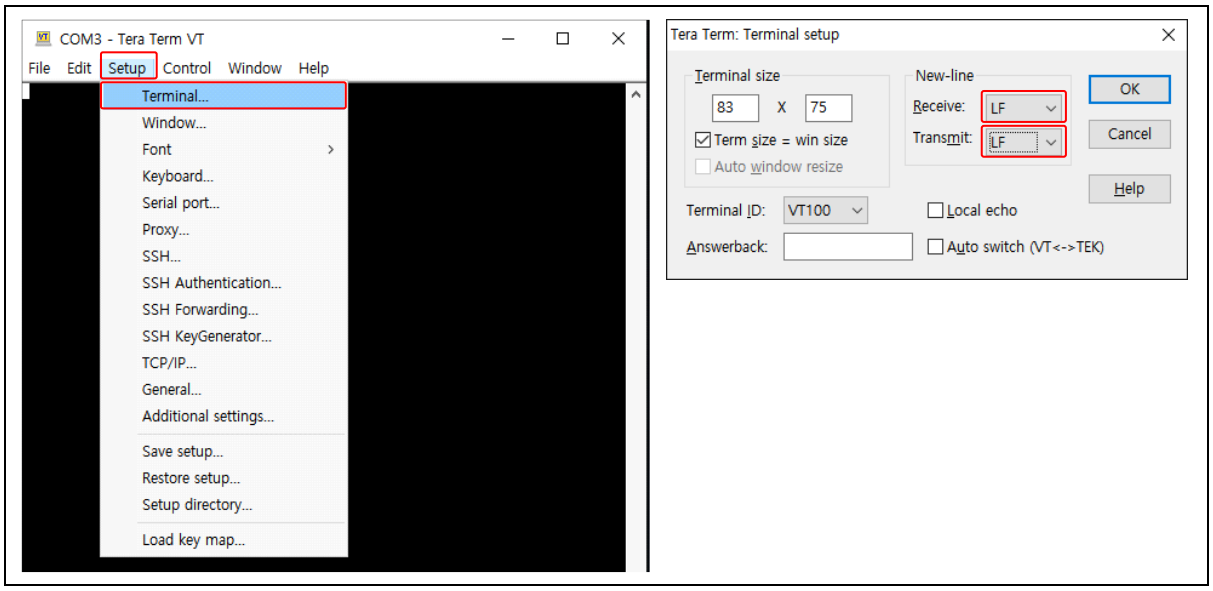


8.4 Setup the new line in terminal

AUDK32 log facility supports LF in Unix/Linux. The Receive and Transmit of the New-Line option must be set to Line Feed (LF) as shown in Figure 78, so that the log transmission works properly.

Since the Transmit option for New-Line may not be supported depending on TeraTerm version, it is recommended to use the latest version of TeraTerm.

Figure 78. Configuration of new line in PC terminal program



9. Appendix

9.1 Eclipse IDE for Embedded C/C++ Developers

In order to use AUDK32 in Eclipse IDE environment, there are something to consider comparing other IDE environments. You can refer this chapter and complete the preparation to use eclipse IDE.

9.1.1 Download program and plug-in

Eclipse is very flexible IDE as Application Framework for developing software, but it alone is not sufficient for developing software.
As a result, it requires cross-toolchains, additional plug-ins, and specific packages depending on the type of software users are developing and the type of host PC they are using.

Here are necessary components to use AUDK32 in eclipse.

9.1.1.1 Eclipse IDE for Embedded C/C++ Developers

There are many types of eclipse IDEs but 'Eclipse IDE for Embedded C/C++ Developers' should be taken and for now, the latest version is '2023-09'.
It can be downloaded at https://www.eclipse.org/downloads/ which will detect host OS and show a correct version.

'Eclipse IDE for Embedded C/C++ Developers' should be selected during the installation process.

9.1.1.2 GNU ARM cross toolchain

ARM cross toolchain must be installed because Eclipse doesn't contain cross toolchain by default.
It can be downloaded at https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads which doesn't detect host OS and just show list of deliverables. Therefore, you must select the correct version for your host PC.
The latest version is 13.2 Rel1 at this moment.
Just in case of Windows 64bit system, 'arm-gnu-toolchain-13.2.rel1-mingw-w64-i686-arm-none-eabi.exe' should be downloaded.

9.1.1.3 CMSIS Pack plug-in

CMSIS is an opensource solution to develop software in ARM platform and must be embedded to eclipse.
It can be downloaded at https://github.com/ARM-software/cmsis-pack-eclipse/releases/download and the latest version is 2.9.0 at this moment.

9.1.1.4 xPack Windows build tools

The xPack Windows Build Tools is a Windows specific package, customised for the requirements of the Eclipse CDT managed build projects. It includes a recent version of GNU make and a recent version of BusyBox, which provides a convenient implementation for sh/rm/echo.

It can be downloaded at https://github.com/xpack-dev-tools/windows-build-tools-xpack/releases/ and

the latest version is 4.4.1-1 and for Windows 64bit system, ‘xpack-windows-build-tools-4.4.1-1-win32-x64.zip’ should be downloaded.

9.1.2 Installation

This document describes an installation process just in case of Windows 64bit PC only.

9.1.2.1 Eclipse IDE for Embedded C/C++ Developers

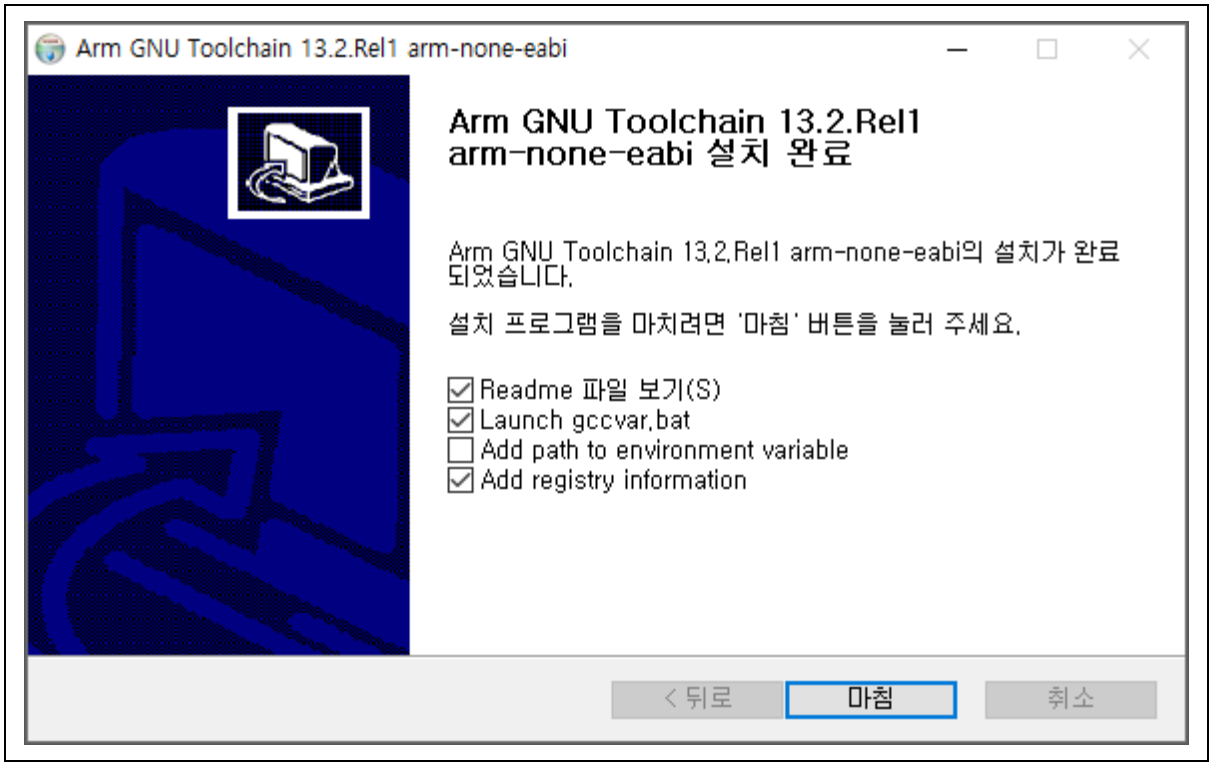
If you click ‘eclipse-inst-jre-win64.exe’ then installation process will be started.
You will be asked to select which IDE to be installed during the process.
Be sure, you must select ‘Eclipse IDE for Embedded C/C++ Developers’ and it will continue to install JRE just after completion of Eclipse IDE installation.

9.1.2.2 GNU ARM cross toolchain

More GNU cross toolchain will be developed and deployed in the future.
So, for your convenient management of the GNU stuff, create ‘C:\GNU\toolchains’ directory to hold many versions of the toolchains and then click ‘arm-gnu-toolchain-13.2.rel1-mingw-w64-i686-arm-none-eabi.exe’.

The installation process will start and ask you to select the installation directory of the toolchain.
‘C:\GNU\toolchains’ directory is preferred for better toolchain management described above.
At the end of the process, you will be asked to check ‘Add path to environment variable’ as below.
Do not check it because it is better approach to manage toolchain by a project base.

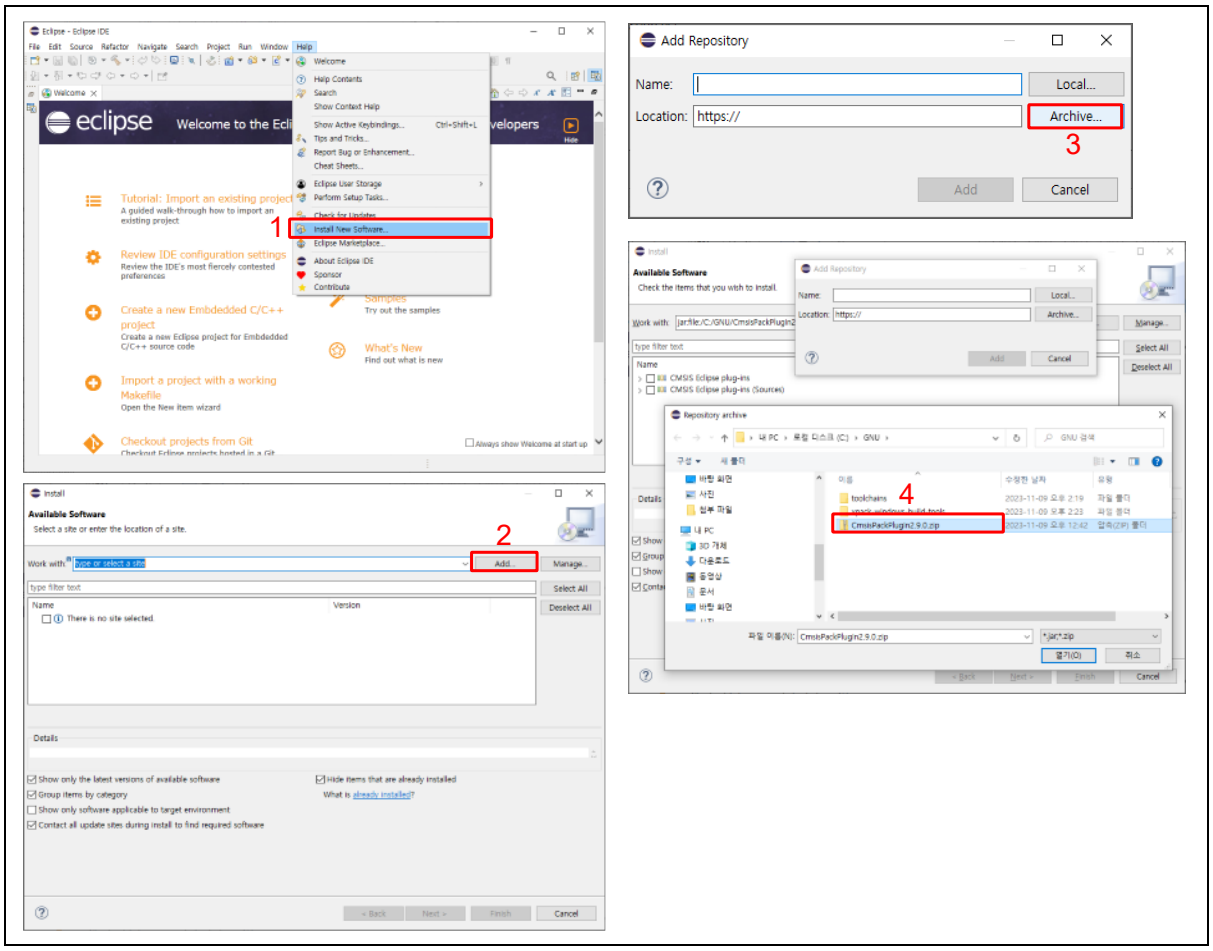
Figure 79. Management of access to GNU toolchain



9.1.2.3 CMSIS Pack plug-in

CMSIS Pack plug-in must be installed into Eclipse IDE and you just need to follow the process below.

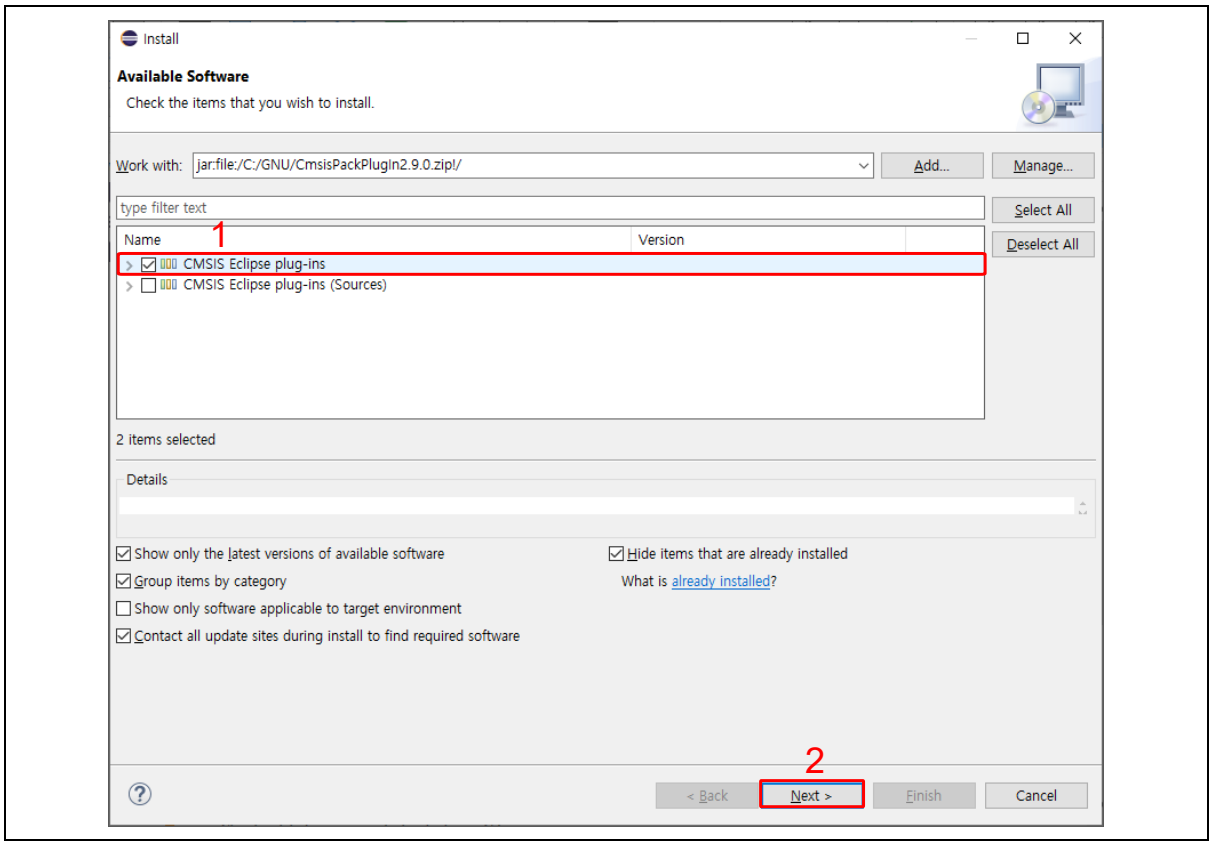
Figure 80. STEP1. Process to register CMSIS Pack plug-in to be installed



There is nothing special in the steps on Figure 80 and it can be described simply as below.

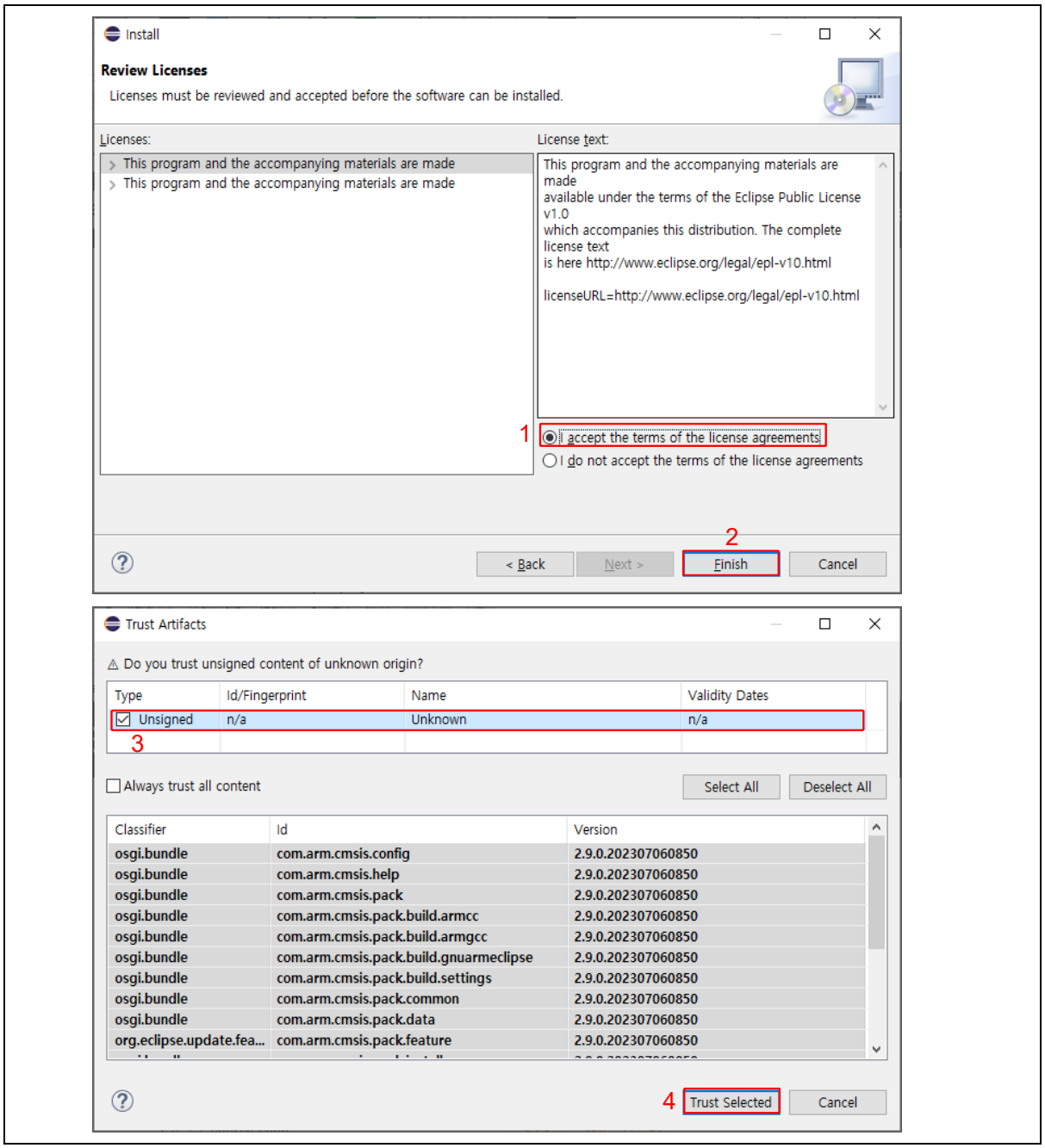
- Help Menu → Install New Software → 'Available Software' Dialog comes up.
- Add → Archive in the Dialog → File Browser comes up.
- Select zip file of CMSIS Pack plug-in downloaded before.

Figure 81. STEP2. Select target plug-in to install



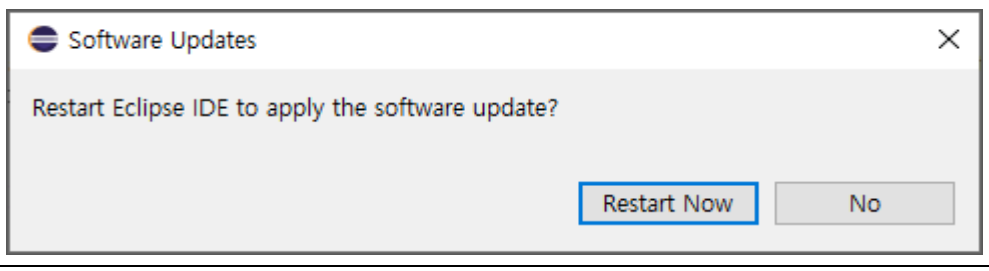
Once STEP1 is completed, you will see plug-ins and the target plug-in must be selected to install.
Normally source code is not essential to develop software.
Therefore check ‘CMSIS Eclipse plug-ins’ only.

Figure 82. STEP3. Complete license agreement and trust artifacts



All steps in above have been completed correctly, you will be asked to restart Eclipse. Restart it.

Figure 83. STEP4. Restart Eclipse IDE



9.1.2.4 xPack Windows build tools

In general GNU world, host needs root file system to develop software running on a cross platform.
xPack Windows Build Tools provides minimum facilities for the root file system.
For example, busybox.exe.

Therefore, you just need to extract zip file of ‘xPack Windows Build Tools’ into a certain meaningful directory such as C:\GNU.

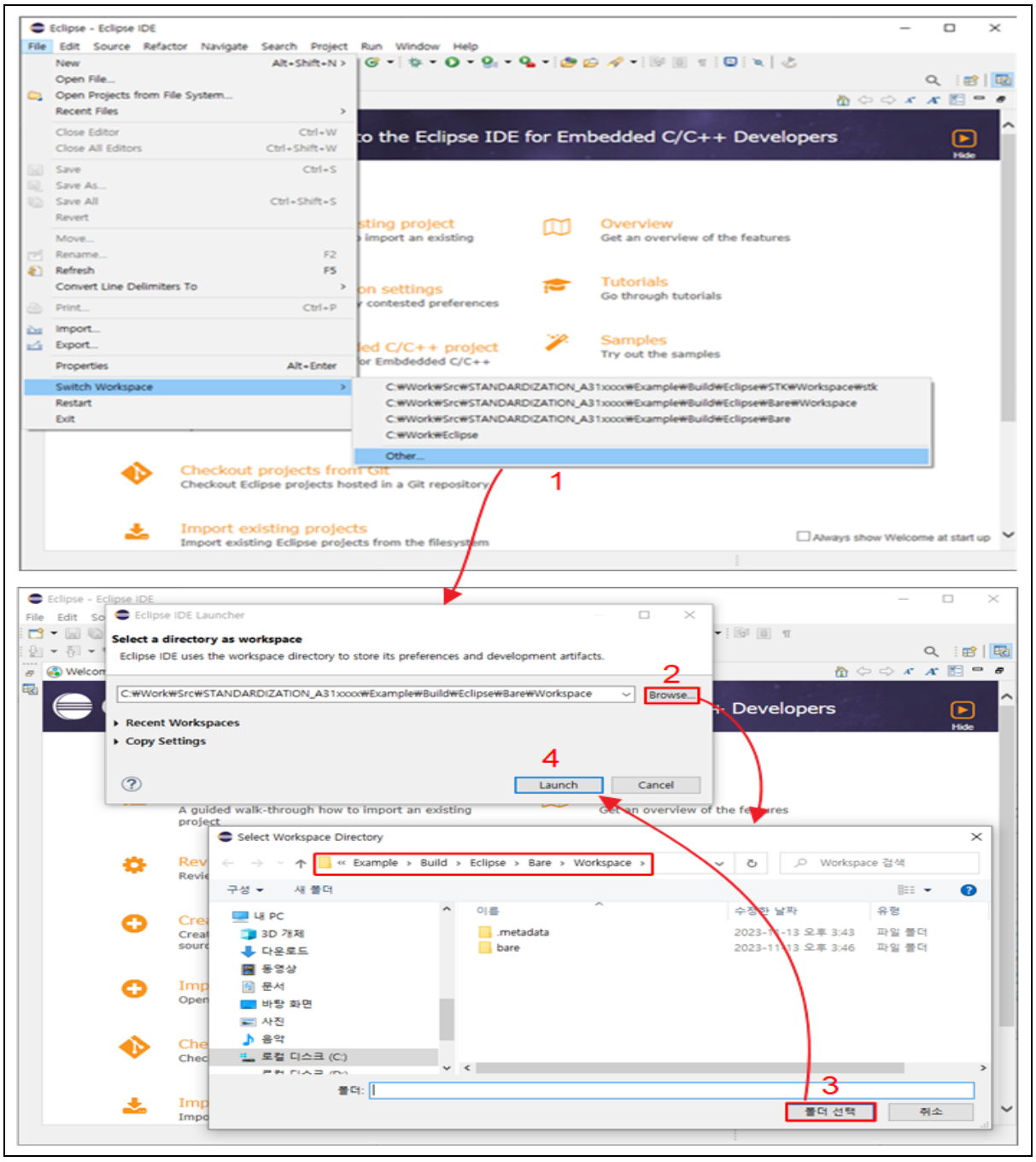
9.1.3 Setup workspace

The several steps are needed to build software and debug it in Eclipse IDE environment.
This section describes it step by step. So, you need to follow all steps carefully.

9.1.3.1 Open workspace

First of all, a workspace has to be registered to Eclipse IDE and then a target project to compile or debug also should be registered later.
If you click ‘Switch Workspace > Other’ and then select the workspace directory you want.

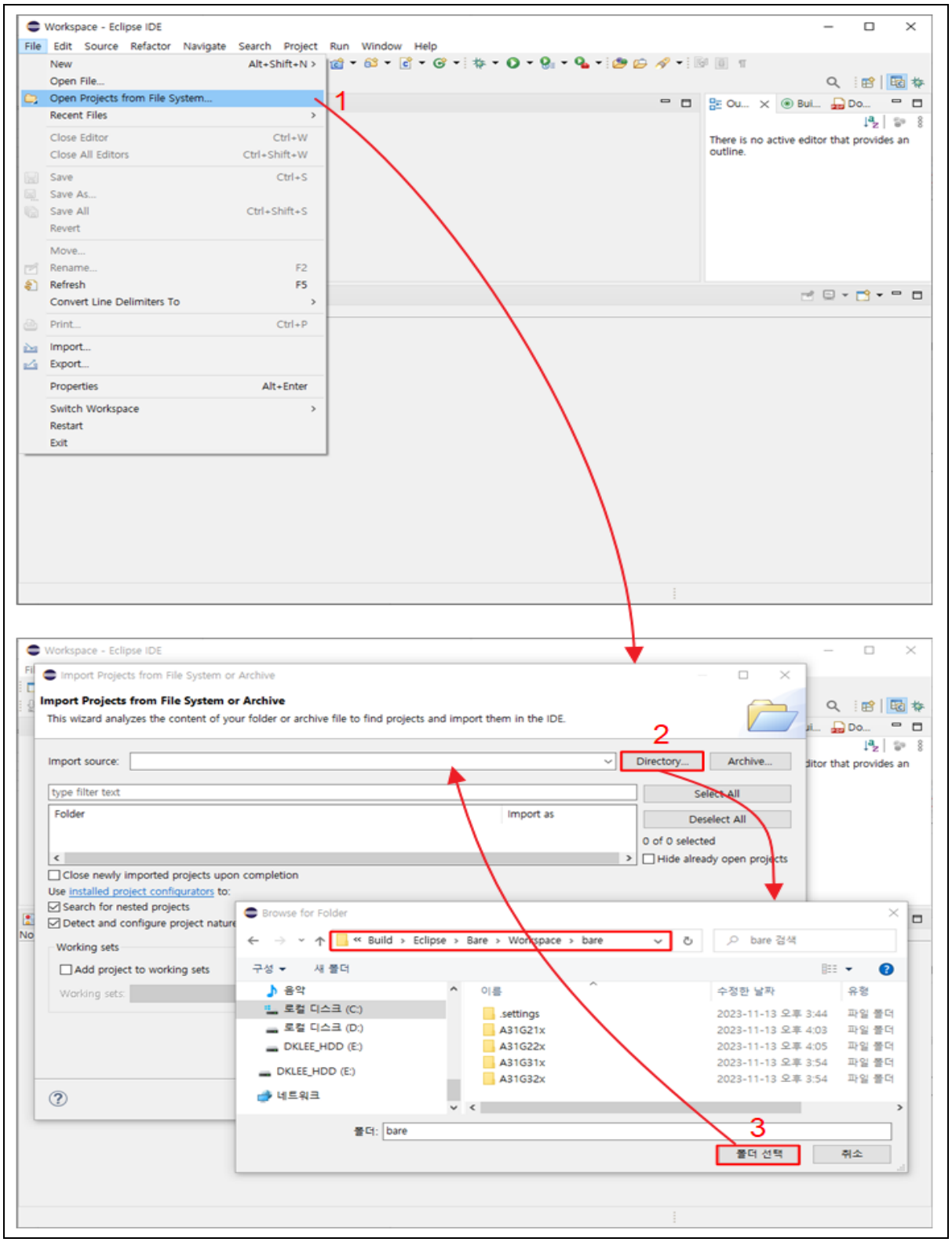
Figure 84. STEP1. Run Eclipse and register workspace



9.1.3.2 Select project

Once a workspace directory is registered, a project directory must be registered accordingly.

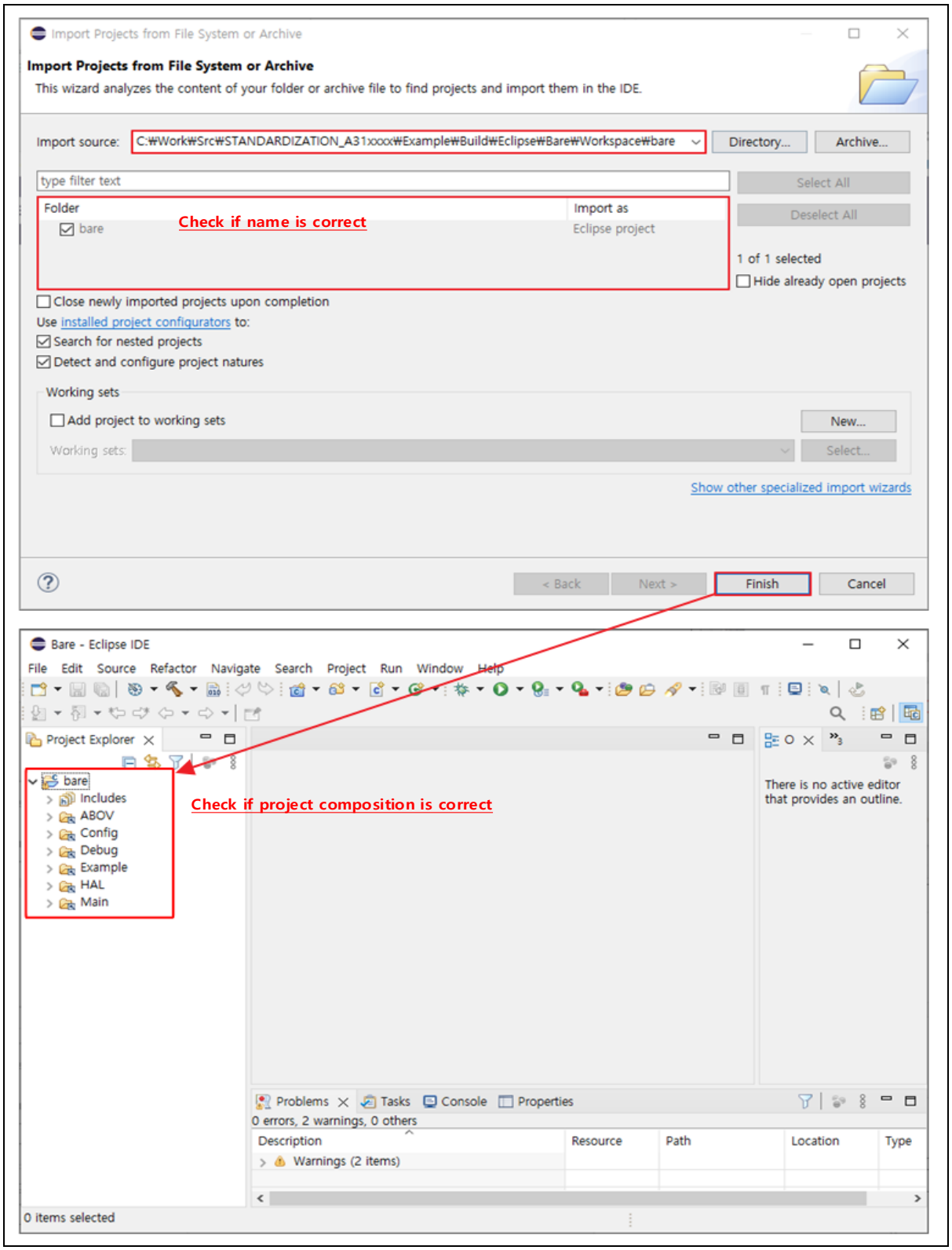
Figure 85. STEP2. Register project directory



9.1.3.3 Check project

If the name of the project is correct, click ‘Finish’ button and then you can find the project composition.

Figure 86. STEP3. Check project



9.1.3.4 Toolchain configuration

Toolchain is configured via ‘Property’ dialog that can be opened by the procedure in Figure 87.

Figure 87. STEP4. Open Property dialog

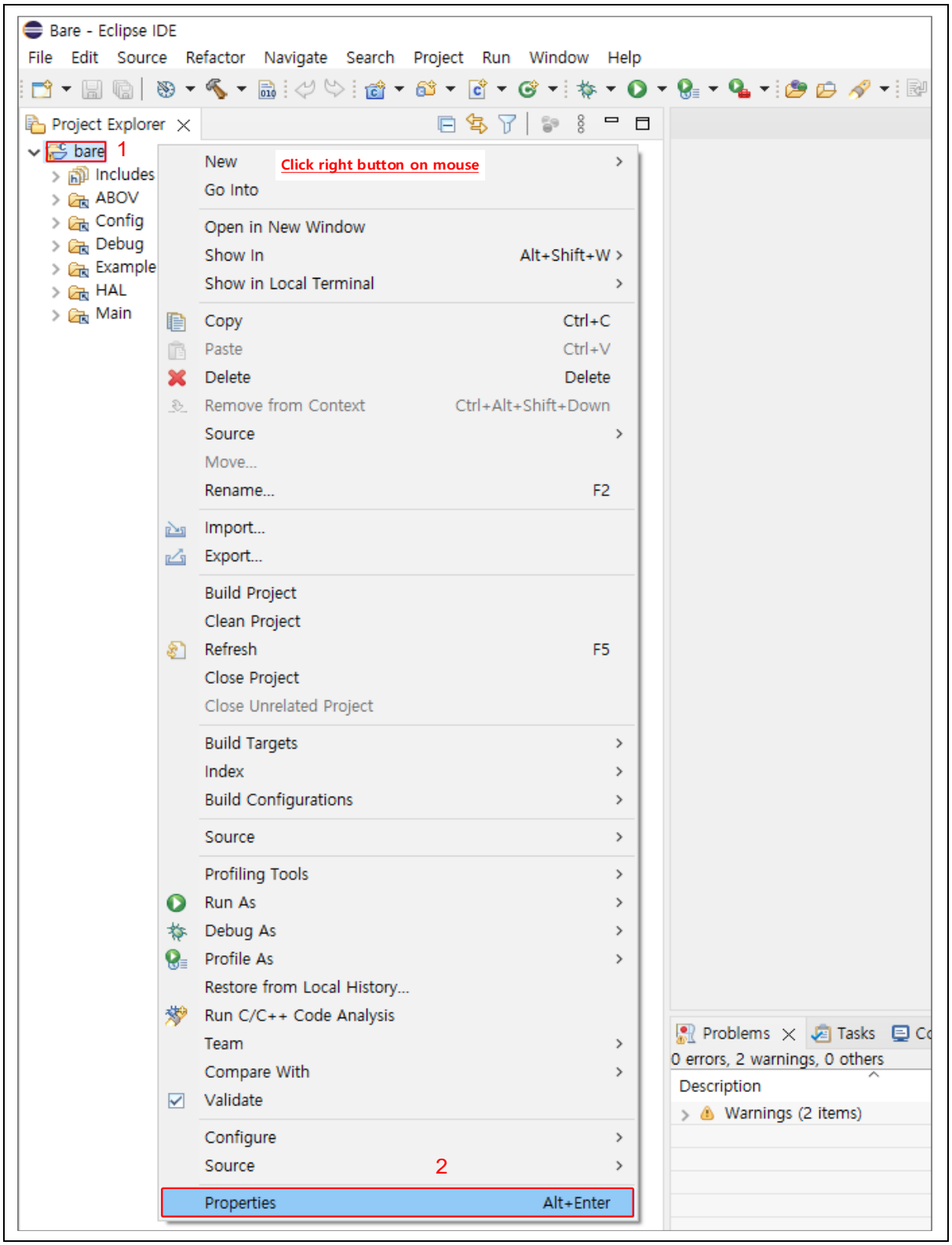
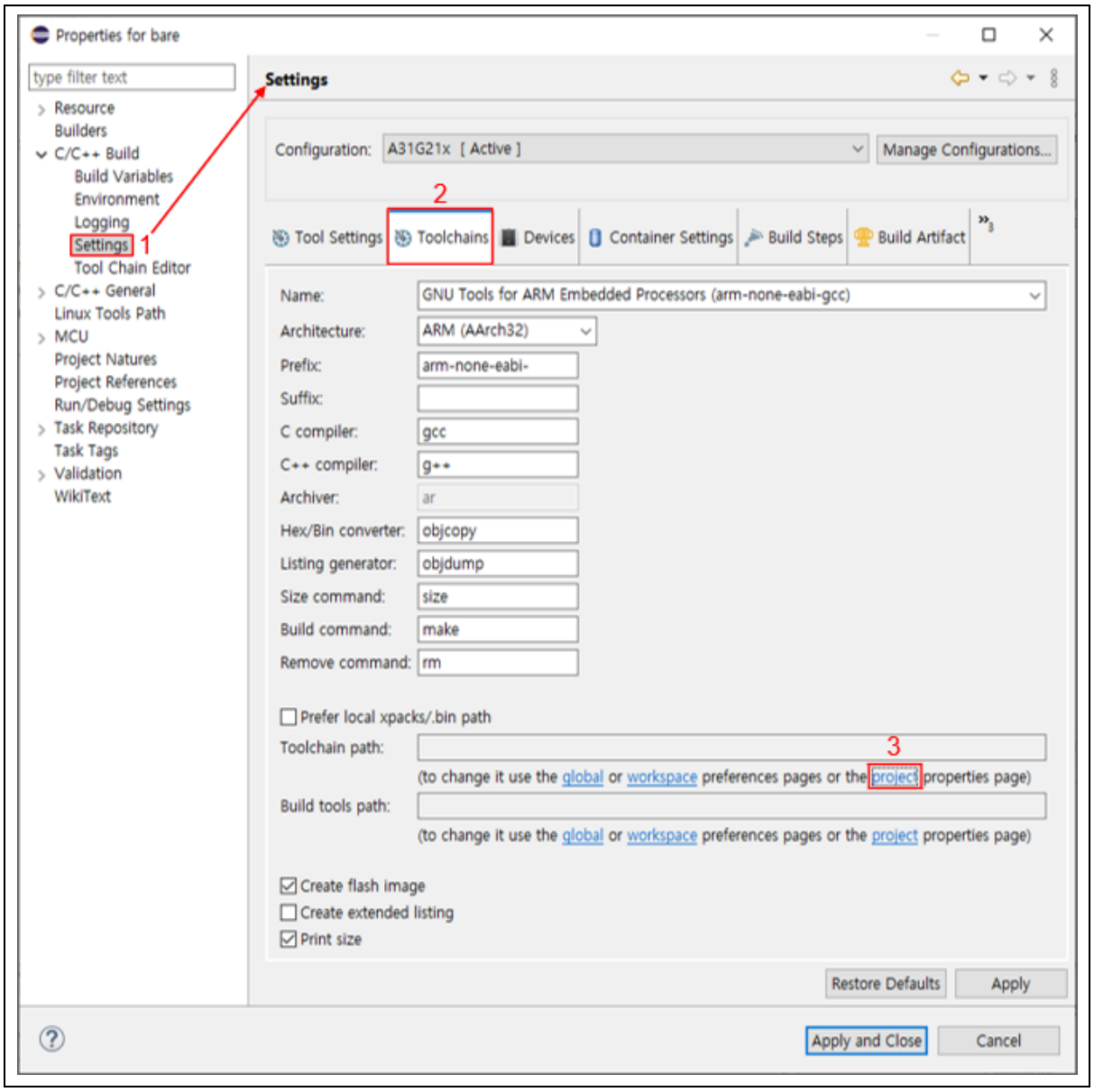


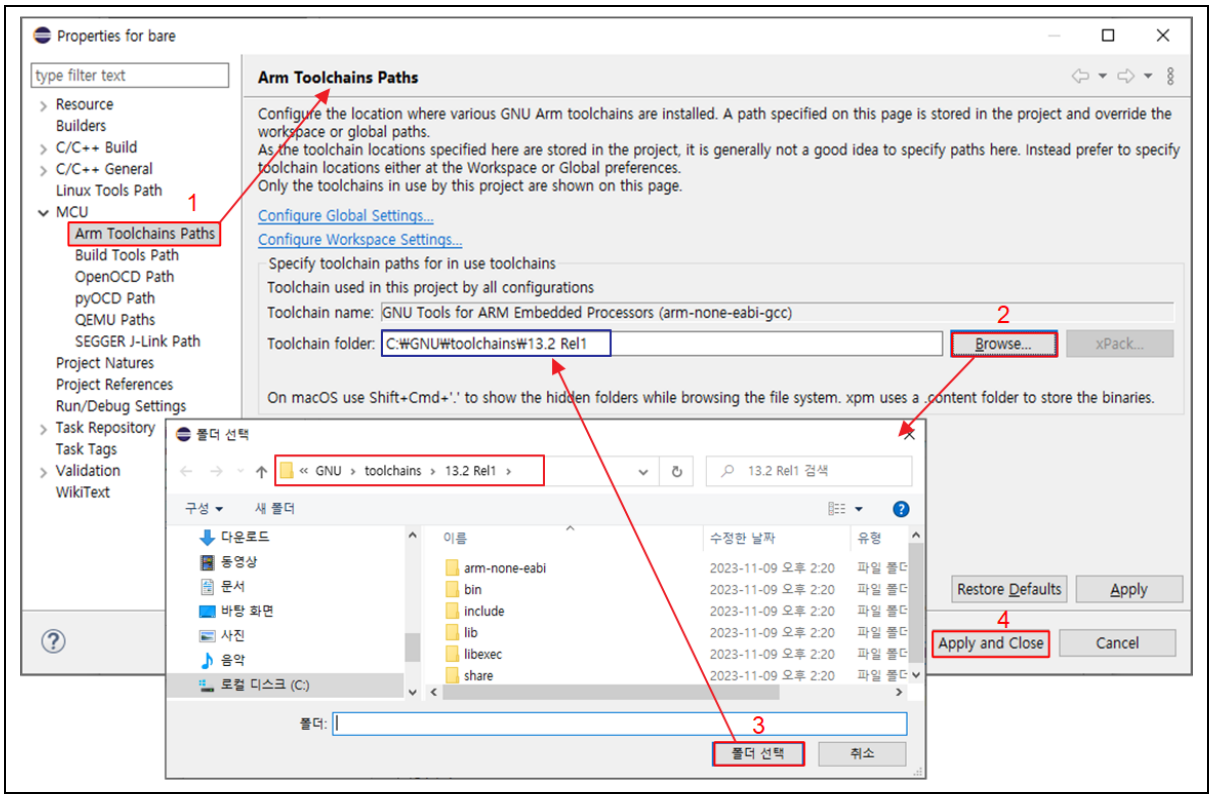
Figure 88. STEP5. Configure toolchain path within project scope



Once ‘Property’ dialog shows up, there are items to configure in a left panel including ‘C/C++ Build’.
If you click ‘settings’ in a left panel and ‘Toolchain’ tab in a right panel, you can find out three scope of registering the toolchain path which was installed before in ‘GNU ARM cross toolchain’ chapter.
The toolchain path can effect on Eclipse IDE with three scopes (global, workspace or project).
Project scope is preferred because it is limited to a working project only.

Therefore, if you click ‘project’, ‘Project Property’ window comes up.
The Project Property window is composed of items to configure in a left panel and the corresponding configuration page in a right panel.
In order to add the toolchain path, click ‘MCU/Arm Toolchains Paths’ in the left panel and browse the location of your toolchain. For more the steps details, refer to Figure 89.

Figure 89. STEP6. Add toolchain path in Project Property window

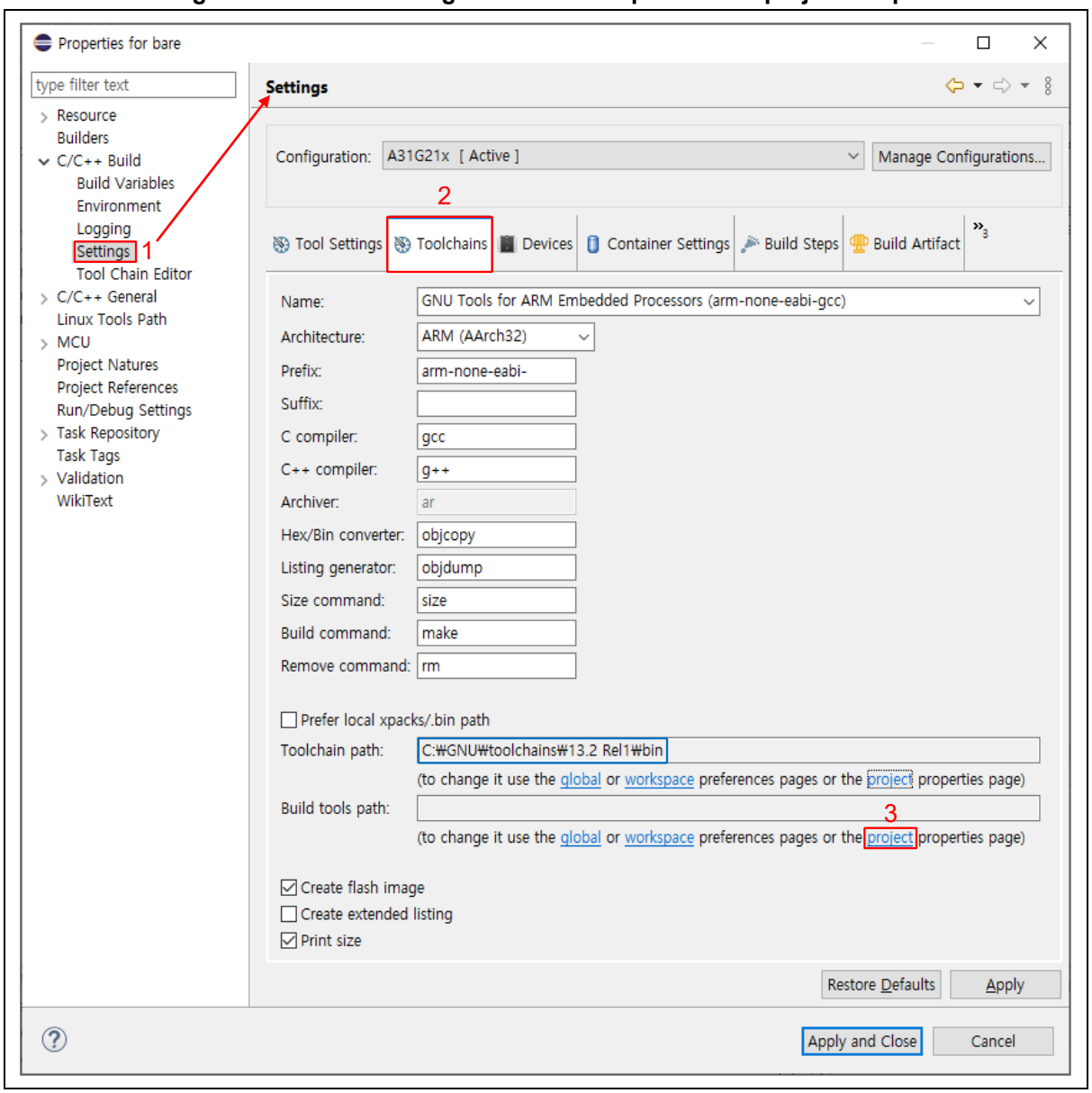


9.1.3.5 xPack Windows build tools configuration

The most important tool in ‘xPack Windows build tools’ is ‘make’.
‘make’ does not need to be installed in Linux or OSX host PC because it was installed globally by default. but Windows PC doesn’t have it. So, ‘make’ has to be installed in Windows PC.
For user’s convenience, ‘make’ can be installed locally to eclipse IDE only if ‘xPack Windows build tools’ is registered to eclipse IDE.

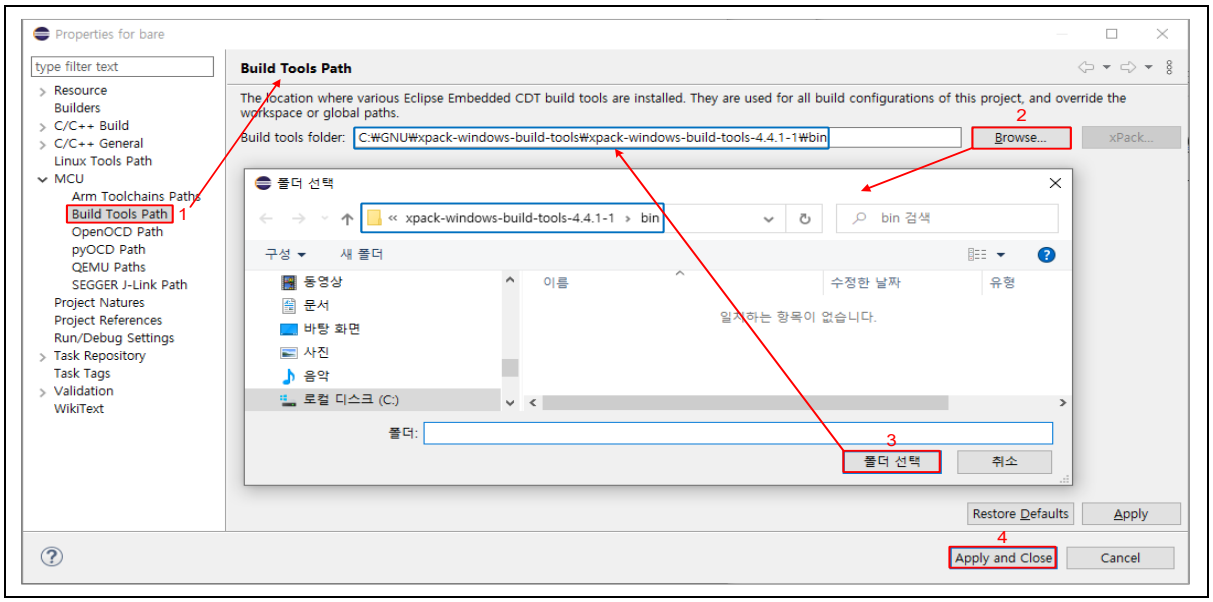
Once Toolchain path is configured completely in ‘Project Property’ window before, ‘xPack Windows build tools’ needs to be installed via ‘Build tools path’ accordingly.
The required action from user is the same as configuring toolchain path.

Figure 90. STEP7. Configure Build tools path within project scope



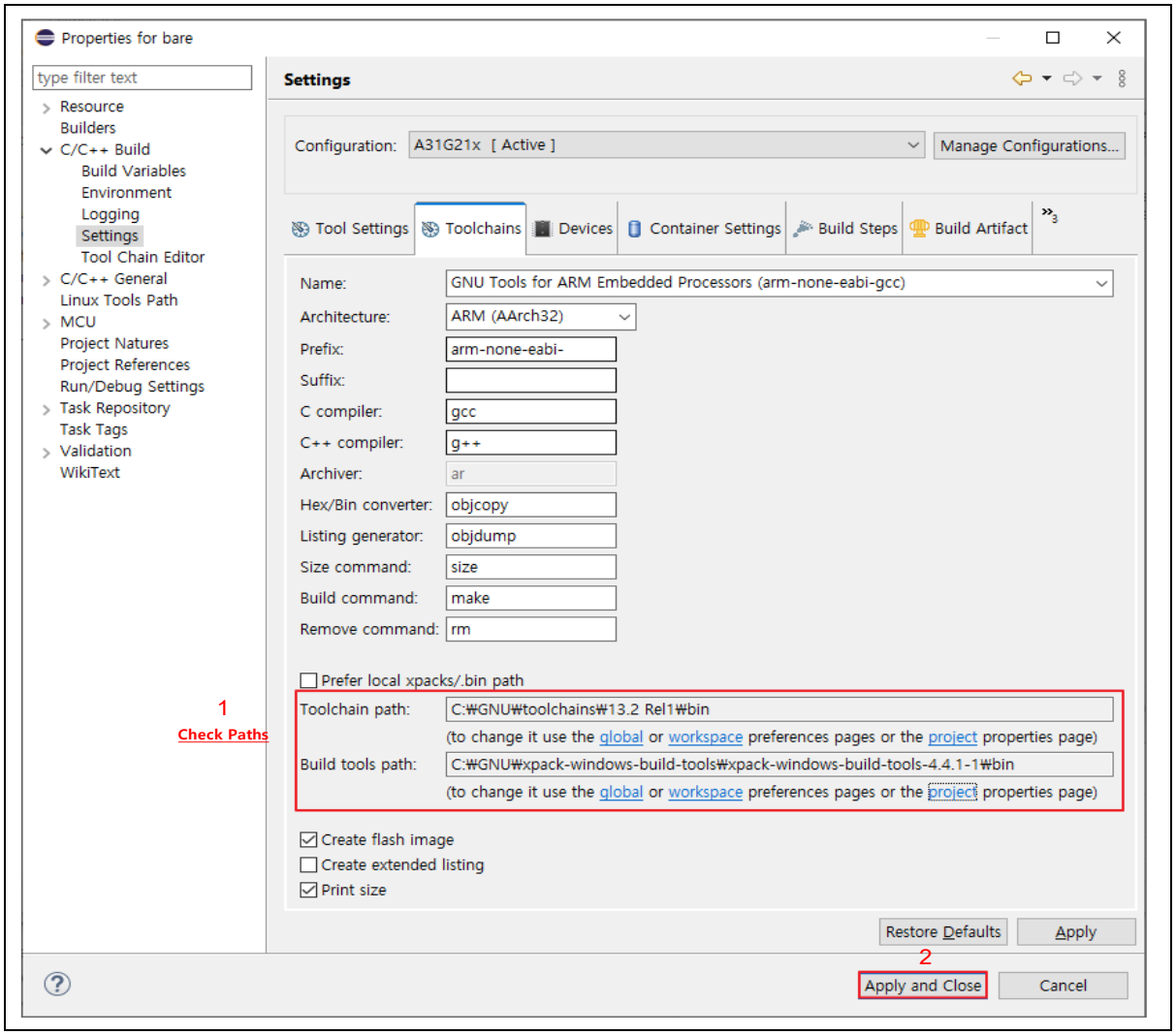
If you click ‘project’, ‘Build Tools Path’ window comes up and you can set the path of ‘xPack Windows build tools’ by browsing the location where it was extracted before as Figure 91.

Figure 91. STEP8. Add ‘xPack Windows build tools’ path in ‘Build Tools Path’ window



Now check all paths you set and then click ‘Appy and Close’ button as Figure 92.

Figure 92. STEP9. Check path of Tool chain and ‘xPack Windows build tools’



Revision history

Revision	Date	Notes
1.00	Sep. 30, 2022	First release.
1.01	May. 12, 2023	• Fix typo. • Replace incorrect figures by new one. • Correct external link. • Add Scatter file table. • Fix misaligned document style.
1.10	Nov. 14, 2023	• Fix typo and incorrect words. • Add appendix about ‘Eclipse IDE for Embedded C/C++ Developer’. • Add AUDK32 in Eclipse Embedded C/C++.

Korea

Regional Office, Seoul
R&D, Marketing & Sales
8th Fl., 330, Yeongdong-daero,
Gangnam-gu, Seoul,
06177, Korea

Tel: +82-2-2193-2200
Fax: +82-2-508-6903
www.abovsemi.com

HQ, Ochang
R&D, QA, and Test Center
93, Gangni 1-gil, Ochang-eup,
Cheongwon-gun,
Chungcheongbuk-do,28126, Korea

Tel: +82-43-219-5200
Fax: +82-43-217-3534
www.abovsemi.com

Domestic Sales Manager
Tel: +82-2-2193-2206
Fax: +82-2-508-6903
Email: sales_kr@abov.co.kr

Global Sales Manager
Tel: +82-2-2193-2281
Fax: +82-2-508-6903
Email: sales_gl@abov.co.kr

China Sales Manager
Tel: +86-755-8287-2205
Fax: +86-755-8287-2204
Email: sales_cn@abov.co.kr

ABOV Disclaimer

IMPORTANT NOTICE – PLEASE READ CAREFULLY

ABOV Semiconductor ("ABOV") reserves the right to make changes, corrections, enhancements, modifications, and improvements to ABOV products and/or to this document at any time without notice. ABOV does not give warranties as to the accuracy or completeness of the information included herein. Purchasers should obtain the latest relevant information of ABOV products before placing orders. Purchasers are entirely responsible for the choice, selection, and use of ABOV products and ABOV assumes no liability for application assistance or the design of purchasers' products. No license, express or implied, to any intellectual property rights is granted by ABOV herein. ABOV disclaims all express and implied warranties and shall not be responsible or liable for any injuries or damages related to use of ABOV products in such unauthorized applications. ABOV and the ABOV logo are trademarks of ABOV. All other product or service names are the property of their respective owners. Information in this document supersedes and replaces the information previously supplied in any former versions of this document.

© 2023 ABOV Semiconductor – All rights reserved